

Universidad de Oviedo



ESCUELA POLITÉCNICA DE INGENIERÍA DE GIJÓN

MÁSTER UNIVERSITARIO EN INGENIERÍA INFORMÁTICA

ÁREA DE ARQUITECTURA Y TECNOLOGÍA DE COMPUTADORES

TRABAJO FIN DE MÁSTER

Sistema de vectorización multimodal para la recuperación semántica de eventos en contenido audiovisual

D. Sergio Botana Suárez

TUTOR: *D. Rubén Usamentiaga Fernández*

COTUTOR: *D. Oscar Díaz Pedrayes*

FECHA: noviembre 2026



Universidad de Oviedo



DECLARACIÓN DE ORIGINALIDAD

D. Sergio Botana Suárez con DNI 53783252Y como alumno del Máster Universitario en Ingeniería Informática de la Universidad de Oviedo, DECLARO que el Trabajo Fin de Máster titulado
Sistema de vectorización multimodal para la recuperación semántica de eventos en contenido audiovisual es de mi autoría, es original y las fuentes bibliográficas utilizadas han sido debidamente citadas.

En Gijón, a 15 de noviembre de 2026

Firmado: Sergio Botana Suárez

Resumen

El crecimiento exponencial de los archivos audiovisuales hace inviable el visionado manual y el etiquetado previo en tareas de búsqueda temporal de información. Los sistemas clásicos de Recuperación Aumentada por Generación (*Retrieval-Augmented Generation*, RAG) aplicados a vídeo fragmentaban la señal mediante ventanas temporales rígidas y traducciones intermedias a texto plano (OCR y ASR), destruyendo la disposición espacial y la sincronía entre imagen y locución. El presente Trabajo Fin de Máster diseña, formaliza e implementa una arquitectura de recuperación omnimodal en dos etapas sin metadatos previos que integra tres motores de vectorización intercambiables bajo el protocolo común `MultimodalVectorizer`: la API en la nube *Gemini Embedding 2* (`3072d`) y dos modelos de pesos abiertos ejecutados íntegramente en local sobre Apple Silicon M3 (`mps`, `bfloat16`), *Qwen3-VL-Embedding-2B* (`2048d`) y *EmbeddingGemma 2* (`768d`).

En la fase de ingesta, el prototipo segmenta el metraje en una única pasada mediante el Índice de Similitud Estructural (SSIM) con cota temporal máxima, extrae tres fotogramas clave de máxima entropía de Shannon por plano y alinea la transcripción fonética local de *Whisper* (`base`, $\Delta_{\text{mín}} = 0,2$ s) mediante fusión temprana. Para escalar el almacenamiento sin reentrenamiento, se aplica truncamiento prefijo guiado por *Matryoshka Representation Learning* (MRL) con renormalización euclídea L_2 (`mrl.py`) sobre colecciones aisladas en *Qdrant* (*HNSW*). En la segunda etapa, un árbitro bimodal basado en *Gemini 3.5 Flash Lite* evalúa de forma disjunta la evidencia visual y auditiva para reordenar los candidatos y descartar falsos positivos.

La validación empírica sobre 164 fragmentos y 138 consultas anotadas en dos dominios contrastados (`python_tutorial.mp4` con locución sincronizada y `animals.mp4` con *B-roll* visual desacoplado) demuestra que: (i) la fusión temprana audio-visual eleva el *Recall@1* en Etapa 1 del $67,9\,\%$ al $85,7\,\%$ en metraje técnico ($p_{\text{McNemar}} = 0,0020$), mientras el árbitro bimodal alcanza el $91,2\,\%$ global de *Recall@1* (MRR $= 0,9289$, $p_{\text{McNemar}} = 0,0018$), logra $96,6\,\%$ de precisión multi-objetivo documental y rechaza el $100,0\,\%$ de falsos positivos en control negativo ($0/240$); (ii) la compresión MRL en *Gemini* (3072d $\rightarrow$ 768d) retiene el $98,8\,\%$ del *Recall@1* ($p = 1,0000$), reduce un $75,0\,\%$ la memoria y acelera la búsqueda en *Qdrant* un $44,4\,\%$; y (iii) en las 102 consultas literales globales de Etapa 1 no existen diferencias estadísticamente significativas entre *Gemini* ($79,4\,\%$, MRR $= 0,8688$) y los modelos locales *Qwen3-VL-2B* ($76,5\,\%$, $p_{\text{Wilcoxon}} = 0,2915$) o *EmbeddingGemma 2* ($74,5\,\%$, $p_{\text{Wilcoxon}} = 0,2333$), destacando *Qwen3-VL-2B* en *B-roll* visual ($78,26\,\%$ *Recall@1*) e indexación (126–142 s) y *EmbeddingGemma 2* en latencia de consulta (~ 30 ms, 15–16$\times$ más rápido) y consumo Metal ($2,55$–$2,59$ GB).

Palabras clave: *Video-RAG*, *embeddings* multimodales, *Matryoshka Representation Learning*, fusión temprana audio-visual, reordenamiento bimodal, modelos de pesos abiertos, *Qdrant*.

Abstract

The exponential growth of audiovisual repositories renders manual review and prior metadata annotation unfeasible for temporal event search. Traditional Video Retrieval-Augmented Generation (Video-RAG) pipelines segmented footage into rigid fixed-time windows and forced visual content into plain text via OCR and ASR, discarding spatial layout and audio-visual synchrony. This Master's Thesis designs, formalizes, and implements a metadata-free, two-stage omnimodal retrieval architecture integrating three interchangeable embedding engines behind a unified `MultimodalVectorizer` protocol: the cloud API *Gemini Embedding 2* (`3072d`) and two open-weight models running natively on Apple Silicon M3 (`mps`, `bfloat16`), namely *Qwen3-VL-Embedding-2B* (`2048d`) and *EmbeddingGemma 2* (`768d`).

During ingestion, the prototype performs single-pass adaptive shot boundary segmentation using the Structural Similarity Index Measure (SSIM) with a maximum duration bound, extracts three maximum Shannon-entropy keyframes per shot, and aligns local *Whisper* (`base`, $\Delta_{\text{mín}} = 0{,}2$ s) speech transcripts via early fusion. To scale vector storage without retraining, prefix truncation via Matryoshka Representation Learning (MRL) with explicit L_2 renormalization (`mrl.py`) is applied across isolated *Qdrant* (*HNSW*) collections. In the second stage, a bimodal judge powered by *Gemini 3.5 Flash Lite* disjunctively verifies visual and spoken evidence to rerank candidates and filter out false positives.

Empirical evaluation across 164 video chunks and 138 annotated queries spanning two contrasting domains (`python_tutorial.mp4` with synchronized speech and `animals.mp4` with decoupled visual B-roll) confirms that: (i) audio-visual early fusion boosts Stage 1 *Recall@1* from 67,9 % to 85,7 % on technical speech ($p_{\text{McNemar}} = 0{,}0020$), while the bimodal judge achieves 91,2 % global *Recall@1* (MRR = 0,9289, $p_{\text{McNemar}} = 0{,}0018$), 96,6 % documentary multi-target precision, and 100,0 % negative-query false-positive rejection (0/240); (ii) MRL truncation on *Gemini* (3072d $\rightarrow$ 768d) retains 98,8 % relative *Recall@1* ($p = 1{,}0000$) while cutting vector memory by 75,0 % and *Qdrant* latency by 44,4 %; and (iii) across all 102 Stage 1 literal queries globally, local open-weight engines achieve statistical parity with *Gemini* (79,4 %, MRR = 0,8688), with *Qwen3-VL-2B* (76,5 %, $p_{\text{Wilcoxon}} = 0{,}2915$) leading visual B-roll accuracy (78,26 % *Recall@1*) and indexing speed (126–142 s), and *EmbeddingGemma 2* (74,5 %, $p_{\text{Wilcoxon}} = 0{,}2333$) delivering $\sim$ 30 ms local queries (15–16$\times$ faster than Cloud API) within 2,55–2,59 GB of Metal memory.

Keywords: Video-RAG, multimodal embeddings, Matryoshka Representation Learning, audio-visual early fusion, bimodal reranking, open-weight models, Qdrant.

Índice

Resumen **1**

Abstract **2**

1 Introducción **11**
1.1 Contexto y Motivación . . . 11
1.2 Planteamiento del Problema . . . 11
1.3 Preguntas de Investigación e Hipótesis . . . 11
1.4 Objetivos del Trabajo Fin de Máster . . . 12
1.5 Contribuciones Principales . . . 13
1.6 Estructura del Documento . . . 13

2 Revisión de la Literatura y Marco Teórico **15**
2.1 Fundamentos Matemáticos de los Embeddings Multimodales y Espacios Semánticos Continuos . . . 15
2.1.1 El Problema del Análisis Secuencial y la Representación de Datos No Estructurados . . . 15
2.1.2 El Ocaso de las Canalizaciones OCR y la Ingestión Visual Nativa con M-RoPE 16
2.1.3 Geometría del Espacio Latente y Métricas de Similitud Estadística . . . 17
2.1.4 Aprendizaje de Representación Matryoshka (MRL) y Normalización L_2 . . . 19
2.1.5 Análisis del Compromiso entre Precisión Semántica y Coste de Almacenamiento 20
2.1.6 Aprendizaje Contrastivo (*InfoNCE*) y el Fenómeno de la Brecha Modal (*Modality Gap*) . . . 21
2.2 Funcionamiento y Taxonomía de Modelos de Embeddings Multimodales . . . 22
2.2.1 Evolución Arquitectónica: De los Modelos Alineados (CLIP e ImageBind) a la Omnimodalidad Nativa . . . 22
2.2.2 Tokenización Visual, Codificación M-RoPE y Agregación Temporal de Fotogramas . . . 23
2.2.3 Arquitecturas de Doble Torre (*Bi-Encoder*) para Recuperación Masiva Precomputada . . . 24
2.2.4 Arquitecturas de Torre Única (*Cross-Encoder / Reranker*) y Atención Cruzada . 25
2.2.5 Eficiencia Escalable: El Paradigma de Búsqueda Asimétrica en Dos Etapas (Regla 99 % / 1 %) . . . 26
2.3 Estudio Profundo de Google Gemini Embedding 2 . . . 28
2.3.1 Arquitectura Omnimodal Unificada y Configuración del Espacio de Salida . . 28
2.3.2 Paradigmas de Ingestión Multimedia en el SDK `google-genai`: API Files frente a Agregación en Memoria . . . 29
2.3.3 Modelado Matemático de Cuotas Operativas (RPM, TPM y RPD) y Planificación de Capacidad . . . 30
2.3.4 Ingeniería de Resiliencia: Control de Concurrencia, Ventana de *Backoff* de 60 Segundos y Degradación Elegante . . . 32
2.4 Ecosistema de Modelos Alternativos: Pesos Abiertos (*Open-Weights*) y Ejecución Local 35

2.4.1 Las Dos Macrotendencias de 2026: APIs Propietarias Hiper-escaladas frente a Pesos Abiertos . . . 35
2.4.2 La Familia Qwen3-VL: Recuperación y Reordenación Multimodal a Escala Masiva . . . 35
2.4.3 Adaptación de Modelos Visuales (ColQwen y Sentence Transformers) y Entornos Restringidos (*Multi-RAG*) . . . 37
2.4.4 Evaluación en Bancos de Pruebas Masivos (MMEB-V2 y MVEB) y Compromisos de Despliegue . . . 38
2.5 Sistemas RAG Multimodales (mRAG) y Comprensión de Vídeo de Larga Duración . 41
2.5.1 Arquitecturas de Referencia en Comprensión de Vídeo: *Video-RAG*, *CARVE* y *Multi-RAG* . . . 41
2.5.2 Del Emparejamiento Semántico al Anclaje Causal Espaciotemporal (*Temporal Grounding*) . . . 42
2.5.3 Cuellos de Botella Perceptivos en la Verificación Visual: Resolución Espacial y Compresión JPEG . . . 43
2.5.4 Dilución Semántica en Fragmentos de Larga Duración y Muestreo Estático de Fotogramas . . . 44
2.6 Estrategias Avanzadas de Fragmentación (*Chunking*) y Alineación Temporal . . . 46
2.6.1 Obsolescencia de la Partición Fija, Detección de Planos y Agrupación de Escenas 46
2.6.2 Metodología Híbrida *Video-RAC* y Formulación Matemática del SSIM Gaussiano 47
2.6.3 Selección de Fotogramas Clave mediante Entropía Matemática de Shannon . . 48
2.6.4 Alineación Forzada del Audio (*Forced Alignment*), Silero VAD y Diarización de Hablantes . . . 49
2.6.5 Análisis Comparativo frente a Sistemas de Ingestión de Vídeo en Ventana Fija (*SentrySearch*) . . . 51
2.7 Bases de Datos Vectoriales, Evaluación Empírica y Metodología de Revisión . . . 54
2.7.1 Arquitectura de Bases de Datos Vectoriales e Indexación HNSW . . . 54
2.7.2 Métricas Clásicas de Recuperación y Bancos de Pruebas Masivos (MMEB-V2 y MVEB) . . . 55
2.7.3 Frameworks de Evaluación RAG y Atribución Verificable (RAGAS, ALCE, ARGUE, MIRAGE) . . . 55
2.7.4 Metodología de Revisión Sistemática de la Literatura Asistida por NotebookLM 56

3 Metodología y Propuesta Arquitectónica 58
3.1 Formalización Matemática del Espacio Latente Multimodal y Representaciones Matryoshka . . . 58
3.1.1 Espacio de Ingesta y Proyección Omnimodal . . . 58
3.1.2 Representaciones Anidadas Matryoshka (MRL) . . . 59
3.2 Canalización de Ingesta Adaptativa y Fusión Temprana Audio-Visual . . . 60
3.2.1 Segmentación Temporal Adaptativa mediante Similitud Estructural (SSIM) . 60
3.2.2 Extracción y Alineación Forzada de la Locución Auditiva . . . 60
3.2.3 Fusión Temprana Omnimodal en Solicitud Única . . . 61
3.3 Mecanismo de Recuperación en Dos Etapas y Verificación Bimodal . . . 62

3.3.1 Etapa 1: Recuperación Densa y Deduplicación Temporal 62
3.3.2 Etapa 2: Árbitro Bimodal Multimodal con Gemini 3.5 Flash Lite 62
3.3.3 Discriminación de Falsos Positivos y Ordenación Definitiva 63
3.4 Modelo de Dominio Inmutable y Contratos de Datos para Reproducibilidad 64
3.4.1 Entidades de Dominio Inmutables 64
3.4.2 Trazabilidad Científica Integral mediante *RunAuditor* 65

4 Configuración Experimental y Prototipo 66
4.1 Entorno Experimental, Infraestructura y Gestión de Cuotas 66
4.1.1 Plataforma de Ejecución y Pila Tecnológica 66
4.1.2 Gestión de Cuotas, Concurrencia y Presupuesto de Memoria Unificada 67
4.1.3 Configuración y Aislamiento de Colecciones en Qdrant 67
4.2 Implementación Modular del Prototipo Experimental 68
4.2.1 Módulo de Ingesta, Segmentación y Transcripción 68
4.2.2 Protocolo `MultimodalVectorizer`, Motores Locales y Truncamiento MRL . . 68
4.2.3 Motor de Búsqueda Vectorial Qdrant 69
4.2.4 Árbitro Bimodal Multimodal y Parser Resiliente 69
4.2.5 Fachada de Orquestación, Consultas Multimodales e Interfaz Gráfica 69
4.3 Arnés de Trazabilidad, Observabilidad y Auditoría Experimental 71
4.3.1 Estructura del Registro Experimental 71
4.3.2 Protocolo de Reproducibilidad Desacoplada 71

5 Resultados Experimentales y Discusión Crítica 73
5.1 Banco de Pruebas Experimental, Corpus Audiovisuales y Catálogo de Consultas . . . 73
5.1.1 Caracterización de los Corpus Audiovisuales de Referencia 73
5.1.2 Taxonomía del Catálogo de Consultas Anotadas 73
5.1.3 Métricas Formales de Evaluación 74
5.2 Estudio de Ablación Dimensional con Matryoshka Representation Learning (MRL) . 75
5.2.1 Fundamento Matemático del Truncado Prefijo y Re-Normalización L_2 75
5.2.2 Comportamiento en Consultas Literales de Evento Único 75
5.2.3 Validación Cuantitativa de la Hipótesis 2 (H_2) 77
5.3 Evaluación Empírica de la Integración Audio-Visual y Verificación Bimodal 78
5.3.1 Comparativa de Rendimiento: Línea Base Visual frente a Fusión Bimodal . . . 78
5.3.2 Discusión de Fenómenos Perceptuales y Significación Estadística Pareada . . . 78
5.3.3 Estudio de Casos Críticos: Falsos Negativos, Resiliencia Fonética y Desambiguación Sintáctica 79
5.3.4 Divergencia Audio-Visual en Metraje *B-Roll* y Deduplicación por Transcripción 80
5.3.5 Validación Cuantitativa de la Hipótesis 1 (H_1) 81
5.4 Evaluación en Consultas Complejas: Multi-Objetivo y Control Negativo 82
5.4.1 Consultas Generales y Multi-Objetivo (Batería 2) 82
5.4.2 Consultas Negativas de Control e Inmunidad a Falsos Positivos (Batería 3) . . 83
5.4.3 Validación Cuantitativa de la Hipótesis 3 (H_3) 84
5.5 Análisis de Eficiencia Computacional, Latencias y Huella de Almacenamiento 85
5.5.1 Desglose Granular de Latencias 85

5.5.2 Escalabilidad de la Huella de Almacenamiento Vectorial 85
5.5.3 Discusión y Recomendaciones de Diseño 86
5.6 Evaluación Empírica Comparada: Modelos Locales Open-Weights y Edge frente a Cloud API 87
5.6.1 Metodología Experimental y Restricciones de Hardware 87
5.6.2 Análisis de Precisión Inter-Dominio (`python_tutorial.mp4` vs. `animals.mp4`) 88
5.6.3 Curvas de Ablación Matryoshka (MRL) y Normalización L_2 90
5.6.4 Rendimiento Computacional, Latencia y Huella de Memoria 92
5.6.5 Criterios de Selección Arquitectónica Cloud vs. Local 93

6 Conclusiones y Trabajo Futuro **95**
6.1 Validación Formal de Hipótesis y Cumplimiento de Objetivos 95
6.1.1 Validación de las Hipótesis de Investigación 95
6.1.2 Cumplimiento de los Objetivos Específicos 96
6.2 Contribuciones Científicas y Metodológicas al Estado del Arte 97
6.3 Limitaciones del Estudio y Restricciones Técnicas 98
6.4 Líneas de Investigación Futura 99

Índice de figuras

1 Comparativa arquitectónica entre (a) la canalización tradicional basada en conversión intermedia a texto plano mediante OCR y (b) la proyección multimodal directa en un espacio semántico continuo mediante modelos VLM y codificación M-RoPE [1, 2]. 17
2 Geometría del espacio vectorial compartido $\mathbb{R}^d$: los vectores normalizados de distintas modalidades (consulta textual $\mathbf{q}_{\text{texto}}$, fotogramas de vídeo $\mathbf{v}_{\text{vídeo}}$ y pista de audio $\mathbf{v}_{\text{audio}}$) que describen un mismo evento convergen con un ángulo $\theta \to 0°$ sobre la hipersfera unitaria, permitiendo recuperar la marca temporal exacta mediante Similitud del Coseno [2]. 18
3 Estructura jerárquica de un vector entrenado mediante *Matryoshka Representation Learning* (MRL): las primeras dimensiones concentran la mayor carga semántica, permitiendo truncar el vector de salida para reducir exponencialmente el consumo de memoria RAM y latencia con penalizaciones mínimas de precisión [2, 3]. 19
4 Evolución taxonómica de los modelos de representación vectorial: desde la alineación estática dependiente de transcripción textual (CLIP/Whisper [4, 5]) hasta los modelos omnimodales nativos de tercera generación capaces de proyectar texto, imagen, vídeo, audio y PDF en un único clúster topológico [1, 2, 6]. 23
5 Mecanismo de agregación intermodal en *Gemini Embedding 2*: los tres fotogramas representativos de un `VideoChunk` se empaquetan como objetos `Part` individuales dentro de un único contenedor `types.Content`, permitiendo que el modelo fusione la información temporal y devuelva un único vector denso por fragmento [2, 7]. 24
6 Comparativa estructural entre (a) un *Bi-Encoder* de doble torre que codifica consulta y vídeo de forma independiente para indexación pasiva en bases de datos vectoriales, y (b) un *Cross-Encoder* (*Reranker*) que aplica atención cruzada conjunta sobre el subconjunto de candidatos recuperados [3, 6]. 26
7 Flujo del paradigma de búsqueda asimétrica en dos etapas (Regla 99 % / 1 %): el *Bi-Encoder* filtra masivamente el repositorio mediante similitud del coseno en Qdrant y el *Cross-Encoder* o verificador visual inspecciona únicamente el subconjunto Top-K para garantizar máxima precisión con latencia mínima [3, 6]. 27
8 Comparativa técnica de los dos flujos de ingestión en el SDK `google-genai`: (a) subida de archivos completos a la API Files con bucle de espera activa hasta alcanzar el estado `ACTIVE`, y (b) codificación JPEG en memoria de múltiples fotogramas empaquetados en un único objeto `types.Content` para llamadas asíncronas de baja latencia [2]. 30
9 Modelado de capacidad y cortafuegos de cuota para `gemini-embedding-2-preview`: la validación previa aborta ejecuciones que superen los 800 fragmentos para proteger el límite diario (1.000 RPD), mientras que la densidad de $\sim$ 900 *tokens* por fragmento fija el caudal efectivo en $\sim$ 33 *chunks*/minuto bajo el techo de 30.000 TPM [2]. 32
10 Cronograma temporal del algoritmo de *Exponential Backoff* calibrado (`retry_base_delay = 4.0`, `max_retries = 4`): la suma geométrica de las cuatro esperas alcanza exactamente 60 segundos, permitiendo superar bloqueos completos de la ventana de 30.000 TPM sin abortar la vectorización del fragmento. 33

11 Arquitectura de Degradación Elegante (*Graceful Degradation*) durante la búsqueda: si los servidores del modelo secundario (`gemini-3.5-flash-lite`) sufren picos de saturación (`503 UNAVAILABLE`) tras agotar los reintentos, el sistema conmuta limpiamente al orden de similitud vectorial calculado por Qdrant con *Gemini Embedding 2*. 34
12 Clasificación de las arquitecturas de modelos fundacionales para Video RAG en 2026: coexistencia de modelos hiper-escalados servidos mediante API comercial frente a ecosistemas de pesos abiertos desplegables en infraestructura local [2, 6, 8]. 36
13 Resolución de consultas altamente composicionales mediante la arquitectura en dos etapas de la familia Qwen3-VL: mientras el *Bi-Encoder* puede recuperar candidatos con atributos invertidos, las capas de *Cross-Attention* de `Qwen3-VL-Reranker` vinculan cada adjetivo y acción con su parche visual exacto para predecir el *logit* de probabilidad real [3, 6]. 37
14 Flujo de codificación visual directa en arquitecturas como *ColQwen* y *Sentence Transformers* multimodales: la combinación de teselas visuales con codificación posicional M-RoPE proyecta diapositivas, gráficos y escenas de vídeo directamente al espacio del lenguaje, eliminando el preprocesamiento intermedio mediante OCR [1, 9]. 38
15 Comparativa de compromisos de ingeniería entre el despliegue sobre API propietaria en la nube y la ejecución local con modelos de pesos abiertos para sistemas de recuperación de vídeo [2, 6]. 39
16 Comparativa de tres arquitecturas representativas de sistemas RAG de vídeo en la literatura reciente: (a) alineación de metadatos auxiliares en *Video-RAG*, (b) indexación multigranular en *CARVE* y (c) inferencia adaptativa resiliente en *Multi-RAG* [8, 10]. . . 42
17 Contraste entre el emparejamiento temático tradicional y el Anclaje Causal Espaciotemporal (*Temporal Grounding*), el cual proporciona trazabilidad determinista y auditable sobre la línea de tiempo del vídeo [2, 11]. 43
18 Diagnóstico del cuello de botella perceptivo en la etapa de verificación visual: el escalado a 512 px combinado con compresión JPEG agresiva (12-20) impide el OCR implícito de sintaxis de programación en pantalla, mientras que elevar `resize_width` al rango 768-1280 px y la calidad JPEG a 50-90 restaura la detección sin disparar el consumo de cuota. 44
19 Mecanismo de dilución semántica y pérdida de fotogramas clave en vídeos con baja variación visual: (a) un fragmento de casi 40 s muestreado con solo 3 fotogramas omite el instante en que aparece el diccionario y mezcla múltiples temas; (b) la imposición de un límite superior (`max_shot_frames` de 10-15 s) o la extracción proporcional (un fotograma extra cada 5 s) garantiza la cohesión semántica. 45
20 Comparativa estructural entre (b) la división arbitraria en intervalos fijos, que introduce ruido contextual al cortar escenas y oraciones a la mitad, y (c) la segmentación semántica adaptativa (*Video-RAC*), que alinea exactamente los límites de cada `VideoChunk` con las transiciones reales del contenido [7, 11]. 47
21 Flujo algorítmico del cálculo del Índice de Similitud Estructural (SSIM [12]) mediante filtrado Gaussiano en OpenCV (`compute_ssim`) y criterio de consolidación de objetos `VideoChunk` en `process_video_single_pass`. 49

22 Selección de fotogramas clave basada en Entropía de Shannon (`extract_key_frames`) según la metodología *Video-RAC*: extracción del fotograma inicial, el fotograma final y el fotograma intermedio que maximiza la densidad de información visual $H(F_t)$ [7]. 50
23 Arquitectura de procesamiento paralelo y alineación forzada intermodal implementada en el prototipo (`video_chunker.py` + `audio_transcriber.py` con `ffmpeg` y Whisper `base` [5]), mostrando las extensiones opcionales descritas en la literatura (CLIP híbrido, Silero VAD y WhisperX [7, 11, 13]). 51
24 Comparativa arquitectónica y de eficiencia computacional entre la ingestión de vídeo bruto en ventanas fijas a 1 FPS (*SentrySearch*) y la canalización adaptativa multimodal por entropía de Shannon propuesta en el TFM [2, 7, 14]. 53
25 Arquitectura de almacenamiento y recuperación en Qdrant: separación de vectores densos indexados mediante grafos HNSW y metadatos temporales alojados en el *payload* para consultas top-K ultra rápidas [2]. 54
26 Estructura combinada de evaluación empírica mediante el framework RAGAS y métricas de atribución verificable (ALCE/MIRAGE) para validar el rendimiento del sistema RAG de vídeo [7, 15]. 56
27 Flujo metodológico de cuatro etapas para la revisión sistemática de la literatura apoyado en gestores bibliográficos y entornos RAG acotados [16, 17]. 57
28 Entidades fundamentales del modelo de datos inmutable implementadas en el sistema. . 64
29 Compromiso dimensional Matryoshka (MRL): (a) retención prácticamente plana de *Recall@1* (79,4 % → 78,4 %) con leve mejora en *Recall@5* (97,1 % → 99,0 %) frente a (b) reducción del 75 % en memoria RAM (12 KB → 3 KB por vector) y descenso del 44,4 % en la latencia de búsqueda en Qdrant. 76
30 Mecanismo de corrección en la consulta `"Bucles while"`: la fusión temprana eleva el fragmento objetivo del séptimo al primer puesto en Qdrant (+0,1064 en coseno) y el verificador bimodal valida el código pese a la transcripción fonética *"Guile"*, descartando los fragmentos adyacentes sobre bucles `for`. 80
31 Comparativa de precisión inter-dominio en dimensión nativa (Etapa 1): *Gemini Embedding 2* lidera el tutorial técnico basado en código y locución sincronizada, pero los modelos locales invierten la jerarquía en metraje documental con *B-roll* visual desacoplado. 89
32 Curvas de ablación Matryoshka (MRL) globales: `qwen3-vl-2b` exhibe invarianza perfecta del 100 % en *Recall@1* (76,5 %) desde 2048d hasta 768d, mientras que `gemma2@512` alcanza el máximo global de *Interval Recall* multi-objetivo (82,1 %) y conserva 73,5 % de *Recall@1* incluso a 256d (1 KB/chunk). 91
33 Compromiso entre rendimiento temporal y recursos de memoria en Apple Silicon M3 (16 GB): `gemma2` reduce la latencia de consulta local en un factor de 15–16× frente a Cloud API consumiendo casi la mitad de memoria MPS que `qwen3-vl-2b`, mientras que `qwen3-vl-2b` consigue la indexación de vídeo más veloz. 93

Índice de tablas

1 Comparativa técnica del impacto de la dimensionalidad vectorial y el soporte MRL sobre la precisión semántica y el coste de almacenamiento en bases de datos vectoriales [2, 6]. . 20

2 Comparativa técnica entre arquitecturas *Bi-Encoder*, *Cross-Encoder* (*Reranker*) y Verificadores VLM Generativos en sistemas Video RAG [2, 3, 6]. 27
3 Parámetros técnicos calibrados en los módulos de vectorización y orquestación para operar con `gemini-embedding-2-preview` sin colapsos de cuota [2]. 34
4 Síntesis comparativa de las principales arquitecturas y modelos del ecosistema multimodal analizados en la literatura (2025-2026) [1, 2, 6, 8]. 39
5 Diagnóstico empírico de parámetros de preprocesamiento visual y temporal en sistemas Video RAG con verificación secundaria (*Gemini Embedding 2* + *Gemini Flash*) [2]. 46
6 Comparativa de parámetros entre la literatura de referencia (*Video-RAC*, Silero VAD/WhisperX) y el prototipo implementado [5, 7, 11–13]. 51
7 Comparativa técnica y de costes operativos entre *SentrySearch* y la arquitectura desarrollada en el TFM [2, 7, 14]. 53
8 Síntesis de métricas, bancos de pruebas y metodologías de evaluación en sistemas Video RAG [6, 7, 15, 16]. 57
9 Resultados de la ablación dimensional MRL sobre consultas literales de evento único (102 consultas, 164 fragmentos en precisión `float32`) en la Etapa 1 (Qdrant). 76
10 Comparativa empírica entre la arquitectura puramente visual y la canalización bimodal con fusión temprana de audio (*Whisper*) y verificación en dos etapas. 78
11 Auditoría detallada de fragmentos en las consultas críticas "`Diccionarios`" y "`Bucles while`" sobre `python_tutorial.mp4`. 81
12 Efecto de la deduplicación por transcripción idéntica (`run_20261109_183310`) y del desacoplamiento de audio sobre `animals.mp4`. 82
13 Rendimiento en consultas multi-objetivo (20 consultas, 106 intervalos esperados, $k_1 = 10$) en Etapa 1 (Qdrant) y Etapa 2 (Juez Bimodal). 82
14 Desglose por consulta general y taxonómica sobre `animals.mp4` (10 consultas, 58 intervalos temporales esperados) en Etapa 1 (E1) y Etapa 2 (E2). 83
15 Evaluación de consultas negativas de control (16 consultas, 240 candidatos analizados en Etapa 2) a través de las tres dimensiones MRL. 84
16 Proyección de memoria vectorial en función del volumen de metraje y la dimensión MRL. 85
17 Resultados comparativos completos en Etapa 1 (`rerank=False`) por dominio audiovisual, motor de embeddings y dimensión MRL. 89
18 Degradación empírica de la norma euclídea (L_2) en prefijos Matryoshka sin renormalizar frente a la restitución unitaria con `truncate_and_normalize_mrl`. 91
19 Rendimiento computacional, latencia de consulta y huella máxima de memoria por vídeo y backend (configuraciones nativas). 92

1 Introducción

1.1 Contexto y Motivación

La evolución contemporánea de los sistemas de Recuperación Aumentada por Generación (RAG, por sus siglas en inglés) ha transitado desde arquitecturas puramente textuales hacia paradigmas multimodales capaces de procesar, alinear e interpretar información visual, auditiva y textual dentro de un único espacio semántico continuo [1]. En el ámbito de la recuperación de eventos de interés sobre secuencias de vídeo de larga duración (disciplina denominada en la literatura académica como *Long-form Video Understanding* o *Video RAG*), tanto la investigación científica como el desarrollo industrial han experimentado un avance significativo entre los años 2024 y 2026 [8]. Este progreso responde a la necesidad de identificar, localizar y extraer eventos específicos dentro de archivos de vídeo extensos (tales como la mención de un producto, la aparición de un objeto o un cambio de escena), eliminando la necesidad del visionado manual.

1.2 Planteamiento del Problema

Históricamente, la arquitectura tradicional para recuperar información en vídeo obligaba a procesar los fotogramas por separado, emplear modelos de transcripción fonética básica para el audio (como Whisper) y extraer texto mediante Reconocimiento Óptico de Caracteres (OCR) para indexarlo todo como texto plano [10]. El estado del arte ha demostrado empíricamente que convertir toda la información visual a texto plano genera un cuello de botella informacional y una pérdida severa de contexto temporal, espacial, estructural y emocional, haciendo desaparecer la disposición espacial (*layout*) y las relaciones semánticas entre los objetos visuales [1]. Frente a estas limitaciones, la aparición de modelos de *embeddings* omnimodales permite ingerir texto, vídeo, audio y documentos estructurados (PDF) de forma nativa para proyectarlos en un único vector numérico sin requerir metadatos anotados manualmente [2].

No obstante, la proyección conjunta de vídeo y audio en espacios vectoriales continuos plantea interrogantes abiertos de investigación: ¿cómo interacciona la pista de voz con el contenido visual cuando existe desacoplamiento temático (*B-roll*)? ¿cuánta fidelidad semántica se pierde al truncar representaciones multimodales densas mediante técnicas como *Matryoshka Representation Learning* (MRL)? y ¿cómo evitar falsos positivos derivados de la similitud coseno en búsquedas aproximadas? Este trabajo aborda estas cuestiones de forma empírica y metodológica.

1.3 Preguntas de Investigación e Hipótesis

Para guiar la investigación de manera sistemática, se formulan tres preguntas de investigación (*Research Questions*, RQs) y sus correspondientes hipótesis de partida:

RQ1: ¿En qué medida la fusión temprana de fotogramas clave y transcripción fonética continua alineada dentro de un espacio vectorial denso omnimodal mejora la precisión de recuperación frente a canalizaciones unimodales o desacopladas?

↪ **Hipótesis 1 (H_1):** La fusión temprana de imágenes y locución temporalmente alineada en un único embedding reduce el cuello de botella contextual, incrementando de forma medible el *Recall@1* y el *MRR* en contenido con locución sincronizada, mientras que preserva la recuperabilidad de los fragmentos en metraje con audio y vídeo desacoplados (*B-roll*).

RQ2: ¿Cuál es la tasa de degradación de las métricas de recuperación al comprimir el espacio latente de 3072 dimensiones hasta 128 dimensiones mediante *Matryoshka Representation Learning* (MRL)?

↪ **Hipótesis 2 (H_2):** El truncamiento dimensional guiado por MRL [18] retiene más del 90 % del rendimiento original de recuperación (*MRR*, *NDCG@5* y *Hit@1*) en subespacios de hasta 512 dimensiones, permitiendo reducir el consumo de memoria vectorial y acelerar la búsqueda aproximada con un coste métrico mínimo.

RQ3: ¿Cómo contribuye una arquitectura de recuperación en dos etapas (*Retrieval* denso + Reordenación multimodal con LLM) a la erradicación de falsos positivos y a la resolución de consultas complejas?

↪ **Hipótesis 3 (H_3):** Un juez bimodal de segunda etapa basado en un modelo fundacional multimodal mitiga los falsos positivos derivados de coincidencias de similitud coseno espurias (alcanzando tasas de rechazo del 100 % en consultas negativas de control) y resuelve eficazmente consultas que combinan texto, imágenes y audio.

1.4 Objetivos del Trabajo Fin de Máster

El **Objetivo General** de este trabajo consiste en: *Diseñar, formalizar, implementar y evaluar empíricamente una arquitectura de recuperación de información multimedia basada en vectorización omnimodal y reordenación bimodal, demostrando su viabilidad técnica, precisión y eficiencia en la localización temporal de eventos sobre secuencias de vídeo de larga duración sin metadatos previos.*

Para alcanzar este fin, se definen cinco **Objetivos Específicos**:

OE1: Analizar el estado del arte tecnológico y los fundamentos matemáticos de los espacios vectoriales multimodales, los modelos fundacionales omnimodales y las técnicas de fragmentación temporal para *Video-RAG*.

OE2: Formalizar la metodología y arquitectura del sistema, definiendo la fragmentación adaptativa por planos visuales (SSIM), la alineación temporal forzada de locución fonética (Whisper) y la proyección multimodal temprana.

OE3: Desarrollar un prototipo experimental (PoC) funcional, desacoplado e instrumentado con trazabilidad integral (*RunAuditor*), estructurado en torno al protocolo común `MultimodalVectorizer` (`base.py`), truncamiento `mrl.py`, factoría `factory.py` y adaptadores específicos para API Cloud (*Gemini Embedding 2*) e inferencia local acelerada en `bfloat16` sobre Metal MPS (`LocalMultimodalVectorizer` en `local_vectorizer.py` para *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2*), junto con almacenamiento aislado en *Qdrant* y verificación de segunda etapa con *Gemini 3.5 Flash Lite*.

OE4: Ejecutar un plan de evaluación empírica riguroso, analizando la ablación multimodal (visual vs. audio-visual), la compresión dimensional MRL en cada motor ($\{3072, 1536, 768\}$ en Gemini, $\{2048, 1536, 768\}$ en Qwen3-VL-2B y $\{768, 512, 256\}$ en EmbeddingGemma 2), la comparativa sistemática Cloud frente a modelos abiertos locales, y el comportamiento ante consultas multi-objetivo y negativas de control.

OE5: Evaluar la viabilidad computacional y costes, midiendo latencias de indexación y búsqueda, consumo de cuota de inferencia y escalabilidad para la transferencia a entornos productivos y líneas de investigación futuras.

1.5 Contribuciones Principales

Las principales contribuciones científicas y metodológicas de esta investigación se resumen en:

- **Metodología de Fusión Temprana Audio-Visual:** Integración de planos visuales clave extraídos por similitud estructural (SSIM) con locución transcrita y alineada temporalmente, empaquetados en una única llamada de vectorización omnimodal.
- **Estudio Empírico de Ablación MRL en Vídeo:** Evaluación sistemática del comportamiento de representaciones Matryoshka implementadas en `mrl.py` sobre *Gemini Embedding 2* (3072, 1536, 768), *Qwen3-VL-Embedding-2B* (2048, 1536, 768) y *EmbeddingGemma 2* (768, 512, 256), analizando el compromiso entre precisión y ahorro de memoria.
- **Arquitectura de Recuperación y Verificación Bimodal en Dos Etapas:** Mecanismo de reordenación asistido por un modelo de lenguaje multimodal que combina collages visuales y transcripciones para desambiguar candidatos y erradicar falsos positivos.
- **Arnés Experimental Reproducible y Conjunto de Datos Anotado:** Banco de pruebas automatizado con métricas objetivas de Information Retrieval y trazabilidad completa de cada ejecución para auditoría experimental.
- **Comparativa Empírica Multi-Backend (Cloud vs. Open-Weights vs. Edge):** Arquitectura extensible mediante el protocolo `MultimodalVectorizer` (`base.py`), la factoría `factory.py` y el adaptador local `local_vectorizer.py` (`bfloat16` sobre Apple Silicon MPS), contrastando precisión semántica, latencia y huella de memoria entre *Gemini Embedding 2*, *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2* bajo colecciones Qdrant aisladas.

1.6 Estructura del Documento

La presente memoria se organiza en seis secciones:

- **Sección 1 (Introducción):** Presenta el contexto, motivación, preguntas de investigación, hipótesis, objetivos y contribuciones del trabajo.
- **Sección 2 (Revisión de la Literatura):** Examina los fundamentos matemáticos de los embeddings, la taxonomía de modelos omnimodales, arquitecturas *Video-RAG*, técnicas de fragmentación temporal y métricas de evaluación.

- **Sección 3 (Metodología y Propuesta Arquitectónica):** Formaliza matemáticamente el espacio latente multimodal, la ingesta adaptativa y el proceso de recuperación y verificación en dos etapas.
- **Sección 4 (Configuración Experimental y Prototipo):** Detalla el banco de pruebas experimental, la implementación del prototipo y el arnés de observabilidad y trazabilidad.
- **Sección 5 (Resultados Experimentales y Discusión Crítica):** Expone y analiza cuantitativamente los resultados de los estudios de ablación (audio-visual y dimensional MRL), las consultas complejas, los costes computacionales y la comparativa empírica entre la API Cloud y los modelos locales abiertos (*Qwen3-VL-2B* y *EmbeddingGemma 2*).
- **Sección 6 (Conclusiones y Trabajo Futuro):** Contrasta los resultados contra las hipótesis de partida, expone las lecciones aprendidas y propone líneas de investigación futuras.

2 Revisión de la Literatura y Marco Teórico

Esta sección establece el marco teórico y metodológico necesario para fundamentar las preguntas de investigación e hipótesis planteadas en la Sección 1. Se analizan en profundidad los fundamentos matemáticos de los espacios latentes y la similitud métrica, la taxonomía y evolución de los modelos fundacionales omnimodales (con especial atención a *Gemini Embedding 2* y el mecanismo *Matryoshka Representation Learning*), el ecosistema de modelos abiertos locales y las arquitecturas de Recuperación Aumentada por Generación sobre vídeo (*Video-RAG*). Asimismo, se examinan formalmente las técnicas de fragmentación temporal adaptativa, alineación forzada de locución y las métricas estandarizadas de evaluación en recuperación de información, sentando las bases conceptuales para el diseño del sistema y la experimentación posterior.

2.1 Fundamentos Matemáticos de los Embeddings Multimodales y Espacios Semánticos Continuos

2.1.1 El Problema del Análisis Secuencial y la Representación de Datos No Estructurados

El crecimiento sostenido de los repositorios multimedia en entornos académicos, industriales y de radiodifusión ha puesto de manifiesto un problema metodológico fundamental en la ingeniería de datos: la extrema ineficiencia del análisis secuencial humano frente a volúmenes masivos de información no estructurada [1]. Históricamente, la tarea de localizar un evento singular dentro de un archivo de vídeo de larga duración (disciplina denominada en la literatura científica como *Long-form Video Understanding* o *Video RAG*) exigía bien un visionado manual exhaustivo por parte de operadores humanos, o bien una dependencia absoluta de metadatos descriptivos y etiquetas manuales generadas previamente a la indexación [8]. En escenarios reales que abarcan desde archivos históricos de televisión y auditorías de videovigilancia hasta repositorios de clases universitarias grabadas o historiales médicos complejos, el etiquetado manual resulta técnica y económicamente inviable.

Para superar la barrera del procesamiento manual, los sistemas de Recuperación de Información (*Information Retrieval*) evolucionaron hacia el paradigma de la Recuperación Aumentada por Generación (RAG, por sus siglas en inglés) [1]. No obstante, mientras que las arquitecturas RAG de primera generación operaban exclusivamente sobre cadenas de texto plano, los flujos multimedia contemporáneos requieren procesar, alinear e interpretar simultáneamente señales heterogéneas compuestas por secuencias de fotogramas, pistas de audio y texto libre [1, 8].

La base matemática que permite a un sistema computacional operar sobre formatos tan dispares sin recurrir a reglas heurísticas manuales es el concepto de **representación vectorial densa** o ***embedding*** [2]. Un *embedding* es una proyección matemática que transforma un objeto de entrada arbitrario x (ya sea una oración en lenguaje natural, una matriz de píxeles correspondiente a un fotograma de vídeo o una onda acústica) en un vector de números reales en coma flotante $\mathbf{v} \in \mathbb{R}^d$ dentro de un espacio métrico continuo de dimensión d, según se formaliza en la Ecuación (1) [2]:

$$f_\theta : \mathfrak{X}_{\text{texto}} \cup \mathfrak{X}_{\text{imagen}} \cup \mathfrak{X}_{\text{vídeo}} \cup \mathfrak{X}_{\text{audio}} \cup \mathfrak{X}_{\text{PDF}} \longrightarrow \mathbb{R}^d \tag{1}$$

donde f_θ representa una red neuronal profunda parametrizada por los pesos θ (un Modelo Fundacional Multimodal) y $\mathbb{R}^d$ constituye el **espacio semántico compartido** o espacio latente unificado [2]. La propiedad fundamental de este mapeo unificado reside en que la topología del espacio vectorial preserva la semántica subyacente con independencia de la modalidad de origen: para que un sistema inteligente comprenda que la fotografía de un objeto o ser vivo (por ejemplo, la imagen de un perro), su manifestación acústica (el ladrido grabado en una pista de audio) y su representación simbólica (la palabra escrita "perro") expresan el mismo concepto, el modelo proyecta las tres entradas hacia coordenadas geométricas sumamente próximas entre sí dentro del mismo plano matemático [2].

2.1.2 El Ocaso de las Canalizaciones OCR y la Ingestión Visual Nativa con M-RoPE

Durante la primera mitad de la década de 2020, el diseño estándar para indexar contenido visual y de vídeo en sistemas RAG imponía una canalización rígida, escalonada y unimodal [1]. Este flujo tradicional obligaba a extraer fotogramas aislados del vídeo, aplicar algoritmos clásicos de detección de contornos, someter cada imagen a un motor de Reconocimiento Óptico de Caracteres (OCR) para obtener cadenas de texto plano, transcribir en paralelo la pista de audio mediante sistemas de Reconocimiento Automático del Habla (ASR, como el modelo Whisper) e inyectar finalmente todo el texto concatenado en un modelo de *embeddings* puramente textual [1, 10].

La literatura científica publicada entre 2024 y 2026 ha demostrado empíricamente el colapso operativo de este paradigma basado en traducción forzosa a texto plano [1]. Convertir toda la riqueza de una señal visual en secuencias léxicas mediante OCR introduce tres deficiencias estructurales críticas [1]:

1. **Cuello de botella informacional:** La traducción de una escena tridimensional o de un fotograma complejo a palabras sueltas descarta toda la información visual no textual, tales como acciones físicas, colores, texturas, expresiones emocionales e interacciones implícitas entre objetos [1].
2. **Destrucción del diseño espacial (*Layout*):** Cuando un sistema procesa diapositivas académicas, diagramas arquitectónicos, tablas financieras o editores de código fuente extrayendo únicamente los caracteres visibles mediante OCR, la disposición bidimensional (*layout*) y las relaciones jerárquicas y topológicas entre los bloques visuales desaparecen por completo [1].
3. **Propagación de errores en cascada:** Cualquier fallo de reconocimiento en el motor OCR (frecuente ante cambios de iluminación, tipografías pequeñas, compresión de vídeo o fondos oscuros) corrompe irreversiblemente el texto indexado, impidiendo que el motor de búsqueda recupere el segmento aunque el contenido visual original sea perfectamente claro para un observador [1].

La Figura 1 contrasta esquemáticamente el flujo tradicional dependiente de OCR frente al paradigma contemporáneo de vectorización multimodal directa [1].

Para erradicar esta pérdida de fidelidad, el estado del arte ha adoptado arquitecturas de recuperación visual directa lideradas por modelos como **ColQwen** y sistemas basados en *Vision-Language Models* (VLMs) de última generación como la familia **Qwen3-VL** y **Gemini Embedding 2** [1, 2, 6]. En lugar de extraer caracteres de forma aislada, estos sistemas ingieren las imágenes y los fotogramas de vídeo a su resolución nativa y dividen el lienzo visual en una cuadrícula de parches o teselas (*visual patches*),

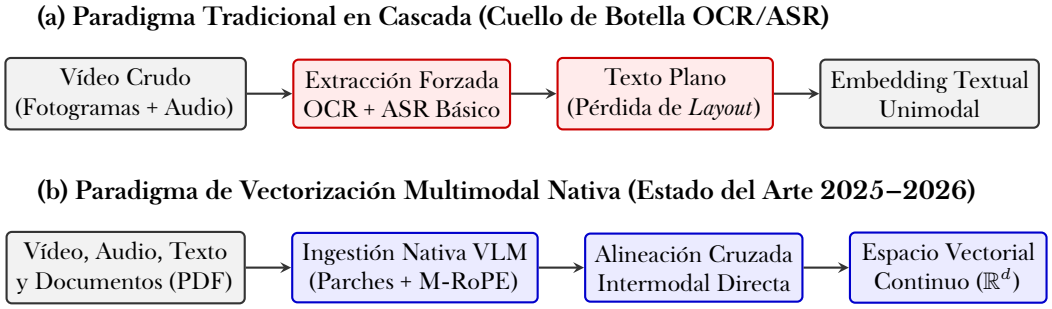


Figura 1: Comparativa arquitectónica entre (a) la canalización tradicional basada en conversión intermedia a texto plano mediante OCR y (b) la proyección multimodal directa en un espacio semántico continuo mediante modelos VLM y codificación M-RoPE [1, 2].

siguiendo el principio arquitectónico de los *Vision Transformers* (ViT) [1].

El avance matemático determinante que permite a estos modelos conservar la estructura geométrica de la escena es la incorporación de ***Multimodal Rotary Position Embeddings*** **(M-RoPE)** [1]. Mientras que las codificaciones posicionales rotatorias clásicas (RoPE) en modelos de lenguaje operan sobre una única dimensión secuencial (el índice t del *token* en una oración), M-RoPE descompone y generaliza la matriz de rotación posicional para codificar coordenadas multidimensionales de manera nativa [1]. Al procesar un fragmento de vídeo, cada parche visual recibe una codificación posicional que preserva su posición espacial bidimensional (x, y) dentro del fotograma y su evolución temporal t a lo largo de la secuencia, permitiendo procesar resoluciones variables sin deformar la relación de aspecto y alineando directamente las teselas visuales en el mismo espacio vectorial que los *tokens* de texto del modelo subyacente [1].

2.1.3 Geometría del Espacio Latente y Métricas de Similitud Estadística

Una vez que el modelo fundacional proyecta tanto los fragmentos del archivo de vídeo como la consulta en lenguaje natural del usuario dentro del espacio vectorial compartido $\mathbb{R}^d$, el problema de localizar el instante exacto de un evento (*timestamp*) se reduce a un cálculo geométrico de cercanía estadística [11].

Sea $\mathbf{q} = (q_1, q_2, \ldots, q_d)^\top \in \mathbb{R}^d$ el vector de *embedding* correspondiente a una consulta textual (por ejemplo, *"¿En qué momento se muestra un diccionario en el código?"*) y sea $\mathbf{v}_i = (v_{i,1}, v_{i,2}, \ldots, v_{i,d})^\top \in \mathbb{R}^d$ el vector denso asociado al i-ésimo fragmento temporal $[t^{(i)}_{\text{inicio}}, t^{(i)}_{\text{fin}}]$ de un vídeo indexado. La literatura y las bases de datos vectoriales modernas emplean fundamentalmente dos operadores algebraicos para cuantificar la afinidad semántica entre $\mathbf{q}$ y $\mathbf{v}_i$ [2, 3]:

2.1.3.1 Similitud del Coseno (*Cosine Similarity*):

Mide el coseno del ángulo θ formado entre ambos vectores en el hiperplano d-dimensional mediante la Ecuación (2), evaluando su alineación direccional con independencia de sus magnitudes absolutas:

$$\text{sim}_{\cos}(\mathbf{q}, \mathbf{v}_i) = \cos(\theta) = \frac{\mathbf{q} \cdot \mathbf{v}_i}{\|\mathbf{q}\|_2 \, \|\mathbf{v}_i\|_2} = \frac{\sum_{j=1}^{d} q_j \, v_{i,j}}{\sqrt{\sum_{j=1}^{d} q_j^2} \sqrt{\sum_{j=1}^{d} v_{i,j}^2}} \in [-1, 1] \tag{2}$$

Un valor cercano a 1 indica una coincidencia semántica elevada (vectores colineales que apuntan hacia la misma región conceptual del espacio latente), un valor próximo a 0 denota ortogonalidad (conceptos no relacionados) y valores negativos reflejan oposición semántica.

2.1.3.2 Producto Punto Escalar (*Scalar Dot Product*):

Definido algebraicamente en la Ecuación (3) como la suma de los productos componente a componente [3]:

$$\mathrm{sim}_{\mathrm{dot}}(\mathbf{q}, \mathbf{v}_i) = \mathbf{q}^\top \mathbf{v}_i = \sum_{j=1}^{d} q_j \, v_{i,j} \tag{3}$$

Cuando todos los vectores almacenados en la base de datos vectorial han sido previamente normalizados a longitud unitaria mediante la norma euclídea L_2 ($\|\mathbf{q}\|_2 = \|\mathbf{v}_i\|_2 = 1$), los denominadores de la Ecuación (2) se vuelven exactamente igual a la unidad, haciendo que la Similitud del Coseno y el Producto Punto Escalar sean matemáticamente idénticos ($\mathrm{sim}_{\cos}(\mathbf{q}, \mathbf{v}_i) = \mathbf{q}^\top \mathbf{v}_i$) [3]. Esta equivalencia resulta vital en sistemas de recuperación a escala de petabytes, ya que reduce la búsqueda a multiplicaciones matriciales altamente paralelizadas en hardware moderno [3].

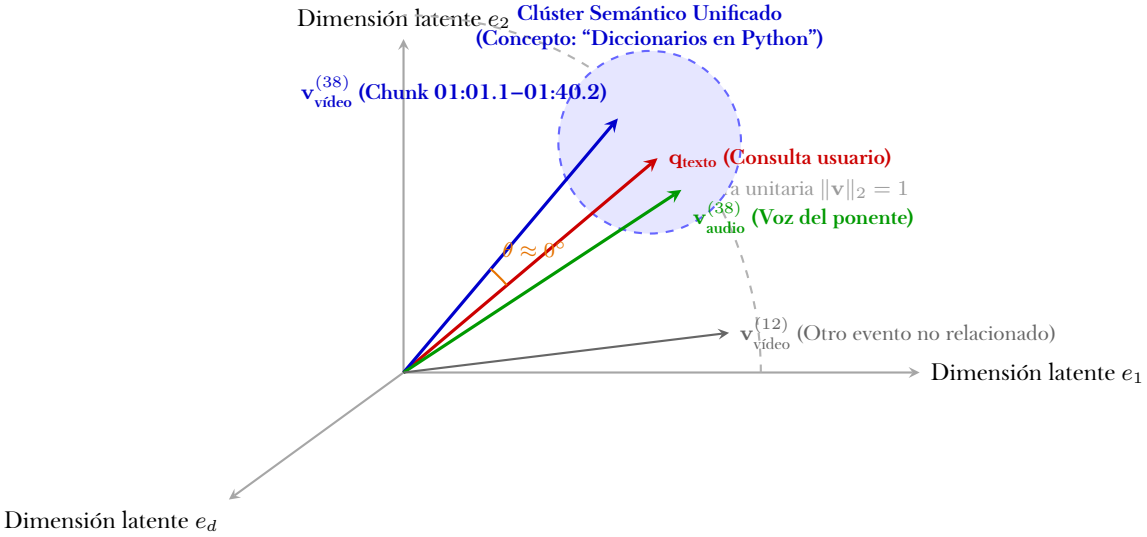


Figura 2: Geometría del espacio vectorial compartido $\mathbb{R}^d$: los vectores normalizados de distintas modalidades (consulta textual $\mathbf{q}_{\text{texto}}$, fotogramas de vídeo $\mathbf{v}_{\text{vídeo}}$ y pista de audio $\mathbf{v}_{\text{audio}}$) que describen un mismo evento convergen con un ángulo $\theta \to 0°$ sobre la hipersfera unitaria, permitiendo recuperar la marca temporal exacta mediante Similitud del Coseno [2].

La Figura 2 ilustra esta convergencia geométrica sobre la hipersfera unitaria: dado que la consulta en texto libre y el fragmento de vídeo comparten la misma región del espacio latente, el motor de búsqueda identifica el intervalo temporal exacto del suceso evaluando qué vectores $\mathbf{v}_i$ maximizan $\cos(\theta)$ respecto a $\mathbf{q}$ [2].

2.1.4 Aprendizaje de Representación Matryoshka (MRL) y Normalización L_2

Uno de los mayores retos de ingeniería al escalar sistemas de vectorización multimodal a millones de fragmentos de vídeo reside en la rígida dependencia entre la dimensionalidad de salida d del modelo y el consumo de memoria RAM y almacenamiento en la base de datos vectorial [3, 6]. En los modelos de *embeddings* tradicionales, toda la información semántica se distribuye de manera uniforme y difusa a lo largo de todas las coordenadas del vector $(v_1, v_2, \ldots, v_d)$; en consecuencia, recortar un vector clásico sin volver a entrenar la red neuronal destruye su estructura topológica y provoca una caída drástica en la precisión de búsqueda [3].

Para dotar de elasticidad dimensional a las arquitecturas modernas (tales como **Gemini Embedding 2**, **Qwen3-VL-Embedding** y **EmbeddingGemma 2**), el estado del arte ha estandarizado el **Aprendizaje de Representación Matryoshka** (*Matryoshka Representation Learning*, **MRL**) [2, 6]. Inspirado en las muñecas rusas anidadas (*matryoshkas*), el entrenamiento mediante la función de pérdida *MatryoshkaLoss* optimiza simultáneamente un conjunto de sub-dimensiones anidadas $\mathcal{M} = \{m_1, m_2, \ldots, m_k\}$ donde $m_1 < m_2 < \ldots < m_k = d$ (por ejemplo, $\mathcal{M} = \{64, 128, 256, 512, 768, 1024, 1536, 3072, 4096\}$) [2, 3, 6]. Formalmente, la pérdida total durante el entrenamiento contrastivo se expresa mediante la Ecuación (4) como la suma ponderada de las pérdidas evaluadas sobre cada prefijo truncado $\mathbf{v}_{1:m} \in \mathbb{R}^m$ [3]:

$$\mathcal{L}_{\text{MRL}}(\mathbf{q}, \mathbf{v}) = \sum_{m \in \mathcal{M}} c_m \cdot \mathcal{L}_{\text{contrastiva}}(\mathbf{q}_{1:m}, \mathbf{v}_{1:m}) \tag{4}$$

Esta formulación obliga a la red neuronal a jerarquizar la varianza semántica, comprimiendo los atributos conceptuales más críticos y globales en las primeras coordenadas del vector y reservando las dimensiones finales para matices granulares de grano fino [3]. En la práctica operativa, esto permite descargar un modelo de alta capacidad como *Qwen3-VL-Embedding-8B* (cuya dimensión nativa es $d = 4096$) o invocar *Gemini Embedding 2* ($d = 3072$) y truncar dinámicamente sus vectores de salida a 1536, 1024, 768, 512, 256, 128 o incluso 64 dimensiones en tiempo de ejecución sin requerir ningún reentrenamiento [2, 3].



Figura 3: Estructura jerárquica de un vector entrenado mediante *Matryoshka Representation Learning* (MRL): las primeras dimensiones concentran la mayor carga semántica, permitiendo truncar el vector de salida para reducir exponencialmente el consumo de memoria RAM y latencia con penalizaciones mínimas de precisión [2, 3].

Ahora bien, desde el punto de vista de la implementación algorítmica rigurosa, truncar un vector unitario $\mathbf{v} \in \mathbb{R}^D$ (donde $\|\mathbf{v}\|_2 = 1$) a sus primeras $d < D$ componentes ($\mathbf{v}_{1:d}$) hace que la suma de cuadrados de las coordenadas retenidas sea estrictamente menor que la unidad ($\sum_{j=1}^{d} v_j^2 < 1$). Dado que los vectores truncados (por ejemplo, al configurar $d = 768$ o $d = 1536$ en lugar de las 3072 dimensiones

completas en *Gemini Embedding 2*) no vienen normalizados de fábrica tras el corte, resulta obligatorio aplicar una etapa explícita de **re-normalización mediante la norma euclídea** L_2 según se define en la Ecuación (5) antes de insertarlos en la base de datos vectorial [2]:

$$\hat{\mathbf{v}}_{1:d} = \frac{\mathbf{v}_{1:d}}{\|\mathbf{v}_{1:d}\|_2} = \frac{(v_1, v_2, \ldots, v_d)^\top}{\sqrt{\sum_{j=1}^{d} v_j^2}} \tag{5}$$

Esta operación proyecta de nuevo el sub-vector truncado sobre la superficie de la hipersfera unitaria de dimensión d, garantizando que las búsquedas posteriores por Similitud del Coseno en motores como Qdrant mantengan su consistencia matemática y velocidad de cómputo.

2.1.5 Análisis del Compromiso entre Precisión Semántica y Coste de Almacenamiento

La elección de la dimensionalidad operativa del espacio latente introduce un compromiso directo (*trade-off*) de arquitectura entre la granularidad de discriminación semántica y los costes de infraestructura [2, 6]. Como se sintetiza en el Cuadro 1, trabajar con el espacio completo de **3.072 dimensiones** en *Gemini Embedding 2* (o **4.096 dimensiones** en *Qwen3-VL-Embedding-8B*) permite capturar un nivel de detalle semántico superior, lo cual resulta determinante cuando la base de datos acumula millones de registros multimodales y se requiere evitar falsos positivos ante consultas altamente específicas [2, 6].

Tabla 1: Comparativa técnica del impacto de la dimensionalidad vectorial y el soporte MRL sobre la precisión semántica y el coste de almacenamiento en bases de datos vectoriales [2, 6].

Configuración	**Modelos de Referencia**	**Peso (FP32)**	**Norm. L_2**	**Caso de Uso Óptimo**
4.096 dim (Máx. Open)	Qwen3-VL-Embedding-8B	16,0 KB	Nativa	Máxima resolución en servidor GPU local
3.072 dim (Máx. Cloud)	Gemini Embedding 2	12,0 KB	Nativa	*Baseline* de máxima precisión y millones de reg.
2.048 dim (Intermedia)	Qwen3-VL-Embedding-2B	8,0 KB	Nativa / MRL	Despliegue local equilibrado en VRAM
1.536 dim (MRL Medio)	Gemini Embedding 2	6,0 KB	Post-corte	Reducción del 50 % en RAM con alta fidelidad
768 dim (Estándar)	Gemini 2 / EmbeddingGemma 2	3,0 KB	Post-corte	Ahorro del 75 % en BD y consultas ultrarrápidas
64 - 256 dim (Ultraligero)	Qwen3-VL / EmbeddingGemma 2	0,25 - 1,0 KB	Post-corte	*Edge computing*, dispositivos móviles y pre-filtrado

Por el contrario, operar sobre un estándar de **768 dimensiones** (que constituye el tope nativo de modelos ligeros modulares como *EmbeddingGemma 2* (con posibilidad de truncado adicional a 256 o 128 dimensiones) y uno de los puntos de corte recomendados en *Gemini Embedding 2*) reduce exactamente en un 75 % la huella de memoria de cada vector respecto a las 3,072 dimensiones (pasando de 12, 0 KB a 3, 0 KB por fragmento en precisión simple de 32 bits). Esta compresión hace que la base de datos vectorial resultante sea significativamente más económica de alojar en memoria RAM y más rápida de consultar localmente, a cambio de asumir una ligera reducción en la granularidad semántica fina frente al vector completo de 3,072 dimensiones.

2.1.6 Aprendizaje Contrastivo (*InfoNCE*) y el Fenómeno de la Brecha Modal (*Modality Gap*)

Desde el punto de vista de la optimización neuronal, la alineación geométrica entre consultas textuales y fragmentos audiovisuales se entrena mediante funciones objetivo contrastivas simétricas basadas en la pérdida ***InfoNCE*** (*Noise-Contrastive Estimation*) [4, 19]. Dado un lote (*mini-batch*) de B pares emparejados $\{(\mathbf{q}_i, \mathbf{v}_i)\}_{i=1}^{B}$ proyectados sobre la hiperesfera unitaria $\mathbb{S}^{d-1}$, el entrenamiento maximiza la similitud de los pares positivos en la diagonal frente a los $B-1$ distractores negativos del lote, modulando la concentración de la distribución mediante un hiperparámetro de temperatura $\tau > 0$:

$$\mathscr{L}_{\text{InfoNCE}} = -\frac{1}{2B}\sum_{i=1}^{B}\left[\log\frac{\exp(\text{sim}_{\cos}(\mathbf{q}_i, \mathbf{v}_i)/\tau)}{\sum_{j=1}^{B}\exp(\text{sim}_{\cos}(\mathbf{q}_i, \mathbf{v}_j)/\tau)} + \log\frac{\exp(\text{sim}_{\cos}(\mathbf{v}_i, \mathbf{q}_i)/\tau)}{\sum_{j=1}^{B}\exp(\text{sim}_{\cos}(\mathbf{v}_i, \mathbf{q}_j)/\tau)}\right] \tag{6}$$

En la Ecuación (6), la temperatura τ controla la severidad con la que se penalizan los ejemplos negativos cercanos. Sin embargo, Liang et al. [20] demostraron un fenómeno topológico inherente a los espacios contrastivos multimodales denominado ***Modality Gap*** **(Brecha Modal)**: debido a que los codificadores de distinta modalidad nacen en regiones cónicas disjuntas de $\mathbb{S}^{d-1}$ tras la inicialización aleatoria, y a que la pérdida InfoNCE depende exclusivamente de las *diferencias relativas* entre los pares positivos y negativos escaladas por $1/\tau$, la optimización converge preservando un desplazamiento vectorial sistemático entre el subespacio textual y el subespacio visual/audiovisual.

Esta propiedad explica rigurosamente un hallazgo empírico central observado en la literatura y en nuestros experimentos sobre *Gemini Embedding 2* y *Qwen3-VL-Embedding*: incluso cuando un par consulta-fragmento constituye un acierto semántico inequívoco de rango #1, su similitud del coseno cruzada (*cross-modal*) rara vez se aproxima a $1,0$, agrupándose de manera natural dentro de la banda $[0{,}28;\ 0{,}50]$ (mientras que pares intramodales texto-texto alcanzan valores mucho mayores). En consecuencia, el *Modality Gap* invalida cualquier mecanismo de filtrado basado en umbrales fijos de coseno en la Etapa 1 y fundamenta la necesidad teórica de delegar la discriminación definitiva y el rechazo de falsos positivos a un verificador bimodal de Etapa 2.

2.2 Funcionamiento y Taxonomía de Modelos de Embeddings Multimodales

2.2.1 Evolución Arquitectónica: De los Modelos Alineados (CLIP e ImageBind) a la Omnimodalidad Nativa

El diseño de redes neuronales capaces de proyectar múltiples modalidades de datos dentro de un espacio métrico común ha atravesado tres generaciones claramente diferenciadas en la literatura científica reciente [1, 6]. Comprender las limitaciones estructurales de las dos primeras generaciones resulta indispensable para justificar la adopción de los modelos fundacionales omnimodales desarrollados entre finales de 2024 y 2026 [2, 6].

Durante la primera etapa evolutiva, el estándar de la industria estuvo dominado por modelos de alineación dual o múltiple como **CLIP** (*Contrastive Language-Image Pre-training*, desarrollado por OpenAI [4]) e **ImageBind** (desarrollado por Meta [21]) [1]. Estas arquitecturas lograron un hito fundacional al demostrar que era posible entrenar codificadores separados (un codificador visual y un codificador textual) mediante aprendizaje contrastivo [4, 19] para alinear texto e imágenes estáticas (y, en casos puntuales como ImageBind, señales de audio [21]) dentro de un mismo hiperplano vectorial [1]. Sin embargo, cuando estas arquitecturas de primera generación se aplicaban a la recuperación de información en secuencias de vídeo de larga duración, presentaban una limitación insalvable: carecían de una comprensión temporal nativa e integrada [1, 22]. Como consecuencia, la arquitectura tradicional obligaba a descomponer artificialmente el flujo multimedia: los ingenieros debían extraer y procesar los fotogramas de manera aislada, emplear modelos de transcripción fonética independientes para la pista de audio (como Whisper [5]) y terminar indexando toda la información resultante como texto plano [10]. Este flujo fragmentado no solo resultaba computacionalmente torpe, sino que destruía el contexto temporal y la sincronía entre la acción visual y el suceso acústico [1].

Una segunda generación de transición surgió a principios de 2025 con propuestas académicas como **VLM2Vec** y arquitecturas de interacción tardía como **ColQwen** [1]. El mérito científico de *VLM2Vec* (construido sobre modelos de lenguaje visuales genéricos como LLaVA y Phi) consistió en demostrar empíricamente cómo transformar un Modelo de Lenguaje Visual (*Vision-Language Model*, VLM) originalmente concebido para la generación autorregresiva de texto en un extractor masivo de *embeddings* multimodales para texto, imagen y vídeo. En paralelo, sistemas como *ColQwen* demostraron que era posible eliminar los motores OCR intermedios proyectando directamente las teselas visuales hacia el espacio latente del lenguaje [1].

Finalmente, entre finales de 2024 y el primer trimestre de 2026 se consolida la tercera generación: los **Modelos de Embedding Omnimodales (*Omni-modality*)** [2, 6]. Liderados en el ámbito comercial por **Gemini Embedding 2** (Google) y en el ecosistema de pesos abiertos por la familia **Qwen3-VL-Embedding** (Alibaba) y **EmbeddingGemma 2** (Google DeepMind), estos sistemas han sido diseñados desde su pre-entrenamiento para ingerir de forma nativa texto, imágenes, secuencias de vídeo, pistas de audio y documentos estructurados (archivos PDF) proyectándolos directamente en un único vector denso sin pasos de traducción textual intermedios [2, 6].

La Figura 4 sintetiza gráficamente la transición arquitectónica a través de estas tres generaciones de modelos de representación.

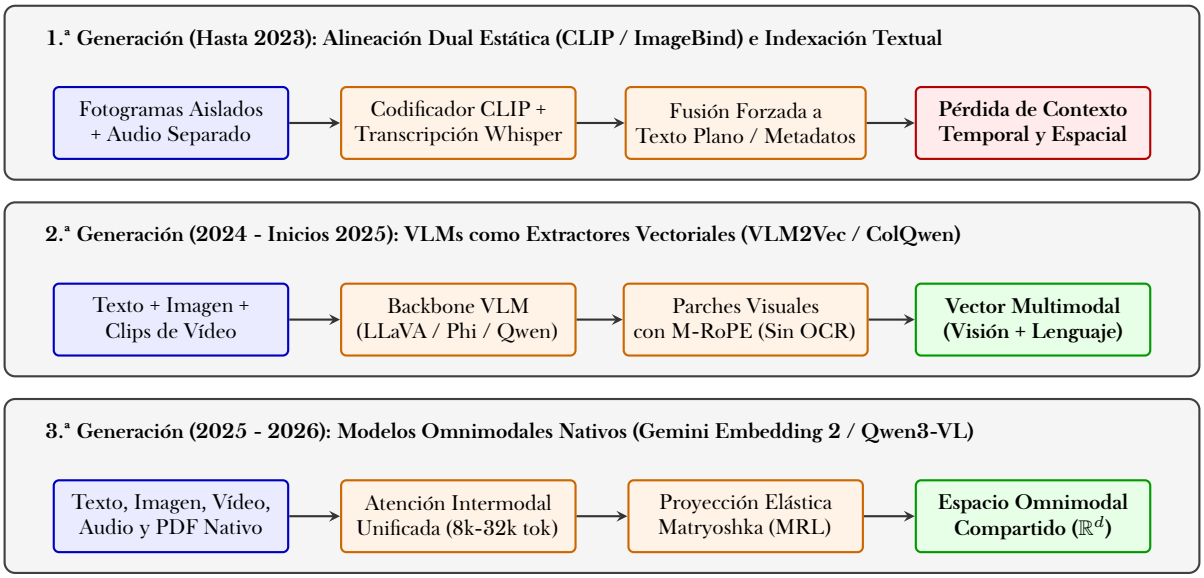


Figura 4: Evolución taxonómica de los modelos de representación vectorial: desde la alineación estática dependiente de transcripción textual (CLIP/Whisper [4, 5]) hasta los modelos omnimodales nativos de tercera generación capaces de proyectar texto, imagen, vídeo, audio y PDF en un único clúster topológico [1, 2, 6].

2.2.2 Tokenización Visual, Codificación M-RoPE y Agregación Temporal de Fotogramas

A nivel interno, el funcionamiento de un modelo de *embedding* multimodal contemporáneo se apoya en la unificación de la representación de entrada mediante teselas o parches (*visual patches*) procesados por arquitecturas inspiradas en *Vision Transformers* (ViT) [1]. Cuando el sistema recibe un fotograma de vídeo o una página de un documento visual a su resolución nativa, divide la matriz bidimensional de píxeles en una secuencia de sub-bloques que se proyectan linealmente como *tokens* visuales dentro de la misma secuencia de entrada que los *tokens* léxicos del texto [1].

Para evitar que esta linealización destruya la geometría bidimensional de la imagen o la secuencia temporal del vídeo, las arquitecturas modernas incorporan ***Multimodal Rotary Position Embeddings*** **(M-RoPE)** [1]. M-RoPE codifica directamente la información posicional bidimensional y temporal sobre las representaciones latentes de cada parche visual, lo que resulta determinante al procesar fotogramas de vídeo con resoluciones variables, garantizando que las relaciones espaciales entre los objetos visuales (como un bloque de código fuente en una esquina de la pantalla y el ponente en el lado opuesto) queden alineadas en el mismo espacio vectorial que los *tokens* del Modelo de Lenguaje Grande (LLM) subyacente [1].

Un aspecto arquitectónico fundamental en la vectorización de fragmentos de vídeo (`VideoChunk`) radica en el mecanismo de **agregación temporal de múltiples fotogramas** [2]. Cuando un algoritmo de segmentación semántica adaptativa (como *Video-RAC*) extrae tres fotogramas representativos de un plano (el fotograma inicial, el de máxima entropía visual y el fotograma final), existen dos formas opuestas de invocar al modelo fundacional [2, 7]:

1. **Codificación Independiente por Fotograma (Incorrecta para cohesión de *chunk*):** Si los tres foto-

gramas se envían a la API como tres objetos de contenido separados, el modelo genera tres vectores independientes ($\mathbf{v}_1, \mathbf{v}_2, \mathbf{v}_3 \in \mathbb{R}^d$). Esto triplica el coste de almacenamiento en la base de datos vectorial y obliga a realizar promedios heurísticos posteriores que diluyen la semántica temporal del plano.

2. **Agregación Nativa en un Único Objeto `Content` (Estado del Arte):** En arquitecturas como *Gemini Embedding 2*, al codificar los tres fotogramas clave en memoria (por ejemplo, en formato JPEG) e inyectarlos como múltiples objetos `Part` dentro de una única estructura `types.Content`, el mecanismo de atención interna del modelo evalúa conjuntamente la evolución entre los tres instantes y devuelve **un único *embedding* denso agregado** $\mathbf{v}_{\text{chunk}} \in \mathbb{R}^d$ que sintetiza el significado completo del evento [2].

La Figura 5 ilustra cómo la agrupación de múltiples partes multimedia dentro de un mismo contenedor `Content` permite condensar la dinámica del fragmento temporal en un único punto del espacio latente.

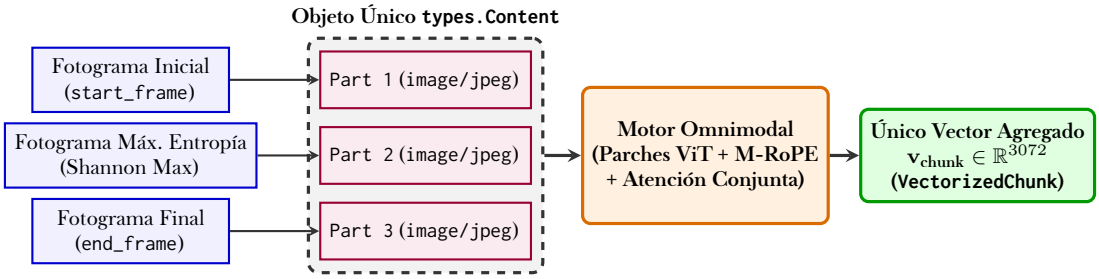


Figura 5: Mecanismo de agregación intermodal en *Gemini Embedding 2*: los tres fotogramas representativos de un `VideoChunk` se empaquetan como objetos `Part` individuales dentro de un único contenedor `types.Content`, permitiendo que el modelo fusione la información temporal y devuelva un único vector denso por fragmento [2, 7].

2.2.3 Arquitecturas de Doble Torre (*Bi-Encoder*) para Recuperación Masiva Precomputada

En el diseño de sistemas de recuperación de información sobre grandes volúmenes de vídeo, la primera etapa del procesamiento recae sobre los modelos con **arquitectura de Doble Torre (*Bi-Encoder*)**, representados en el estado del arte por sistemas como *Gemini Embedding 2*, *Qwen3-VL-Embedding* (en sus variantes de 2B y 8B de parámetros) y *EmbeddingGemma 2* [2, 6].

La característica definitoria de un *Bi-Encoder* es que procesa de manera completamente independiente el documento multimedia candidato D_i (un fragmento de vídeo, una imagen o una pista de audio) y la consulta textual del usuario Q [3, 6]. Matemáticamente, sean f_θ^{doc} y f_θ^{query} las funciones de proyección de ambas torres (que en los modelos fundacionales modernos suelen compartir los mismos pesos neuronales θ, diferenciándose únicamente por el prefijo de instrucción de entrada), según se define en la Ecuación (7) [9]:

$$\mathbf{v}_i = f_\theta^{\text{doc}}(D_i) \in \mathbb{R}^d, \qquad \mathbf{q} = f_\theta^{\text{query}}(P_{\text{task}} \oplus Q) \in \mathbb{R}^d \tag{7}$$

donde P_{task} representa un prefijo corto de instrucción de tarea que se antepone exclusivamente a las entradas de texto para orientar la consulta en el espacio latente (técnica empleada explícitamente en modelos como *EmbeddingGemma 2* y *Qwen3-VL-Embedding*, pero que no se aplica a las imágenes, vídeos ni audios) [6].

Esta independencia algebraica entre $\mathbf{v}_i$ y $\mathbf{q}$ constituye la piedra angular de la escalabilidad industrial por tres motivos fundamentales [3]:

1. **Indexación Pasiva *Offline*:** Permite pre-computar una sola vez los vectores $\mathbf{v}_i$ de un archivo histórico compuesto por millones de horas de vídeo y almacenarlos pasivamente en bases de datos vectoriales indexadas estáticamente como Qdrant o Milvus [3].
2. **Complejidad de Inferencia Constante respecto al Modelo:** Cuando el usuario introduce una búsqueda en lenguaje natural en tiempo de ejecución, la red neuronal profunda solo se ejecuta una vez para codificar la consulta $\mathbf{q}$ [3].
3. **Maximización de la Tasa de Recuerdo (*Recall*) a Escala de Petabytes:** La recuperación se reduce a comparar el vector $\mathbf{q}$ recién generado contra los millones de vectores pre-computados mediante operaciones vectoriales ultrarrápidas (como el producto punto escalar o la similitud del coseno), logrando tiempos de respuesta de milisegundos [3, 6].

2.2.4 Arquitecturas de Torre Única (*Cross-Encoder / Reranker*) y Atención Cruzada

A pesar de su extraordinaria velocidad, los modelos *Bi-Encoder* sufren una limitación estructural derivada de comprimir todo un fragmento de vídeo en un único vector estático $\mathbf{v}_i$ sin conocer de antemano qué preguntará el usuario [3, 6]. Al no existir interacción directa entre los *tokens* de la consulta y los parches visuales del vídeo durante la codificación, ciertos detalles composicionales finos o textos pequeños en pantalla pueden quedar diluidos dentro de la representación global.

Para resolver esta pérdida de precisión sin sacrificar la escalabilidad, el estado del arte introduce los modelos con **arquitectura de Torre Única (*Cross-Encoder*)**, denominados operativamente ***Rerankers*** (reordenadores) o verificadores visuales de segunda etapa, entre los que destacan **Qwen3-VL-Reranker** (en variantes de 2B y 8B) y el uso de modelos ligeros de visión-lenguaje como **Gemini 1.5 Flash Lite** o **Gemini Flash 3.5** en tareas de validación visual [3, 6].

A diferencia del *Bi-Encoder*, un *Cross-Encoder* no genera vectores independientes ni permite pre-computar representaciones de los vídeos [3]. En su lugar, recibe simultáneamente en una única secuencia concatenada la consulta del usuario Q y el fragmento multimedia candidato D_k, procesándolos conjuntamente a través de todas las capas de **Atención Cruzada (*Cross-Attention*)** del *Transformer*, como se formula en la Ecuación (8) [3, 6]:

$$s_k = g_\phi(Q, D_k) = \sigma(\mathbf{W}_{\text{head}} \cdot \text{CrossAttention}_\phi([\text{Tokens}(Q);\ \text{Patches}(D_k)])) \in [0, 1] \tag{8}$$

Mediante el mecanismo de *Cross-Attention*, cada *token* de la pregunta del usuario (por ejemplo, la palabra *"diccionarios"*) atiende directamente a cada parche visual específico de los fotogramas del candidato

D_k (por ejemplo, la región de la pantalla donde aparece escrito el código `dict {"name": "Brais"}`). Finalmente, como refleja el Cuadro de especificaciones de la familia Qwen3-VL, el *Cross-Encoder* no devuelve un vector de *embedding*, sino una **probabilidad discreta en el rango** $[0, 1]$ (o un veredicto de validación visual) que cuantifica la relevancia exacta del candidato respecto a la consulta [6].

La Figura 6 contrasta detalladamente la topología interna de un *Bi-Encoder* frente a la de un *Cross-Encoder* [3, 6].

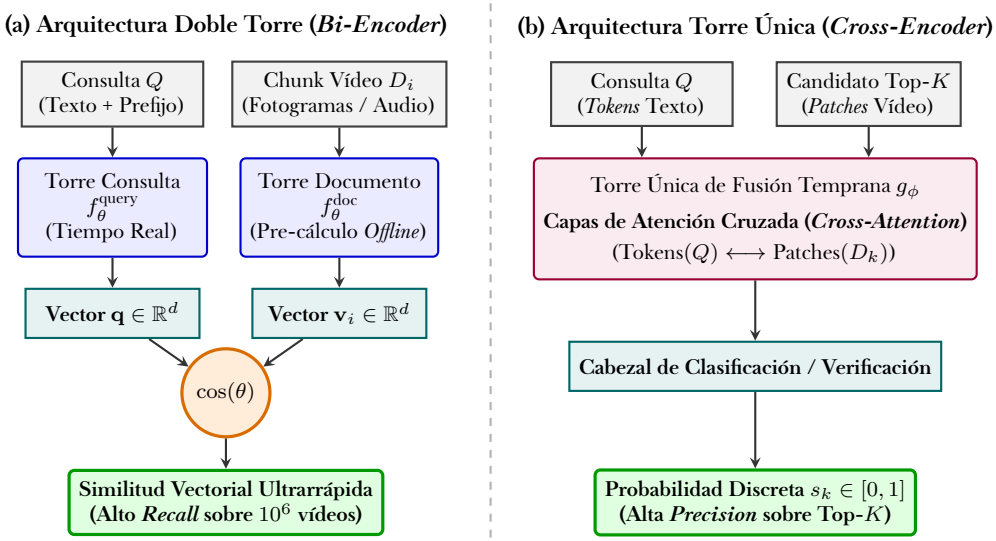


Figura 6: Comparativa estructural entre (a) un *Bi-Encoder* de doble torre que codifica consulta y vídeo de forma independiente para indexación pasiva en bases de datos vectoriales, y (b) un *Cross-Encoder* (*Reranker*) que aplica atención cruzada conjunta sobre el subconjunto de candidatos recuperados [3, 6].

2.2.5 Eficiencia Escalable: El Paradigma de Búsqueda Asimétrica en Dos Etapas (Regla 99 % / 1 %)

El resurgimiento y perfeccionamiento de la dualidad entre *Bi-Encoders* (para recuperación inicial) y *Cross-Encoders* o verificadores VLM (para refinamiento) responde directamente a los límites físicos de energía, latencia y procesamiento de hardware en sistemas de vídeo a gran escala [3, 22]. Computar de forma cruzada cada búsqueda del usuario contra terabytes de fragmentos de vídeo utilizando modelos generativos masivos (como intentar emplear un modelo tipo Llama-Vision o Gemini Pro para comparar fotograma a fotograma todo un repositorio) resulta técnica y financieramente inasumible [3].

Para resolver esta dicotomía, el estado del arte de 2026 establece una **aproximación asimétrica radical en dos etapas** conocida como la regla del 99 %/1 % [3, 6]:

- **Primera Etapa - Filtrado Masivo del 99 % (*Bi-Encoder* + Base de Datos Vectorial):** Se delega el 99 % del descarte de datos a una vectorización paralela, densa e independiente (empleando modelos como *Gemini Embedding 2* o *Qwen3-VL-Embedding*) almacenada en un motor vectorial como Qdrant [2, 6]. Ante una consulta del usuario, la búsqueda inicial se resuelve mediante un cálculo vectorial de similitud del coseno sobre índices estáticos, reduciendo instantáneamente miles de fragmentos a una lista acotada de los K mejores candidatos (por ejemplo, Top-$K = 5$ o Top-$K = 20$) [3].

- **Segunda Etapa - Refinamiento Intensivo del 1 % (*Cross-Encoder* / Verificación Visual):** El cómputo pesado, iterativo y apoyado en mecanismos de atención cruzada se reserva exclusivamente para ese 1 % de resultados candidatos recuperados en la primera fase [3]. En esta etapa, bien un modelo específico de reordenación (*Qwen3-VL-Reranker*) recalcula la probabilidad exacta de coincidencia $[0, 1]$, o bien un verificador visual ligero (*Gemini 1.5 Flash Lite* / *Gemini Flash 3.5*) inspecciona los fotogramas del *chunk* candidato para descartar falsos positivos antes de presentar los intervalos de tiempo (`start_time` - `end_time`) definitivos al usuario [3, 6].

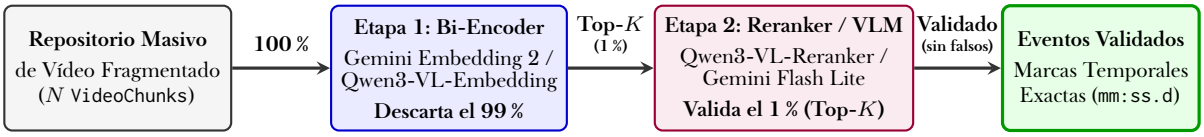


Figura 7: Flujo del paradigma de búsqueda asimétrica en dos etapas (Regla 99 % / 1 %): el *Bi-Encoder* filtra masivamente el repositorio mediante similitud del coseno en Qdrant y el *Cross-Encoder* o verificador visual inspecciona únicamente el subconjunto Top-K para garantizar máxima precisión con latencia mínima [3, 6].

El Cuadro 2 resume las diferencias arquitectónicas, operativas y matemáticas entre las tres familias de modelos que intervienen en el ciclo de recuperación y validación multimodal.

Tabla 2: Comparativa técnica entre arquitecturas *Bi-Encoder*, *Cross-Encoder* (*Reranker*) y Verificadores VLM Generativos en sistemas Video RAG [2, 3, 6].

Atributo Técnico	**Bi-Encoder (1.ª Etapa)**	**Cross-Encoder (2.ª Etapa)**	**Verificador VLM (2.ª Etapa)**
Modelos de referencia	Gemini Emb. 2 / Qwen3-Emb	Qwen3-VL-Reranker (2B/8B)	Gemini 1.5 Flash Lite / Flash 3.5
Interacción Consulta-Vídeo	Tardía (Producto escalar)	Temprana (*Cross-Attention*)	Generativa multimodal guiada
Pre-cálculo *Offline*	Sí (Indexado en Qdrant)	No (Cálculo en tiempo real)	No (Evaluación bajo demanda)
Formato de salida	Vector $\mathbf{v} \in \mathbb{R}^d$ (MRL)	Probabilidad discreta $[0, 1]$	Score + Justificación visual
Volumen procesado	100 % corpus (filtra 99 %)	1 % candidatos (Top-K)	1 % candidatos (Top-K)
Métrica optimizada	Recuerdo (*Recall@K*)	Precisión (*nDCG@K* / *Hit@1*)	Descarte Falsos Positivos

Como demuestra esta taxonomía, la combinación de una primera etapa de vectorización densa omnimodal con una segunda etapa de atención cruzada y verificación visual constituye la arquitectura idónea para localizar eventos específicos en vídeo sin incurrir en costes computacionales prohibitivos [3, 6].

2.3 Estudio Profundo de Google Gemini Embedding 2

2.3.1 Arquitectura Omnimodal Unificada y Configuración del Espacio de Salida

Dentro de la macrotendencia de modelos fundacionales propietarios accesibles mediante interfaz de programación de aplicaciones (API), **Gemini Embedding 2** (identificado técnicamente en el catálogo de desarrollo como `gemini-embedding-2-preview`) constituye el hito arquitectónico de Google hacia la vectorización omnimodal nativa [2, 23]. A diferencia de las arquitecturas multimodales de primera generación que requerían codificadores separados y proyecciones post-hoc para cada tipo de dato, `gemini-embedding-2-preview` ha sido entrenado de extremo a extremo para ingerir texto libre, imágenes estáticas, secuencias de vídeo, pistas de audio y documentos estructurados (archivos PDF) proyectándolos directamente en un único espacio vectorial continuo [2, 24].

Esta unificación topológica implica que los vectores generados a partir de una consulta textual en lenguaje natural habitan exactamente la misma variedad geométrica que los vectores extraídos de los fotogramas de un vídeo o de una locución hablada [2]. De este modo, la localización de un evento específico dentro de un repositorio audiovisual prescinde por completo de etiquetas manuales previas o de traducciones intermedias a texto plano, resolviéndose mediante el cálculo directo de la similitud del coseno entre la consulta vectorizada y los fragmentos multimedia indexados [2].

En cuanto a la configuración algebraica de sus tensores de salida, el modelo incorpora soporte nativo para *Matryoshka Representation Learning* (MRL) a través del parámetro de configuración `output_dimensionality` integrado en la estructura `types.EmbedContentConfig` del SDK oficial `google-genai` [2]. Aunque cuando se invoca el punto de enlace sin parámetros adicionales para pruebas básicas de texto el sistema devuelve por defecto un vector compacto de 768 dimensiones, la arquitectura permite solicitar representaciones de mayor densidad (destacando los estándares operativos de 768, 1,536 y el máximo nativo de 3,072 dimensiones) [2].

Desde la perspectiva de la precisión numérica, existe una distinción crítica en el post-procesamiento de los vectores devueltos por la API según la dimensionalidad seleccionada [2]:

- **Operación en Dimensionalidad Máxima ($d = 3{,}072$):** Los vectores de 3,072 números en coma flotante devueltos por `response.embeddings[0].values` se entregan ya normalizados a longitud unitaria por el propio motor de inferencia de Google, ofreciendo la máxima capacidad de discriminación semántica para desambiguar escenas visualmente similares [2].
- **Operación en Dimensionalidades Truncadas ($d < 3{,}072$):** Cuando el ingeniero configura `output_dimensionali` en valores inferiores a 3,072 (tales como 768 o 1,536) con el objetivo de reducir el consumo de memoria en la base de datos vectorial, el recorte de las coordenadas finales altera la magnitud unitaria del vector original [2]. Por consiguiente, tal y como recomienda la documentación técnica de Google, el cliente de software debe calcular explícitamente la norma euclídea $\|\mathbf{v}\|_2 = \sqrt{\sum_{j=1}^{d} v_j^2}$ (por ejemplo, mediante `np.linalg.norm(values)`) y dividir cada componente entre dicha norma ($\hat{v}_j = v_j / \|\mathbf{v}\|_2$) siempre que $\|\mathbf{v}\|_2 > 0$, garantizando así la validez geométrica de la similitud del coseno en etapas posteriores [2].

2.3.2 Paradigmas de Ingestión Multimedia en el SDK `google-genai`: API Files frente a Agregación en Memoria

La integración programática con el modelo `gemini-embedding-2-preview` a través de la librería oficial `google-genai` habilita dos arquitecturas de acoplamiento de datos claramente diferenciadas según la naturaleza del flujo de procesamiento [2, 23]:

2.3.2.1 Ingestión de Archivos Completos mediante la API de Almacenamiento Temporal (`client.files.upload`):

Orientada al procesamiento directo de ficheros pesados almacenados en disco (clips de vídeo en formato `video/mp4`, pistas de audio, imágenes de alta resolución o documentos PDF) [2, 23]: En este patrón arquitectónico, el sistema no envía los bytes crudos directamente en la llamada de inferencia, sino que ejecuta un protocolo en tres fases [2]:

1. *Transferencia y generación de URI:* Se invoca el método `client.files.upload(file=file_path, config={'mime_type': mime_type})`, el cual aloja temporalmente el recurso multimedia en la infraestructura de Google AI Studio y devuelve un objeto con un identificador único (`uploaded_file.name`) y una referencia uniforme (`uploaded_file.uri`) [2].
2. *Bucle de sondeo de estado (Polling) para recursos temporales:* Dado que los contenedores de vídeo (`video/*`) y audio (`audio/*`) requieren una decodificación e indexación previa en los servidores de Google antes de poder ser leídos por la red neuronal, el archivo recién subido permanece inicialmente en un estado transitorio [2]. El cliente debe implementar un bucle de espera activa controlada (consultando `client.files.get(name=uploaded_file.name)` cada 2 segundos) hasta verificar que la propiedad `file_info.state.name` transiciona al estado `."ACTIVE"` (o abortar lanzando una excepción si deriva al estado `"FAILED"`) [2].
3. *Extracción vectorial y limpieza:* Una vez activo el recurso, se pasa la referencia `uploaded_file` al parámetro `contents` de `client.models.embed_content`, pudiendo invocar posteriormente `client.files.delete(r` para liberar el almacenamiento remoto [2].

2.3.2.2 Ingestión por Agregación de Fotogramas Clave en Memoria (`types.Content` y `types.Part.from`

Si bien la API Files resulta idónea para clips pre-cortados en disco, en una arquitectura de fragmentación semántica adaptativa como *Video-RAC* (donde un vídeo largo se divide dinámicamente en memoria en decenas o cientos de objetos `VideoChunk` mediante OpenCV) escribir cada sub-clip en disco, subirlo a la API Files y esperar 2 segundos de sondeo por cada fragmento introduciría una latencia de entrada/salida inasumible [2, 7].

Para optimizar el rendimiento y el consumo de ancho de banda, el diseño avanzado implementado en el motor de vectorización (`GeminiMultimodalVectorizer`) toma las matrices NumPy (`NDArray[np.uint8]` en espacio de color BGR) correspondientes a los tres fotogramas representativos extraídos de cada `VideoChunk` y los codifica directamente en memoria RAM al estándar JPEG mediante `cv2.imencode(".jpg", frame, [cv2.IMWRITE_JPEG_QUALITY, jpeg_quality])` (con una calidad por defecto de 90 sobre 100) [2]. Cada búfer de bytes resultante se encapsula en un objeto `types.Part.from_bytes(data=buffer.tobytes(),`

`mime_type=ïmage/jpeg"`) y la lista completa de partes del fragmento se empaqueta en una única instancia `types.Content(parts=parts)` que se transmite de forma asíncrona mediante `client.aio.models.embed_content` [2].

La ventaja crucial de este segundo mecanismo es doble: elimina por completo la creación de archivos temporales y el bucle de espera de la API Files, y consigue que el modelo fusione internamente todos los objetos `Part` del contenedor `Content`, devolviendo **un único vector agregado** que representa la totalidad del *chunk* temporal [2].

La Figura 8 ilustra y compara ambos flujos de acoplamiento multimedia dentro de la arquitectura del SDK `google-genai`.

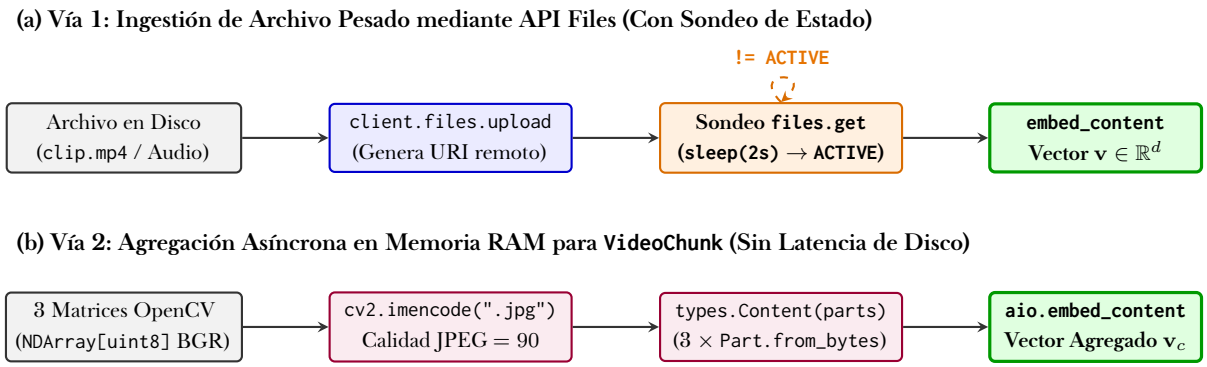


Figura 8: Comparativa técnica de los dos flujos de ingestión en el SDK `google-genai`: (a) subida de archivos completos a la API Files con bucle de espera activa hasta alcanzar el estado `ACTIVE`, y (b) codificación JPEG en memoria de múltiples fotogramas empaquetados en un único objeto `types.Content` para llamadas asíncronas de baja latencia [2].

Asimismo, para cerrar el ciclo de búsqueda cruzada y multimodal, el mismo componente vectorizador expone el método `embed_query` (respaldado por `build_query_payload` y manteniendo `embed_text` por retrocompatibilidad), capaz de empaquetar en un único contenedor `types.Content` cualquier combinación de consulta textual, ficheros `.txt`, imágenes de referencia codificadas a JPEG (`types.Part.from_bytes`) y transcripciones de pistas de audio de entrada obtenidas con `transcribe_query_audios`. Al invocar `aio.models.embed_content` en una sola llamada de red con idéntica configuración dimensional (`output_dimensionality`) y normalización L_2, el sistema garantiza la compatibilidad algebraica exacta entre los fragmentos de vídeo indexados y las consultas multimodales del usuario [2].

2.3.3 Modelado Matemático de Cuotas Operativas (RPM, TPM y RPD) y Planificación de Capacidad

Un aspecto crítico en el diseño de arquitecturas de ingeniería basadas en modelos fundacionales en la nube es la caracterización matemática de los límites de tasa (*Rate Limits*) impuestos por el proveedor del servicio [2]. En el entorno de experimentación y validación de Google AI Studio (capa gratuita o *Free Tier*) para el modelo `gemini-embedding-2-preview`, la infraestructura impone simultáneamente tres umbrales restrictivos [2]:

- **RPM (*Requests Per Minute*):** Límite máximo de $\text{RPM}_{\text{máx}} = 100$ peticiones por minuto [2].

- **TPM (*Tokens Per Minute*):** Límite máximo de $\text{TPM}_{\text{máx}} = 30{,}000$ *tokens* procesados por minuto [2].
- **RPD (*Requests Per Day*):** Límite máximo acumulado de $\text{RPD}_{\text{máx}} = 1{,}000$ peticiones por día [2].

Un análisis superficial podría sugerir que el sistema es capaz de vectorizar 100 fragmentos de vídeo (`VideoChunk`) por minuto atenidos al umbral de $\text{RPM}_{\text{máx}} = 100$. Sin embargo, el modelado riguroso de la carga útil multimedia demuestra que el verdadero cuello de botella por minuto reside en el consumo de *tokens* ($\text{TPM}_{\text{máx}}$) [2]. Cada imagen o fotograma enviado a los modelos de la familia Gemini consume una tasa base de aproximadamente $T_{\text{img}} \approx 258$–$300$ *tokens* [2]. Dado que el algoritmo de selección por entropía extrae $N_{\text{frames}} = 3$ fotogramas clave por cada `VideoChunk`, el coste en *tokens* de una única petición de fragmento asciende según la Ecuación (9) a:

$$T_{\text{chunk}} = N_{\text{frames}} \times T_{\text{img}} \approx 3 \times 300 = 900 \text{ tokens/chunk} \tag{9}$$

Si el orquestador enviase el máximo permitido de 100 peticiones en un minuto, la demanda de caudal ascendería a $100 \times 900 = 90{,}000$ *tokens*/minuto, triplicando el límite de $\text{TPM}_{\text{máx}} = 30{,}000$ y provocando el rechazo inmediato de las peticiones con código de error HTTP `429 (Too Many Requests)`. En consecuencia, el caudal máximo efectivo sostenido ($\lambda_{\text{efectivo}}$) en fragmentos por minuto queda determinado según la Ecuación (10) por el mínimo entre la restricción de peticiones y la restricción de *tokens*:

$$\lambda_{\text{efectivo}} = \text{mín}\left(\text{RPM}_{\text{máx}}, \left\lfloor \frac{\text{TPM}_{\text{máx}}}{T_{\text{chunk}}} \right\rfloor\right) = \text{mín}\left(100, \left\lfloor \frac{30{,}000}{900} \right\rfloor\right) \approx 33 \text{ chunks/minuto} \tag{10}$$

Esto restringe la velocidad de ingestión real a un rango de entre 30 y 35 fragmentos de vídeo por minuto, lo que significa que un vídeo de prueba que genere entre 100 y 200 *chunks* requerirá entre 3 y 6 minutos de procesamiento continuo.

De forma análoga, el límite diario de $\text{RPD}_{\text{máx}} = 1{,}000$ peticiones acota la duración máxima de vídeo que puede indexarse en una jornada de experimentación. Bajo una segmentación semántica adaptativa (*Video-RAC*), un vídeo educativo o divulgativo promedio experimenta un cambio de plano cada $\bar{d}_{\text{plano}} \in [3, 7]$ segundos. Asumiendo una media conservadora de $\bar{d}_{\text{plano}} = 5$ segundos por fragmento, cada minuto de metraje produce $C_{\text{min}} = 60/5 = 12$ *chunks*/minuto, lo que arroja un límite teórico absoluto de duración diaria ($D_{\text{máx}}$) formulado en la Ecuación (11):

$$D_{\text{máx}} = \frac{\text{RPD}_{\text{máx}}}{C_{\text{min}}} = \frac{1{,}000 \text{ chunks/día}}{12 \text{ chunks/minuto}} \approx 83{,}3 \text{ minutos/día} \tag{11}$$

Para evitar que el procesamiento accidental de un único archivo de larga duración agote el 100 % de la cuota diaria e impida realizar búsquedas o iteraciones de calibración durante el resto del día, la arquitectura de ingeniería implementa dos salvaguardas: acotar los vídeos de validación de la Prueba de Concepto a clips de entre **5 y 15 minutos** (que generan entre 100 y 200 *chunks*, consumiendo apenas un 10 %–20 % del RPD), e incorporar en el orquestador un cortafuegos estático (*fail-safe*) definido por la

constante `MAX_CHUNKS_PER_VIDEO = 800`. Si tras la detección de bordes de plano con OpenCV el vídeo supera los 800 fragmentos, el sistema aborta preventivamente lanzando la excepción `QuotaSafetyError` antes de extraer fotogramas ni consumir una sola petición de la API.

La Figura 9 esquematiza la interacción entre los tres umbrales de cuota y los mecanismos de protección implementados en el flujo de ingestión.

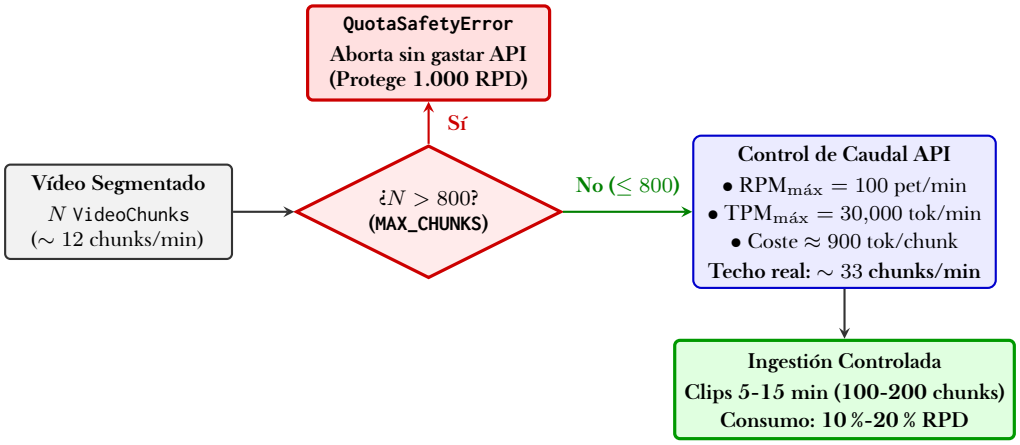


Figura 9: Modelado de capacidad y cortafuegos de cuota para `gemini-embedding-2-preview`: la validación previa aborta ejecuciones que superen los 800 fragmentos para proteger el límite diario (1.000 RPD), mientras que la densidad de $\sim$ 900 *tokens* por fragmento fija el caudal efectivo en $\sim$ 33 *chunks*/minuto bajo el techo de 30.000 TPM [2].

2.3.4 Ingeniería de Resiliencia: Control de Concurrencia, Ventana de *Backoff* de 60 Segundos y Degradación Elegante

Dado que el procesamiento de vídeo combina operaciones intensivas de CPU (decodificación de fotogramas y cálculo de mapas gaussianos SSIM en OpenCV) con operaciones asíncronas de red (llamadas HTTP a la API de Gemini), la arquitectura de software debe evitar tanto el bloqueo del bucle de eventos como la saturación de los servidores remotos.

En primer lugar, las funciones síncronas bloqueantes de visión por computador (`detect_shot_boundaries` y `extract_key_frames`) se delegan a hilos secundarios mediante `asyncio.to_thread`, liberando el hilo principal de Python para gestionar las conexiones de red. En segundo lugar, la emisión paralela de tareas de vectorización en `asyncio.gather` se regula mediante un semáforo asíncrono (`asyncio.Semaphore(max_concurrent`) calibrado específicamente en `max_concurrent_requests = 3` (reducido respecto a un valor inicial de 5) para impedir ráfagas instantáneas que desborden las cuotas por minuto.

Pese al uso del semáforo, dado que la latencia de respuesta de una petición individual es inferior a un segundo, tres corrutinas concurrentes pueden alcanzar el techo de 30,000 *tokens* antes de que se cumpla el minuto natural, recibiendo un error transitorio HTTP `429`. Para absorber este fenómeno sin perder datos (así como fallos transitorios de servidor catalogados en el conjunto inmutable `_RETRYABLE_CODES = {429, 500, 502, 503, 504}` y errores de red `OSError`/`TimeoutError`), el método privado `_embed` implementa un algoritmo de **reintento con espera exponencial (*Exponential Backoff*)**.

La calibración matemática de los parámetros de este *backoff* resulta determinante: si se utilizase una

espera base de $t_0 = 2,0$ s con $R = 3$ reintentos, la suma de las pausas sería únicamente de $2+4+8+16 = 28$ s (o $2+4+8 = 14$ s en tres esperas), tiempo insuficiente para que Google reinicie la ventana deslizante de 60 segundos del límite TPM. Por el contrario, al incrementar los parámetros por defecto a un retardo base de `retry_base_delay` $= 4,0$ s y un máximo de `max_retries` $= 4$ reintentos, el tiempo de espera en el intento $k \in \{0,1,2,3\}$ responde a $\Delta t_k = 4,0 \times 2^k$, generando una ventana acumulada calculada según la Ecuación (12):

$$T_{\text{espera total}} = \sum_{k=0}^{3} t_0 \cdot 2^k = 4,0 \times (2^0 + 2^1 + 2^2 + 2^3) = 4 + 8 + 16 + 32 = 60 \text{ segundos} \tag{12}$$

La Figura 10 muestra cómo esta progresión geométrica cubre exactamente los 60 segundos necesarios para garantizar la renovación de la cuota de *tokens* por minuto antes de agotar el último intento.

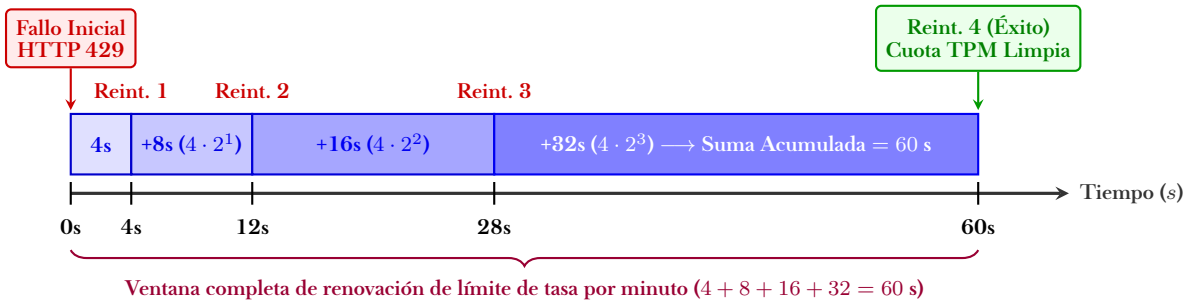


Figura 10: Cronograma temporal del algoritmo de *Exponential Backoff* calibrado (`retry_base_delay = 4.0`, `max_retries = 4`): la suma geométrica de las cuatro esperas alcanza exactamente 60 segundos, permitiendo superar bloqueos completos de la ventana de 30.000 TPM sin abortar la vectorización del fragmento.

Adicionalmente, el diseño incorpora dos mecanismos de **Degradación Elegante (*Graceful Degradation*)** en distintas capas del sistema para garantizar una tolerancia a fallos de nivel industrial:

1. **Aislamiento de fallos parciales en la fase de ingestión (`return_exceptions=True`):** Si tras agotar los 60 segundos de reintentos un fragmento puntual sigue fallando (lanzando `VectorizationError`), la invocación `asyncio.gather(* tasks, return_exceptions=True)` evita que se cancele el lote completo. El orquestador captura la excepción de ese único *chunk*, la documenta en `execution.log` mediante el auditor (`RunAuditor`) y conserva intactos todos los demás fragmentos vectorizados del vídeo.

2. **Respaldo vectorial ante caídas `503 UNAVAILABLE` en la segunda etapa de reordenación visual:** Durante la fase de búsqueda y refinamiento con modelos ligeros de visión-lenguaje (como `gemini-3.5-flash-lite`), los servidores de Google pueden experimentar picos temporales de demanda devolviendo el error `503 UNAVAILABLE ("This model is currently experiencing high demand")`. Cuando esto ocurre y se agotan los reintentos del verificador secundario, el sistema no colapsa ni devuelve un error al usuario: intercepta la indisponibilidad, emite un aviso informando que la reordenación visual no está disponible temporalmente y **conserva automáticamente el orden vectorial original devuelto por Qdrant** en la primera etapa.

La Figura 11 representa este flujo de continuidad operativa y degradación elegante en ambas fases de la arquitectura.

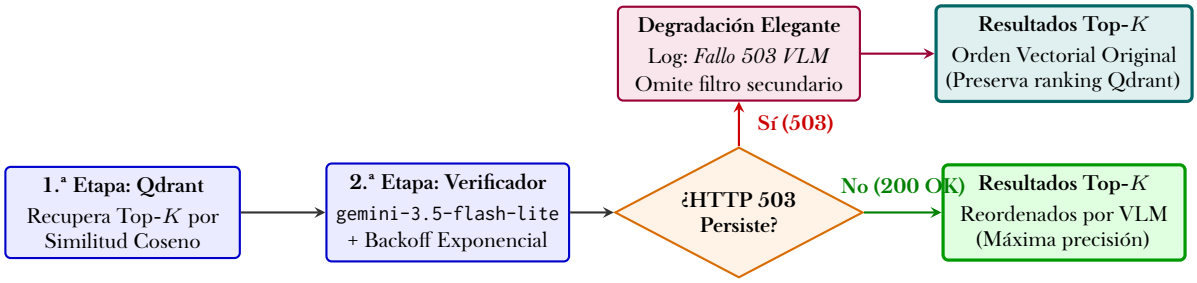


Figura 11: Arquitectura de Degradación Elegante (*Graceful Degradation*) durante la búsqueda: si los servidores del modelo secundario (`gemini-3.5-flash-lite`) sufren picos de saturación (`503 UNAVAILABLE`) tras agotar los reintentos, el sistema conmuta limpiamente al orden de similitud vectorial calculado por Qdrant con *Gemini Embedding 2*.

El Cuadro 3 sintetiza los parámetros definitivos de ingeniería calibrados en el sistema para maximizar la fiabilidad bajo las restricciones operativas de la API de Google.

Tabla 3: Parámetros técnicos calibrados en los módulos de vectorización y orquestación para operar con `gemini-embedding-2-preview` sin colapsos de cuota [2].

Parámetro	Módulo	Valor	Justificación Técnica / MLOps
`model`	`gemini_vectorizer.py`	`gemini-embedding-2`	Modelo omnimodal nativo (espacio compartido)
`output_dimensionality`	`gemini_vectorizer.py`	3,072, 1,536 o 768	MRL con normalización L_2 (`truncate_and_normalize_mrl`)
`jpeg_quality`	`gemini_vectorizer.py`	90 (escala 1-100)	Alta fidelidad visual en RAM sin artefactos
`max_concurrent_requests`	`pipeline.py`	3 (reducido de 5)	Limita peticiones concurrentes en semáforo
`retry_base_delay`	`gemini_vectorizer.py`	4, 0 s (antes 2, 0 s)	Base geométrica para cubrir ventana TPM
`max_retries`	`gemini_vectorizer.py`	4 (antes 3)	Suma $4+8+16+32=60$ s ante error HTTP
`MAX_CHUNKS_PER_VIDEO`	`pipeline.py`	800 fragmentos	Cortafuegos `QuotaSafetyError` (protege RPD)

2.4 Ecosistema de Modelos Alternativos: Pesos Abiertos (*Open-Weights*) y Ejecución Local

2.4.1 Las Dos Macrotendencias de 2026: APIs Propietarias Hiper-escaladas frente a Pesos Abiertos

En el diseño de una plataforma de recuperación profunda de vídeo, la elección de la arquitectura del modelo de *embedding* subyacente constituye el factor crítico que define los límites de escalabilidad, la precisión en la localización de eventos y el coste computacional del sistema [2]. El estado del arte tecnológico en 2026 se estructura en torno a dos macrotendencias complementarias [2, 6]:

1. **Modelos Propietarios Hiper-escalados vía API (*Cloud*):** Representados por servicios como *Google Gemini Embedding 2* (`gemini-embedding-2-preview`), donde la complejidad de la infraestructura de aceleradores (TPUs/GPUs) queda abstraída tras llamadas HTTP, ofreciendo espacios vectoriales masivos de hasta 3,072 dimensiones con soporte nativo para vídeo, audio, imagen, texto y documentos PDF [2, 24].
2. **Ecosistemas de Pesos Abiertos (*Open-Weights*) y Despliegue Local:** Integrados por familias de modelos cuyos pesos neuronales son públicamente accesibles en repositorios como Hugging Face (destacando la serie **Qwen3-VL-Embedding** y **Qwen3-VL-Reranker**, arquitecturas como **ColQwen** y variantes ligeras de la familia **Gemma**), los cuales pueden ejecutarse en servidores locales propios o en instancias privadas sin depender de proveedores externos [1, 6, 9].

Un hallazgo fundamental documentado en la literatura del primer trimestre de 2026 es que las implementaciones de código abierto han logrado cerrar la brecha histórica de rendimiento frente a las APIs propietarias en tareas de recuperación multimodal [6]. Esta convergencia técnica tiene una repercusión directa en el ámbito universitario e industrial: posibilita el diseño de infraestructuras de investigación académica y de producción que no dependen del pago continuo por inferencia (*pay-per-token*) ni se ven estranguladas por límites diarios de peticiones, garantizando además la soberanía y privacidad absoluta sobre archivos de vídeo sensibles [6].

La Figura 12 presenta la taxonomía comparada de ambas corrientes tecnológicas dentro del panorama de modelos fundacionales multimodales de 2026.

2.4.2 La Familia Qwen3-VL: Recuperación y Reordenación Multimodal a Escala Masiva

En el espectro del software libre y los modelos de pesos abiertos, la referencia técnica para la recuperación de información multimodal viene marcada por el lanzamiento en enero de 2026 de la serie **Qwen3-VL-Embedding** y **Qwen3-VL-Reranker** por parte del grupo de investigación de Alibaba [6, 25, 26]. Publicados oficialmente en configuraciones de 2,000 millones (`Qwen/Qwen3-VL-Embedding-2B`) y 8,000 millones de parámetros (`Qwen/Qwen3-VL-Embedding-8B`), junto con adaptaciones especializadas en recuperación de documentos visuales (`tomaarsen/Qwen3-VL-Embedding-2B-vdr`), estos modelos abordan el problema del RAG sobre grandes repositorios de vídeo mediante un diseño acoplado en dos etapas que supera las limitaciones de latencia de los sistemas generativos directos [6, 27–29].

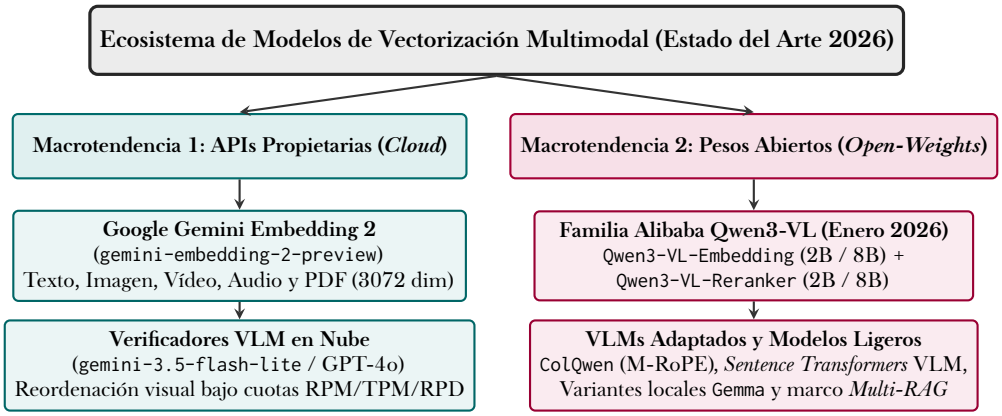


Figura 12: Clasificación de las arquitecturas de modelos fundacionales para Video RAG en 2026: coexistencia de modelos hiper-escalados servidos mediante API comercial frente a ecosistemas de pesos abiertos desplegables en infraestructura local [2, 6, 8].

2.4.2.1 Primera Etapa (`Qwen3-VL-Embedding`):

Actúa como un codificador denso de doble torre (*Bi-Encoder*) construido sobre la arquitectura base de visión y lenguaje de la familia Qwen3 [3, 6]. Gracias a la integración nativa de *Multimodal Rotary Position Embeddings* (M-RoPE), el modelo proyecta parches y teselas visuales de imágenes y secuencias de vídeo directamente hacia el mismo espacio semántico continuo del lenguaje sin requerir preprocesamiento OCR intermedio [1, 6]. Estos vectores se almacenan en bases de datos vectoriales como Qdrant o Milvus, permitiendo realizar búsquedas semánticas directas sobre vídeo bruto a gran velocidad [6].

2.4.2.2 Segunda Etapa (`Qwen3-VL-Reranker`) y Resolución de Consultas Composicionales:

El aporte arquitectónico más relevante de la investigación del equipo Qwen radica en demostrar empíricamente por qué un *Bi-Encoder* aislado resulta insuficiente ante consultas visualmente complejas [3, 6]. Los codificadores de doble torre tienden a degradar su precisión ante **consultas altamente composicionales**, perdiendo relaciones espaciales, direccionales o de atributos cruzados; un caso paradigmático documentado en la literatura es la búsqueda de un evento descrito como *"una persona vistiendo una chaqueta roja que se aleja de una furgoneta blanca"* [3]. En un *Bi-Encoder*, un fragmento de vídeo donde aparezca una furgoneta roja y una persona con chaqueta blanca acercándose a ella puede arrojar una similitud del coseno artificialmente alta debido a la presencia conjunta de los mismos sustantivos y colores en el vector global.

Para erradicar estos falsos positivos composicionales, `Qwen3-VL-Reranker` actúa como un filtro de alta precisión sobre el subconjunto top-K recuperado en la primera fase [3, 6]. Operando bajo una arquitectura de Torre Única (*Cross-Encoder*), el modelo ingiere simultáneamente la consulta textual Q y el fragmento multimedia candidato D_k, permitiendo que sus capas de atención cruzada (*Cross-Attention*) interactúen de manera bidireccional y profunda entre cada *token* léxico del texto y cada parche espacial de los fotogramas del vídeo [3, 6]. En lugar de emitir un vector de representación, la red finaliza prediciendo según la Ecuación (13) un ***logit* de probabilidad absoluta** que cuantifica la relevancia exacta de la coincidencia entre el evento visual y la estructura sintáctica de la consulta, maximizando la precisión en las primeras posiciones del ranking [3, 6].

$$P(\text{Relevante} \mid Q, D_k) = \sigma\big(z_{\text{logit}}(Q, D_k)\big) = \frac{1}{1 + \exp(-\mathbf{w}^\top \mathbf{h}_{\text{Cross-Attention}}(Q, D_k))} \tag{13}$$

La Figura 13 ilustra el mecanismo mediante el cual `Qwen3-VL-Reranker` desambigua relaciones espaciales y de atributos sobre los candidatos recuperados por `Qwen3-VL-Embedding`.

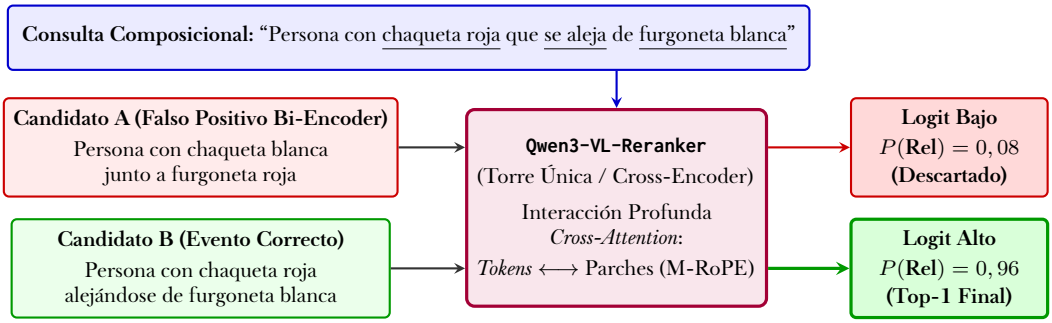


Figura 13: Resolución de consultas altamente composicionales mediante la arquitectura en dos etapas de la familia Qwen3-VL: mientras el *Bi-Encoder* puede recuperar candidatos con atributos invertidos, las capas de *Cross-Attention* de `Qwen3-VL-Reranker` vinculan cada adjetivo y acción con su parche visual exacto para predecir el *logit* de probabilidad real [3, 6].

2.4.3 Adaptación de Modelos Visuales (ColQwen y Sentence Transformers) y Entornos Restringidos (*Multi-RAG*)

Más allá de la serie Qwen3-VL, la comunidad investigadora ha consolidado metodologías reproducibles para entrenar y ajustar (*finetuning*) modelos de *embeddings* y *rerankers* multimodales a partir de arquitecturas de lenguaje visual preexistentes, tal y como documenta el marco de *Sentence Transformers* multimodales de Hugging Face [6, 9].

Dentro de esta línea destaca **ColQwen**, una arquitectura que integra directamente *Multimodal Rotary Position Embeddings* (M-RoPE) sobre el codificador visual para proyectar parches y teselas de documentos complejos, gráficos, ecografías o diapositivas con ecuaciones matemáticas hacia el espacio latente del lenguaje sin pasar por motores OCR [1]. El impacto estructural de este diseño en entornos empresariales y académicos radica en la eliminación de las antiguas cadenas de dependencias compuestas por múltiples APIs de bajo nivel: el modelo fundacional interactúa directamente con la señal visual sin pulir y mapea inherentemente el significado cruzado sin intervención ortográfica, suprimiendo la cascada de errores que un OCR defectuoso propagaba hacia las etapas posteriores del razonamiento algorítmico [1].

La Figura 14 esquematiza este flujo de adaptación de un VLM hacia un motor de indexación directa de parches visuales.

De manera complementaria, cuando los sistemas deben ejecutarse en dispositivos locales con recursos limitados (escenario donde se ubican las variantes ligeras de ejecución local como la familia **Gemma** y marcos de trabajo en tiempo real), la literatura reciente aporta la arquitectura **Multi-RAG** (*A Multimodal Retrieval-Augmented Generation System for Adaptive Video Understanding*) [8]. *Multi-RAG* introduce una capa de aplicación práctica orientada a la comprensión adaptativa de flujos de vídeo capturados en tiempo real, concebida específicamente como soporte cognitivo para la interacción humano-robot en

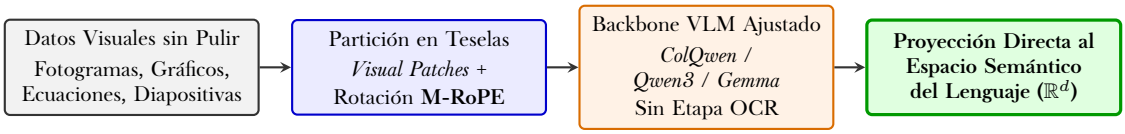


Figura 14: Flujo de codificación visual directa en arquitecturas como *ColQwen* y *Sentence Transformers* multimodales: la combinación de teselas visuales con codificación posicional M-RoPE proyecta diapositivas, gráficos y escenas de vídeo directamente al espacio del lenguaje, eliminando el preprocesamiento intermedio mediante OCR [1, 9].

contextos dinámicos y no estructurados del mundo real [8]. La aportación fundamental de *Multi-RAG* demuestra que los *embeddings* multimodales son capaces de mantener una inferencia semántica de alto rendimiento incluso cuando la entrada visual se ve temporalmente degradada, ocluida o limitada, validando la robustez de estas representaciones vectoriales en escenarios con fuertes restricciones de recursos computacionales [8].

2.4.4 Evaluación en Bancos de Pruebas Masivos (MMEB-V2 y MVEB) y Compromisos de Despliegue

Para cuantificar de manera objetiva y estandarizada la capacidad de recuperación de estos modelos fundacionales sobre vídeo, audio y documentos complejos, la comunidad científica ha establecido entre 2025 y 2026 dos bancos de pruebas (*benchmarks*) masivos de referencia [6]:

- **MMEB-V2 (*Massive Multimodal Embedding Benchmark V2*, 2025):** Amplió la evaluación tradicional de representaciones estáticas texto-imagen para incorporar tareas exigentes de recuperación de fragmentos de vídeo y documentos estructurados (archivos PDF) [6]. Desde el punto de vista metodológico, las métricas empleadas en *MMEB-V2* para clasificación de imágenes y recuperación de vídeo se fundamentan predominantemente en ***Recall@1*** y ***Hit@1***, mientras que los dominios de recuperación de documentos visuales evalúan el ordenamiento mediante ***NDCG@5*** [6].
- **MVEB (*Massive Video Embedding Benchmark*, 2026):** Constituye el estándar específico de evaluación temporal más exhaustivo hasta la fecha, consolidando más de 180 tareas diferenciadas diseñadas para poner a prueba cómo los modelos de *embeddings* recuperan eventos combinando secuencias de vídeo y pistas de audio.

A partir de la evidencia empírica recogida en estos bancos de pruebas y de las especificaciones operativas de cada ecosistema, un ingeniero de datos debe ponderar los compromisos arquitectónicos (*trade-offs*) entre desplegar la solución apoyándose en una API propietaria en la nube frente a alojar modelos de pesos abiertos en infraestructura local [2, 6]. La Figura 15 sintetiza la matriz de decisión técnica entre ambos enfoques.

El Cuadro 4 consolida las especificaciones técnicas de las familias de modelos analizadas en este bloque, detallando su topología, capacidades modales y métricas de referencia en el estado del arte.

Como demuestra este análisis comparado, mientras que los modelos de pesos abiertos como *Qwen3-VL-Embedding* y *Qwen3-VL-Reranker* constituyen la opción óptima cuando se dispone de servidores con

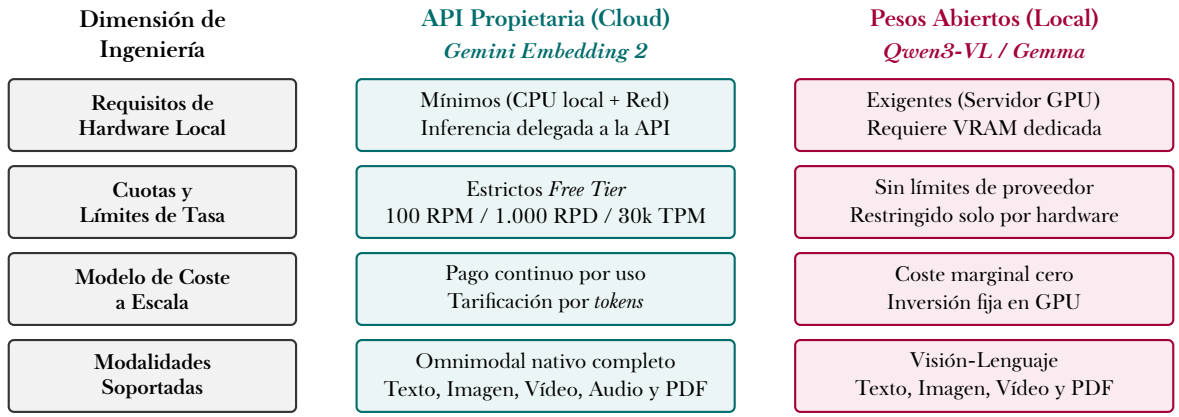


Figura 15: Comparativa de compromisos de ingeniería entre el despliegue sobre API propietaria en la nube y la ejecución local con modelos de pesos abiertos para sistemas de recuperación de vídeo [2, 6].

Tabla 4: Síntesis comparativa de las principales arquitecturas y modelos del ecosistema multimodal analizados en la literatura (2025-2026) [1, 2, 6, 8].

Modelo / Sistema	**Despliegue**	**Topología**	**Innovación Clave**	**Evaluación / Rol Principal**
Gemini Embedding 2	Cloud (API)	Bi-Encoder	Omnimodal nativo (3.072 dim)	*Baseline* PoC sin GPU local
Qwen3-VL-Embedding	Local (2B/8B)	Bi-Encoder	M-RoPE + Vectores MRL	Recuperación masiva en MMEB-V2
Qwen3-VL-Reranker	Local (2B/8B)	Cross-Encoder	*Cross-Attention* + Logit	Consultas composicionales (Top-K)
ColQwen	Local (Open)	Interacción ViT	Parches directos con M-RoPE	Eliminación total de etapa OCR
Multi-RAG / Gemma	Local / Edge	Adaptativo	Inferencia ante vídeo degradado	Robótica y recursos restringidos

aceleración GPU dedicada y se requiere indexar volúmenes masivos sin coste por inferencia, el uso de *Gemini Embedding 2* junto con una base de datos vectorial embebida como Qdrant permite desarrollar y validar una Prueba de Concepto omnimodal completa en ordenadores convencionales sin hardware gráfico especializado, siempre que se instrumenten los controles de concurrencia y cuota detallados en la sección anterior [2, 6].

2.5 Sistemas RAG Multimodales (mRAG) y Comprensión de Vídeo de Larga Duración

2.5.1 Arquitecturas de Referencia en Comprensión de Vídeo: *Video-RAG, CARVE* y *Multi-RAG*

La evolución de los sistemas de Recuperación Aumentada por Generación (RAG) ha transitado desde arquitecturas puramente textuales hacia paradigmas multimodales capaces de procesar, alinear e interpretar información visual, auditiva y textual dentro de un único espacio semántico continuo [1, 8]. En el dominio específico de la recuperación de eventos sobre secuencias de vídeo de larga duración (*Long-form Video Understanding*), esta transición permite localizar sucesos puntuales mediante consultas en lenguaje natural sin depender de metadatos anotados manualmente antes de la indexación [10].

Dentro de la literatura científica consolidada entre 2024 y 2026, destacan tres marcos arquitectónicos que abordan de manera complementaria la alineación visual, la granularidad temporal y la robustez operativa [8, 10]:

- **Marco *Video-RAG* (*Visually-aligned Retrieval-Augmented Long Video Comprehension*):** Propone una arquitectura de recuperación alineada visualmente que complementa los fotogramas del vídeo mediante la extracción e indexación paralela de tres canales auxiliares: texto visible en pantalla obtenido por OCR, transcripciones auditivas y metadatos estructurados de detección de objetos [10]. Al recuperar conjuntamente las señales visuales y sus descriptores alineados, el sistema reduce la carga de contexto sobre el modelo generativo al analizar vídeos extensos [10].
- **Sistema *CARVE* (*Cross-Modal and Multi-Granularity Retrieval*):** Aborda el problema de que los eventos de interés en un vídeo no poseen una duración uniforme (algunos sucesos duran apenas un segundo (como un gesto o la aparición fugaz de una línea de código) mientras que otros abarcan varios minutos (como la explicación completa de un diagrama)). *CARVE* introduce un mecanismo de recuperación intermodal multigranular capaz de evaluar simultáneamente distintas escalas temporales, logrando superar de forma consistente a los métodos tradicionales de ventana única en métricas estándar de ordenación como ***Recall@5*** y ***nDCG@5***.
- **Plataforma *Multi-RAG* (*Adaptive Video Understanding*):** Introduce una capa de aplicación práctica concebida como soporte cognitivo para la interacción humano-robot sobre flujos de vídeo capturados en tiempo real en entornos dinámicos y no estructurados [8]. Su principal aportación científica reside en demostrar que las representaciones vectoriales multimodales mantienen una inferencia semántica de alto rendimiento incluso cuando la señal visual se encuentra temporalmente degradada o limitada, evidenciando la viabilidad de los *embeddings* multimodales bajo fuertes restricciones de recursos [8].

La Figura 16 sintetiza los principios de diseño de estos tres sistemas de referencia en el estado del arte.

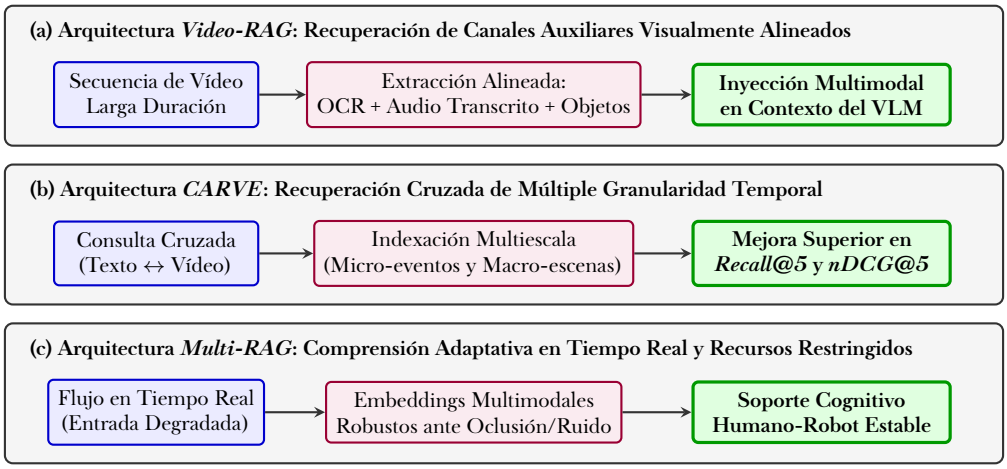


Figura 16: Comparativa de tres arquitecturas representativas de sistemas RAG de vídeo en la literatura reciente: (a) alineación de metadatos auxiliares en *Video-RAG*, (b) indexación multigranular en *CARVE* y (c) inferencia adaptativa resiliente en *Multi-RAG* [8, 10].

2.5.2 Del Emparejamiento Semántico al Anclaje Causal Espaciotemporal (*Temporal Grounding*)

El avance conceptual más profundo que caracteriza a la generación actual de sistemas Video RAG es el desplazamiento cualitativo desde el "análisis temático" hacia el verdadero **razonamiento y anclaje causal espaciotemporal (*Temporal Grounding*)** [2, 11]. Los sistemas RAG tradicionales aplicados a multimedia se limitaban a responder preguntas resumiendo la temática general de un archivo (por ejemplo, devolver una síntesis genérica indicando que *"el vídeo debate sobre economía"*).

Por el contrario, la convergencia de la fragmentación semántica adaptativa (*Video-RAC*), los modelos de transcripción con alineación forzada (*WhisperX*) y la diarización acústica de hablantes con modelos de *embeddings* omnimodales permite que las arquitecturas contemporáneas formulen aserciones fundamentadas con precisión de milisegundos sobre la línea de tiempo [7, 11]:

> *"Exactamente en el segmento 14:32, el segundo hablante señala el cuadrante superior derecho de la tabla, y el gráfico de barras indica un descenso de los ingresos, lo cual contradice el resumen textual proporcionado al inicio de la sesión"* [11].

En este paradigma de *grounding* temporal, el modelo cognitivo no inventa relaciones abstractas ni realiza aproximaciones difusas: se posiciona sobre un eje cronológico determinista $[t_{\text{inicio}}, t_{\text{fin}}]$ y discrimina el orden causal de los acontecimientos dentro de secuencias densas [2, 11]. La Figura 17 contrasta la salida de un sistema de emparejamiento temático clásico frente a un motor dotado de anclaje causal espaciotemporal.

Esta capacidad de anclaje milimétrico y judicialmente auditable habilita cinco dominios estratégicos de aplicación documentados en el proyecto [2]:

1. **Educación y Repositorios Académicos (RAG de Clases Grabadas):** Permite a un estudiante consultar *"minuto donde el profesor explica la mitosis usando el diagrama en la pizarra"* o localizar el instante exacto en que un tutorial de programación explica cómo construir listas, diccionarios o condicionales `if/else`, razonando sobre la conjunción del audio y el código en pantalla.

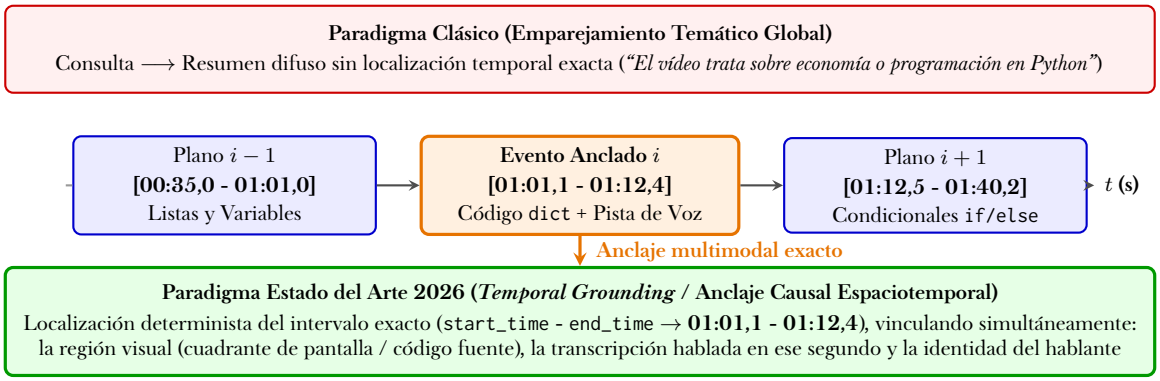


Figura 17: Contraste entre el emparejamiento temático tradicional y el Anclaje Causal Espaciotemporal (*Temporal Grounding*), el cual proporciona trazabilidad determinista y auditable sobre la línea de tiempo del vídeo [2, 11].

2. **Auditoría de Seguridad y Videovigilancia:** Capacidad para inspeccionar cientos de horas de cámaras de seguridad sin supervisión humana mediante consultas en lenguaje natural como *“coche rojo saltándose un semáforo a gran velocidad”* o *“persona abandonando una mochila gris”*, recuperando las marcas de tiempo exactas.
3. **Gestión de Archivos de Medios y *Broadcasting*:** Localización instantánea en archivos históricos de televisión mediante búsquedas que combinan carga visual y entonación acústica, tales como *“momentos de tensión en debates políticos”* o *“personas llorando de alegría”*.
4. **Análisis de Historiales Médicos y Diagnóstico Continuo:** Construcción de un perfil vectorial unificado que proyecta los informes clínicos en texto libre junto con las pruebas diagnósticas de imagen (radiografías, resonancias o ecografías) para encontrar casos clínicos equivalentes o auditar secuencias médicas.
5. **Comercio Electrónico y Búsqueda Visual Cruzada:** Recuperación de artículos específicos en catálogos multimedia a partir de una fotografía o una nota de voz descriptiva del usuario (*“busco un sofá verde esmeralda de terciopelo con patas de madera”*).

2.5.3 Cuellos de Botella Perceptivos en la Verificación Visual: Resolución Espacial y Compresión JPEG

Si bien la teoría de los modelos fundacionales multimodales postula la eliminación de los motores OCR tradicionales gracias a la lectura directa de parches visuales [1], la experimentación empírica en sistemas reales revela que el **OCR implícito** de un modelo de visión-lenguaje es altamente sensible a las transformaciones de preprocesamiento de imagen aplicadas en el *pipeline* de ingestión.

En una arquitectura de búsqueda en dos etapas que utiliza *Gemini Embedding 2* (vía Qdrant) para recuperar los fragmentos candidatos y emplea un modelo ligero secundario (*Gemini 1.5 Flash Lite* o *Gemini Flash 3.5*) como verificador visual para descartar falsos positivos, un ajuste agresivo de los hiperparámetros de codificación visual induce fallos críticos de falso rechazo (falsos negativos).

Durante las pruebas empíricas sobre el vídeo de validación `python_tutorial.mp4` (*“Python en 8*

Minutos"), al ejecutar la consulta `"Diccionarios"`, el motor vectorial recuperó en la quinta posición el intervalo temporal 01:01,1 – 01:40,2 (61, 1 s – 100, 2 s). Aunque la verdad terreno (*ground truth*) del vídeo muestra explícitamente en pantalla entre los segundos 01:01,0 y 01:03,0 la sintaxis de un diccionario en Python (`dict {"name": "Brais"}`), el verificador visual descartó erróneamente el fragmento emitiendo el diagnóstico: *"Descartado (Falso Positivo: 0 %) - Análisis visual: El segmento cubre listas y variables, pero no muestra diccionarios"*.

La auditoría técnica de los parámetros de indexación demuestra que este fallo perceptivo responde a dos cuellos de botella en la codificación de los fotogramas clave antes de su envío a la API:

1. **Resolución espacial destructiva (`resize_width = 512` px):** Redimensionar un videotutorial grabado en alta definición (donde el código fuente ocupa una fracción reducida del editor en pantalla) a un ancho máximo de 512 píxeles reduce la altura de las tipografías a apenas unos pocos píxeles borrosos. Incluso manteniendo una calidad JPEG alta (80), a 512 px los caracteres de la palabra reservada `dict` y las llaves `{}` pierden su definición geométrica, impidiendo que tanto *Gemini Embedding 2* como *Gemini Flash* puedan leer la sintaxis.
2. **Cuantización JPEG agresiva (`jpeg_quality = 12` o `20`):** Cuando además de baja resolución se aplica un factor de calidad JPEG extremo (12 o 20 en la escala 1–100 de `cv2.imencode`), los macrobloques de la Transformada Discreta del Coseno (DCT) difuminan por completo los bordes de alto contraste del texto sobre el fondo del editor, convirtiendo el código en un artefacto ilegible.

La Figura 18 esquematiza el mecanismo exacto por el cual la degradación de resolución y compresión induce alucinaciones de descarte en el verificador visual, así como los rangos de calibración óptimos.

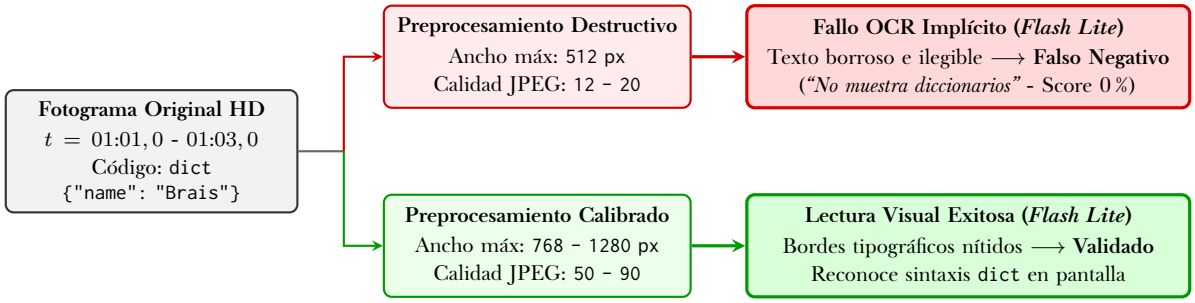


Figura 18: Diagnóstico del cuello de botella perceptivo en la etapa de verificación visual: el escalado a 512 px combinado con compresión JPEG agresiva (12-20) impide el OCR implícito de sintaxis de programación en pantalla, mientras que elevar `resize_width` al rango 768-1280 px y la calidad JPEG a 50-90 restaura la detección sin disparar el consumo de cuota.

2.5.4 Dilución Semántica en Fragmentos de Larga Duración y Muestreo Estático de Fotogramas

El segundo fenómeno crítico identificado durante la validación del sistema es el problema de la **dilución semántica y pérdida de fotogramas clave** en fragmentos de larga duración. En vídeos con un fondo visual altamente estático (tales como clases grabadas, presentaciones de diapositivas o tutoriales de código donde el entorno gráfico del editor permanece inalterado y únicamente cambian unas pocas líneas de texto), el índice de similitud estructural (SSIM) entre fotogramas consecutivos se mantiene elevado

durante largos periodos.

Cuando el umbral de corte SSIM se configura en un valor demasiado permisivo (`threshold = 0.08`) o incluso en un valor moderado (`threshold = 0.60`) sin imponer una duración máxima de plano, el algoritmo agrupa secuencias extensas en un único `VideoChunk` (como el citado fragmento #5 de 39, 1 segundos de duración (01:01,1 – 01:40,2)). A una cadencia estándar de $f_{\text{fps}} = 30$ fotogramas por segundo, un fragmento de $\Delta T = 39,1$ s contiene un total de $N_{\text{total}} = \Delta T \times f_{\text{fps}} \approx 1{,}173$ fotogramas. Dado que la función `extract_key_frames` selecciona un número estático de $k = 3$ fotogramas por fragmento (inicial, final y máxima entropía), la fracción de cobertura temporal cae drásticamente, provocando dos efectos adversos:

- **Omisión por submuestreo (*Keyframe Miss*):** Si el evento buscado (la escritura de `dict {"name": "Brais"}`) ocurre entre los segundos 01:01,0 y 01:03,0, pero el fotograma de máxima entropía global de esos 39, 1 segundos recae en el segundo 01:25,0 (donde la pantalla está más llena de código de listas o variables), ninguno de los 3 fotogramas enviados a la API contendrá el diccionario.
- **Dilución del vector agregado en el espacio latente:** Al mezclar en un único `VideoChunk` casi 40 segundos que abarcan múltiples subtemas distintos (variables, listas y diccionarios), el vector resultante en *Gemini Embedding 2* promedia todas esas temáticas, perdiendo especificidad direccional frente a consultas puntuales.

La Figura 19 ilustra gráficamente cómo se produce esta pérdida por muestreo en un *chunk* gigante y cómo la corrigen las estrategias de acotación temporal o muestreo dinámico.

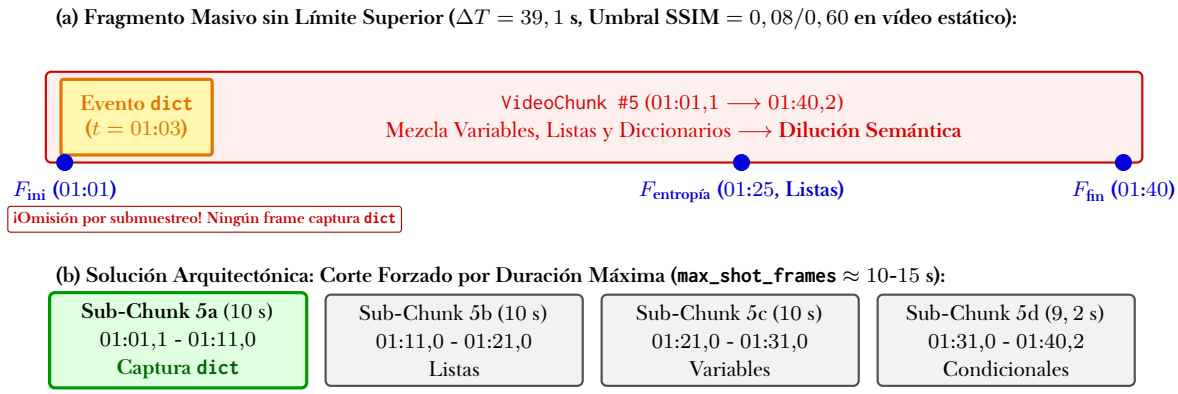


Figura 19: Mecanismo de dilución semántica y pérdida de fotogramas clave en vídeos con baja variación visual: (a) un fragmento de casi 40 s muestreado con solo 3 fotogramas omite el instante en que aparece el diccionario y mezcla múltiples temas; (b) la imposición de un límite superior (`max_shot_frames` de 10-15 s) o la extracción proporcional (un fotograma extra cada 5 s) garantiza la cohesión semántica.

El Cuadro 5 resume la comparativa técnica entre las configuraciones defectuosas analizadas durante la depuración y los rangos corregidos necesarios para preservar tanto la legibilidad del texto en pantalla como la granularidad temporal de los fragmentos.

Estos resultados demuestran que la eficacia de un modelo fundacional omnimodal no depende únicamente de su capacidad neuronal en la nube, sino de que la canalización local de visión por computador preserve la resolución tipográfica suficiente y acote la duración temporal de cada fragmento antes de la vectorización.

Tabla 5: Diagnóstico empírico de parámetros de preprocesamiento visual y temporal en sistemas Video RAG con verificación secundaria (*Gemini Embedding 2* + *Gemini Flash*) [2].

Parámetro	**Configuración Defectuosa**	**Efecto Patológico en el RAG**	**Configuración Calibrada**
Ancho máx. (`resize_width`)	512 px	Texto/código ilegible (fallo OCR implícito)	768 px a 1,280 px (nitidez tipográfica)
Calidad JPEG (`jpeg_quality`)	12 a 20	Artefactos de bloque borran la sintaxis	50-70 (equilibrio) o 80-90 (alta)
Umbral SSIM (`threshold`)	0,08 (o 0,60 estático)	Chunks gigantes ($\sim$ 40 s); dilución vectorial	0,50-0,80 + cota superior temporal
Duración máx. (`max_shot_frames`)	Sin límite (∞)	Pérdida de evento al extraer solo 3 frames	Corte forzado cada 10-15 s máx.
Muestreo (`extract_key_frames`)	Fijo (3 frames / chunk)	Submuestreo severo en planos largos	3 frames o +1 frame cada 5 s

2.6 Estrategias Avanzadas de Fragmentación (*Chunking*) y Alineación Temporal

2.6.1 Obsolescencia de la Partición Fija, Detección de Planos y Agrupación de Escenas

El rendimiento, la precisión de recuperación y la viabilidad técnica de cualquier arquitectura de vectorización multimodal recaen, fundamentalmente, sobre su estrategia de partición de datos o *chunking* [8, 30]. Indexar las pistas de audio, los subtítulos y las secuencias de fotogramas de vídeo de forma desincronizada o mediante divisiones temporales arbitrarias destruye por completo la cohesión semántica y cronológica del documento multimedia [11].

La metodología de fragmentación más elemental (consistente en cortar un archivo de vídeo en intervalos regulares y rígidos, por ejemplo, cada 5 o 10 segundos) constituye un enfoque obsoleto en el estado del arte de 2026 [7, 11]. Aplicar una rejilla temporal ciega produce invariablemente cortes abruptos en mitad de una oración hablada, interrumpe un cambio de plano cinematográfico o divide en dos fragmentos inconexos la transición de una diapositiva académica o la escritura de un bloque de código [7, 11]. Forzar posteriormente al modelo de *embeddings* o al modelo generativo a interpretar fragmentos visuales mutilados introduce un ruido contextual severo que degrada las métricas de relevancia y fidelidad [7].

Para erradicar esta fragmentación destructiva, las arquitecturas contemporáneas aplican algoritmos de **detección de límites de plano (*shot boundary detection*)** acoplados a procesamiento paralelo profundo [11]. Estos algoritmos evalúan fotograma a fotograma la continuidad visual midiendo bien la divergencia de histogramas en espacios de color robustos ante variaciones de iluminación (como el espacio HSV (*Hue, Saturation, Value*), donde un parpadeo de cámara afecta al canal de valor pero no altera el matiz cromático), o bien la similitud estructural (SSIM) entre fotogramas secuenciales, garantizando que cada *chunk* represente una unidad narrativa o visualmente coherente [7, 11].

Por otra parte, cuando el vídeo presenta cortes de cámara muy rápidos y la granularidad del plano resultante es excesivamente fina, la literatura incorpora técnicas de **agrupación de escenas (*scene grouping*)** [11]. Este procedimiento fusiona planos consecutivos cuyas representaciones vectoriales preliminares superan un umbral de similitud predefinido, condensando múltiples micro-planos en intervalos

temporales más largos y semánticamente estables antes de su indexación definitiva [11].

La Figura 20 contrasta el efecto destructivo de la partición por intervalos fijos frente a la preservación de unidades narrativas mediante detección adaptativa de límites de plano y agrupación de escenas.

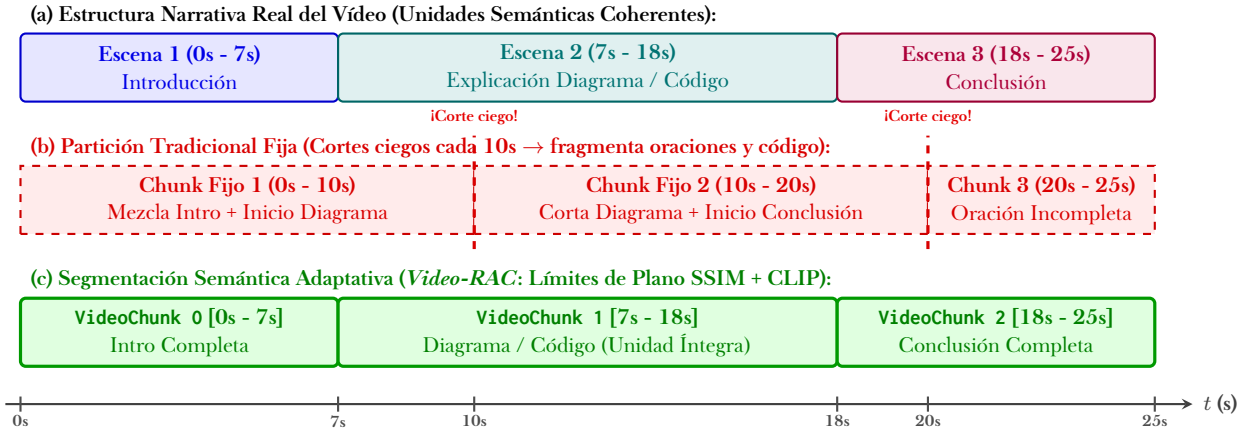


Figura 20: Comparativa estructural entre (b) la división arbitraria en intervalos fijos, que introduce ruido contextual al cortar escenas y oraciones a la mitad, y (c) la segmentación semántica adaptativa (*Video-RAC*), que alinea exactamente los límites de cada `VideoChunk` con las transiciones reales del contenido [7, 11].

2.6.2 Metodología Híbrida *Video-RAC* y Formulación Matemática del SSIM Gaussiano

Dentro del ámbito de los vídeos educativos, académicos y tutoriales técnicos, la arquitectura **Video-RAC (*Retrieval Adaptive Chunking*)** se ha consolidado como el estándar metodológico de referencia [7]. *Video-RAC* abandona las particiones estáticas en favor de un criterio híbrido que evalúa cada transición temporal combinando la similitud del coseno de *embeddings* visuales ligeros generados por CLIP con el Índice de Similitud Estructural (SSIM) [7].

Específicamente, la formulación teórica de *Video-RAC* emplea el modelo `openai/clip-vit-base-patch32` [4] para extraer representaciones rápidas de los fotogramas consecutivos F_t y F_{t+1}, ponderando su similitud semántica con la similitud estructural SSIM [12] según la Ecuación (14) mediante un factor de balance $\alpha = 0,6$ [7]:

$$S_{\text{híbrida}}(F_t, F_{t+1}) = \alpha \cdot \cos(\mathbf{e}_{\text{CLIP}}(F_t),\, \mathbf{e}_{\text{CLIP}}(F_{t+1})) + (1 - \alpha) \cdot \text{SSIM}(F_t, F_{t+1}) \tag{14}$$

Los hiperparámetros canónicos reportados en la literatura para determinar dónde concluye una unidad semántica y comienza la siguiente estipulan un factor de balance $\alpha = 0,6$, un umbral de similitud de *embeddings* CLIP de $\tau_{\text{CLIP}} = 0,85$ y un umbral estructural de $\tau_{\text{SSIM}} = 0,80$ [7].

En implementaciones de producción optimizadas para ejecutarse localmente en CPU sin requerir la carga de pesos adicionales de CLIP durante el preprocesamiento (tal y como adopta nuestro prototipo en el módulo `src/processing/video_chunker.py`), la detección de límites de plano se resuelve directamente mediante la formulación matemática del **Índice de Similitud Estructural (SSIM)** [12] con

ventana Gaussiana en OpenCV (`compute_ssim` dentro de `process_video_single_pass`). Dados dos fotogramas consecutivos convertidos a escala de grises y redimensionados a un ancho de análisis rápido (`resize_width = 160` píxeles para la fase de corte), representados como matrices en coma flotante de doble precisión $a, b \in \mathbb{R}^{H \times W}$ (`np.float64`), el algoritmo aplica un filtro de desenfoque Gaussiano bidimensional `cv2.GaussianBlur` con tamaño de núcleo impar $K \times K$ (`window_size = 7`) y desviación estándar $\sigma = 1,5$ para obtener las medias locales μ_a, μ_b definidas en la Ecuación (15):

$$\mu_a = G_{K\times K,\,\sigma=1,5} * a, \qquad \mu_b = G_{K\times K,\,\sigma=1,5} * b \tag{15}$$

A partir de los cuadrados de las medias (μ_a^2, μ_b^2) y de su producto cruzado ($\mu_{ab} = \mu_a\mu_b$), las varianzas locales σ_a^2, σ_b^2 y la covarianza cruzada local σ_{ab} se calculan matricialmente según la Ecuación (16) mediante convolución Gaussiana:

$$\sigma_a^2 = (G_{K\times K,\,\sigma=1,5} * (a \odot a)) - \mu_a^2, \quad \sigma_b^2 = (G_{K\times K,\,\sigma=1,5} * (b \odot b)) - \mu_b^2, \quad \sigma_{ab} = (G_{K\times K,\,\sigma=1,5} * (a \odot b)) - \mu_{ab} \tag{16}$$

donde $\odot$ denota el producto elemento a elemento (*Hadamard product*). Con estas matrices estadísticas e introduciendo las constantes de estabilidad numérica para un rango dinámico de 8 bits ($L = 255$), definidas como $C_1 = (0,01 \times 255)^2 = 6,5025$ y $C_2 = (0,03 \times 255)^2 = 58,5225$, el mapa espacial SSIM y su escalar medio resultante en el rango $[-1, 1]$ se formulan a través de la Ecuación (17):

$$\mathrm{SSIM}_{\mathrm{map}}(a,b) = \frac{(2\mu_{ab} + C_1)(2\sigma_{ab} + C_2)}{(\mu_a^2 + \mu_b^2 + C_1)(\sigma_a^2 + \sigma_b^2 + C_2)}, \qquad \overline{\mathrm{SSIM}}(a,b) = \frac{1}{H \cdot W} \sum_{y=1}^{H} \sum_{x=1}^{W} \mathrm{SSIM}_{\mathrm{map}}(y,x) \tag{17}$$

Cuando el valor escalar $\overline{\mathrm{SSIM}}(a,b)$ cae por debajo del umbral configurado (`threshold`, con valor por defecto $0,50$ en `ChunkingConfig` y calibrado entre $0,50$ y $0,60$ en evaluación) y el número de fotogramas acumulados desde el corte anterior iguala o supera el mínimo permitido (`min_shot_frames = 12` para filtrar destellos luminosos espurios) - o se alcanza la duración máxima `max_shot_duration` ($8,0$ s por defecto) -, la función `process_video_single_pass` consolida e instancia un objeto inmutable `VideoChunk(index, start_frame, end_frame, start_time, end_time)`.

La Figura 21 detalla este flujo de cálculo matricial implementado en `compute_ssim` y `process_video_single_pass`.

2.6.3 Selección de Fotogramas Clave mediante Entropía Matemática de Shannon

Una vez que el vídeo ha sido particionado en unidades semánticamente coherentes (`VideoChunk`), enviar la totalidad de los fotogramas de cada fragmento (a razón de 30 o 60 fotogramas por segundo) hacia la API del modelo de *embeddings* resultaría computacional y económicamente inviable [2, 7]. Para comprimir el volumen visual sin perder el contenido informativo del plano, la metodología *Video-RAC* establece una estrategia de **selección de fotogramas por entropía** implementada en la función `extract_key_frames` [7].

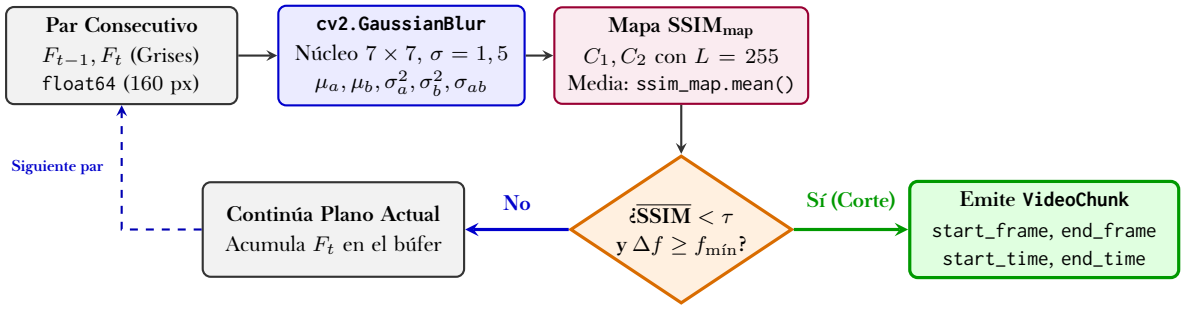


Figura 21: Flujo algorítmico del cálculo del Índice de Similitud Estructural (SSIM [12]) mediante filtrado Gaussiano en OpenCV (`compute_ssim`) y criterio de consolidación de objetos `VideoChunk` en `process_video_single_pass`.

Para cada objeto `VideoChunk` delimitado entre los índices de fotograma $[f_{\text{ini}}, f_{\text{fin}}]$, el algoritmo selecciona exactamente tres fotogramas representativos ordenados cronológicamente [7]:

1. **Fotograma Inicial ($F_{f_{\text{ini}}}$):** Captura el estado visual de partida del evento al abrirse el plano.
2. **Fotograma Intermedio de Máxima Entropía (F_{f^*}):** De entre los fotogramas interiores $t \in (f_{\text{ini}}, f_{\text{fin}})$ (evaluados de forma exhaustiva o mediante un paso de muestreo `entropy_step`), se selecciona aquel que maximiza la **Entropía de Información de Shannon** $H(F_t)$, la cual cuantifica la riqueza y densidad de detalle visual (por ejemplo, el instante en que un diagrama o bloque de código está completamente desplegado en pantalla y no oculto o en transición).
3. **Fotograma Final ($F_{f_{\text{fin}}}$):** Registra el estado consolidado del plano justo antes del corte hacia la siguiente escena.

Matemáticamente, para calcular la entropía de Shannon de un fotograma candidato F_t convertido a escala de grises con $L = 256$ niveles de intensidad ($i \in \{0, 1, \ldots, 255\}$), se obtiene primero su histograma de frecuencias absolutas $h(i)$ mediante `cv2.calcHist` y se normaliza por el número total de píxeles $N_{\text{pix}} = H \times W$ para obtener la distribución de probabilidad discreta $p_i = h(i)/N_{\text{pix}}$, evaluando la Ecuación (18):

$$H(F_t) = -\sum_{\substack{i=0 \\ p_i>0}}^{255} p_i \log_2(p_i), \qquad f^* = \arg \max_{t \in (f_{\text{ini}},\, f_{\text{fin}})} H(F_t) \tag{18}$$

La Figura 22 ilustra la evolución de la curva de entropía $H(F_t)$ dentro de los límites de un `VideoChunk` y la extracción de la terna de matrices NumPy que alimenta al vectorizador multimodal.

2.6.4 Alineación Forzada del Audio (*Forced Alignment*), Silero VAD y Diarización de Hablantes

En paralelo a la segmentación visual, la dimensión auditiva exige una ingeniería de fragmentación igualmente rigurosa [11, 13]. El marco teórico del estado del arte descarta las transcripciones planas carentes de marcas temporales y propone una canalización acústica completa en cuatro etapas [11, 13]:

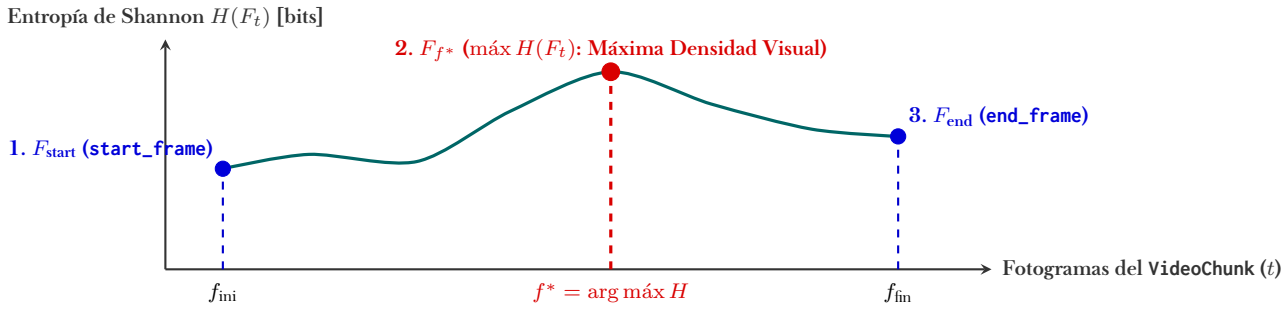


Figura 22: Selección de fotogramas clave basada en Entropía de Shannon (`extract_key_frames`) según la metodología *Video-RAC*: extracción del fotograma inicial, el fotograma final y el fotograma intermedio que maximiza la densidad de información visual $H(F_t)$ [7].

1. **Extracción de Pista Acústica:** A partir del contenedor de vídeo original (`.mp4`), se desmultiplexa y remuestrea la señal de audio a 16 kHz en PCM monocanal empleando herramientas como `ffmpeg` ejecutadas de forma no bloqueante (`asyncio.to_thread`).
2. **Detección de Actividad de Voz (*Voice Activity Detection* - Silero VAD, marco de referencia):** En la literatura avanzada de preprocesamiento acústico, el flujo de audio suele filtrarse previamente mediante redes ultraligeras como **Silero VAD** ($\sim 10^6$ parámetros) para aislar el discurso humano respecto del silencio y del ruido ambiental, mitigando alucinaciones en pausas largas [11, 13].
3. **Transcripción Fonética (*Whisper*), Alineación Forzada (*WhisperX*) y Diarización:** Los segmentos hablados se transcriben mediante modelos fundacionales como **OpenAI Whisper** [5]. En su vertiente extendida descrita en la literatura (*WhisperX* o *Montreal Forced Aligner*), el flujo añade alineación fonética forzada palabra a palabra en milisegundos y diarización de hablantes para etiquetar cada intervención [11, 13].
4. **Superposición y Acoplamiento con `VideoChunk` (`align_transcript_with_chunks`):** Finalmente, los segmentos transcritos se proyectan sobre los intervalos temporales $[t^{(i)}_{\text{start}}, t^{(i)}_{\text{end}}]$ de los objetos `VideoChunk` generados previamente por la etapa visual [11].

Conviene matizar explícitamente la frontera entre dicho marco teórico y la implementación real del prototipo desarrollada en `src/processing/audio_transcriber.py`. Para preservar un presupuesto de memoria unificada estricto (16 GB compartidos entre Whisper y los modelos visuales locales), el prototipo prescinde de las etapas adicionales de Silero VAD y diarización externa e implementa la ruta directa `ffmpeg` (16 kHz mono PCM) $\rightarrow$ `openai-whisper` (variante `base` [5]) $\rightarrow$ `align_transcript_with_chunks`, descargando inmediatamente los pesos acústicos tras concluir la transcripción (`unload_whisper_models`). Dado un `VideoChunk` visual $C_i = [t^{(i)}_{\text{start}}, t^{(i)}_{\text{end}}]$ y un segmento de Whisper $S_j = (\tau^{(j)}_{\text{ini}}, \tau^{(j)}_{\text{fin}}, \text{texto}_j)$, el prototipo vincula la locución mediante la condición de solapamiento mínimo $\Delta\tau_{i,j} \geq \Delta_{\text{mín}} = 0,2$ s formulada en la Ecuación (19):

$$\Delta\tau_{i,j} = \text{máx}\Big(0,\ \text{mín}\Big(t^{(i)}_{\text{end}},\ \tau^{(j)}_{\text{fin}}\Big) - \text{máx}\Big(t^{(i)}_{\text{start}},\ \tau^{(j)}_{\text{ini}}\Big)\Big) \geq \Delta_{\text{mín}} = 0,2\ \text{s} \tag{19}$$

La Figura 23 muestra la arquitectura paralela implementada en el sistema frente a las extensiones avanzadas de la literatura (Silero VAD y WhisperX/diarización), y el Cuadro 6 sintetiza los hiperpará-

metros de referencia junto con los adoptados por el prototipo.

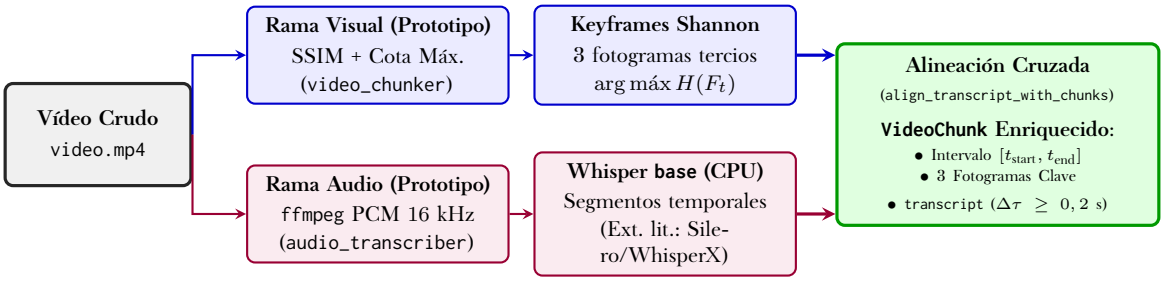


Figura 23: Arquitectura de procesamiento paralelo y alineación forzada intermodal implementada en el prototipo (`video_chunker.py` + `audio_transcriber.py` con `ffmpeg` y Whisper `base` [5]), mostrando las extensiones opcionales descritas en la literatura (CLIP híbrido, Silero VAD y WhisperX [7, 11, 13]).

El Cuadro 6 distingue los parámetros teóricos reportados en el estado del arte frente a los adoptados operativamente por el prototipo.

Tabla 6: Comparativa de parámetros entre la literatura de referencia (*Video-RAC*, Silero VAD/WhisperX) y el prototipo implementado [5, 7, 11–13].

Parámetro	**Modelo / Algoritmo**	**Valor Referencia**	**Función / Adopción en Prototipo**
Codificador visual híbrido	CLIP + SSIM (Lit. *Video-RAC* [4, 7])	$\alpha = 0,6$, $\tau_{\text{CLIP}} = 0,85$	Marco teórico (omitido en prototipo local para ahorrar RAM)
Índice estructural local	SSIM Gaussiano [12] (OpenCV)	$\theta_{\text{SSIM}} = 0,50$ (0,60 Py.)	Implementado en CPU ($K = 7$, $\sigma = 1,5$)
Constantes de estabilidad	Rango dinámico $L = 255$	$C_1 = 6,50$, $C_2 = 58,52$	Evita división por cero en zonas oscuras
Filtro y cota de planos	`min_shot_frames` / $T_{\text{máx}}$	12 fr. / 8,0 s (5,0 s Py.)	Implementado en `ChunkingConfig`
Muestreo de fotogramas	Entropía Shannon $H(F_t)$	3 fotogramas (tercios)	Máxima densidad visual excluyendo bordes (10 %)
Detección voz (Lit.)	Silero VAD (CPU) [13]	$\sim 10^6$ parámetros	Marco teórico (no usado en prototipo)
Transcripción y anclaje	Whisper `base` [5] / Lit. WhisperX	$\Delta_{\text{mín}} = 0,2$ s (Prototipo)	`ffmpeg` 16 kHz + Whisper `base` + solape

2.6.5 Análisis Comparativo frente a Sistemas de Ingestión de Vídeo en Ventana Fija (*SentrySearch*)

En el ecosistema actual de herramientas de código abierto para la búsqueda semántica sobre vídeo mediante *Gemini Embedding 2* y *Qwen3-VL-Embedding* [2, 6], proyectos recientes como ***SentrySearch*** [14] representan el enfoque de partición temporal rígida con ingestión directa de flujo de vídeo. Dicha arquitectura divide los archivos multimedia en fragmentos de duración fija ($\Delta T = 30$ s) con un solapamiento estático ($\Delta t_{\text{overlap}} = 5$ s), reduciendo la resolución a 480p y 5 FPS antes de transmitir el clip de vídeo a la API en la nube o al modelo local [14].

El contraste arquitectónico y matemático entre este enfoque de ventana fija y la canalización adaptativa implementada en el presente Trabajo de Fin de Máster (**SSIM + Entropía de Shannon + Transcripción Alineada**) permite cuantificar cuatro ventajas estructurales de nuestra propuesta [7, 14]:

1. **Reducción superior al $90\,\%$ en facturación de fotogramas y consumo de cuota (*Token Economy*):** La API de *Gemini Embedding 2* extrae, tokeniza y factura nativamente 1 fotograma por cada segundo de vídeo subido (1 FPS), con independencia de la tasa de fotogramas del archivo contenedor [14]. Al dividir 1 hora de vídeo (3,600 s) en fragmentos de 30 s con 5 s de solapamiento, un sistema de ventana fija genera 144 fragmentos y transmite un total de 4,315 fotogramas facturados por hora (dado que los segundos solapados se pagan por duplicado en cada fragmento adyacente) [14]. A una tarifa de referencia de $\$0,00079$ por fotograma de vídeo procesado (equivalente a $\$12,00$ por millón de tokens de vídeo frente a $\$0,45$ por millón de tokens de imagen estática), la ingestión bruta asciende a $\approx \$3,41$ por hora de metraje [2, 14]. Por el contrario, nuestra arquitectura fundamentada en *Video-RAC* [7] evita subir el flujo continuo de vídeo y empaqueta únicamente $k = 3$ fotogramas clave por fragmento (inicial, final y máxima entropía de Shannon) junto con la transcripción fonética dentro de una única estructura `types.Content`. Para una media de 144 fragmentos adaptativos por hora, el sistema procesa únicamente $144 \times 3 = 432$ imágenes en lugar de 4,315 fotogramas de vídeo, logrando una reducción del $\mathbf{89,98}\,\%$ en el volumen visual procesado y manteniendo el consumo dentro de los límites estrictos de la capa gratuita (30,000 TPM y 1,000 RPD).

2. **Superación de las limitaciones de borde temporal y detección heurística de estaticidad:** La propia documentación técnica de *SentrySearch* reconoce como limitaciones abiertas que: (a) la calidad de recuperación se degrada cuando un evento queda dividido entre dos fragmentos fijos, proponiendo como "trabajo futuro" la incorporación de detección inteligente de escenas; y (b) su descarte de fragmentos estáticos depende de una heurística simple que compara el tamaño en bytes de archivos JPEG muestreados, la cual omite movimientos sutiles [14]. Nuestro *pipeline* resuelve nativamente ambos problemas mediante la detección de límites de plano basada en el Índice de Similitud Estructural con filtro Gaussiano 7×7 ($\sigma = 1,5$) sobre OpenCV (`detect_shot_boundaries`) y el cálculo exacto de la densidad de información mediante la Entropía de Shannon sobre el histograma de intensidades (`cv2.calcHist`), acotado por `max_shot_frames` para impedir la dilución semántica en planos estáticos de programación [7].

3. **Explicación teórica y empírica de la distribución de similitud del coseno en consultas específicas:** De acuerdo con el fenómeno de *Modality Gap* analizado en la Subsección 2.1.6 [20], los estudios sobre *Gemini Embedding 2* y *Qwen3-VL-Embedding* confirman que las consultas cruzadas de alta especificidad visual o sintáctica arrojan similitudes del coseno acotadas por debajo de $0,50$ (típicamente $0,28$–$0,39$ en parejas específicas) [14]. Esto fundamenta lo observado posteriormente en el Capítulo 5 sobre `python_tutorial.mp4` para la consulta `"Bucles while"`: una portada gráfica con el título *"Bucles"* alcanza similitud $\mathbf{0,4315}$ (`Chunk 36`, Rango #1), mientras que el fragmento con código real (`while index <5:` en $t = 231,0$ s) obtiene $\mathbf{0,3893}$ (Rango #7).

4. **Necesidad de deduplicación inter-chunk (*Deduplication*) y sobremuestreo antes del *Reranking*:** En secuencias de baja variación visual, fragmentos contiguos acaparan las primeras posiciones vectoriales (como los `Chunks 36, 35 y 33` con $0,4315$, $0,4219$ y $0,4211$), desplazando al fragmento objetivo fuera de un corte `top_k = 5`. Combinar un sobremuestreo (`fetch_k` $>$ top_k) con un filtro de techo de similitud ($\tau_{\text{dedupe}} \approx 0,90$–$0,95$) garantiza diversidad topológica hacia la segunda etapa (*Gemini 3.5 Flash Lite*), preservando degradación elegante (*fallback*) hacia Qdrant ante saturaciones transitorias de red [14].

La Figura 24 ilustra la diferencia estructural y de eficiencia de tokens entre un pipeline de ventana fija a 1 FPS y la arquitectura adaptativa multimodal desarrollada en este trabajo.

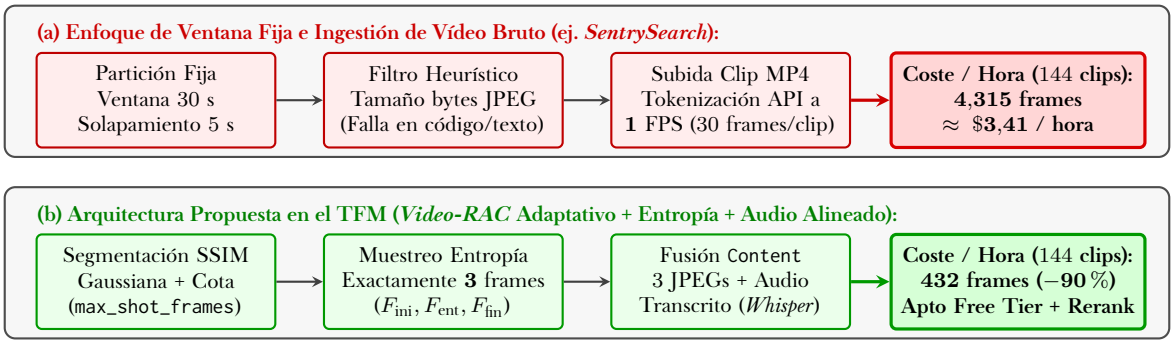


Figura 24: Comparativa arquitectónica y de eficiencia computacional entre la ingestión de vídeo bruto en ventanas fijas a 1 FPS (*SentrySearch*) y la canalización adaptativa multimodal por entropía de Shannon propuesta en el TFM [2, 7, 14].

El Cuadro 7 sintetiza las diferencias técnicas verificables entre ambas implementaciones sobre el mismo ecosistema de modelos fundacionales.

Tabla 7: Comparativa técnica y de costes operativos entre *SentrySearch* y la arquitectura desarrollada en el TFM [2, 7, 14].

Dimensión Técnica	SentrySearch (Ventana Fija)	Arquitectura TFM (Propuesta)
Partición temporal (*Chunking*)	Fija (30 s con 5 s de solapamiento)	Adaptativa por plano (SSIM Gaussiano + cota máx.)
Detección de cambios / información	Heurística por tamaño de archivo JPEG	Matemática (Entropía de Shannon sobre histograma)
Representación visual enviada a API	Vídeo completo tokenizado a 1 FPS (30 frames)	3 fotogramas clave (F_{ini}, $F_{\text{entropía}}$, F_{fin})
Integración del canal auditivo	Sin alineación explícita de transcripción	Transcripción alineada fusionada en `Content` y *Payload*
Fotogramas facturados (1 h vídeo)	4,315 frames/h (≈ $3,41/h en Gemini API)	$\sim$ 432 frames/h ($-\mathbf{89,98}$ % carga visual)
Motor de almacenamiento vectorial	ChromaDB local (colecciones separadas)	Qdrant (`PointStruct`, HNSW, UUIDv5 idempotente)
Refinamiento de Segunda Etapa	VLM sobre clip recortado + `--dedupe 0.9`	*Gemini 3.5 Flash Lite* Bimodal (2 frames + texto) + Fallback

2.7 Bases de Datos Vectoriales, Evaluación Empírica y Metodología de Revisión

2.7.1 Arquitectura de Bases de Datos Vectoriales e Indexación HNSW

Una vez que los fragmentos de vídeo y las consultas en lenguaje natural han sido transformados en representaciones vectoriales densas por los modelos fundacionales, el sistema requiere una infraestructura de almacenamiento persistente capaz de indexar y recuperar alta dimensionalidad en tiempo real [2, 6]. En el estado del arte de 2026, motores especializados como **Qdrant**, **Milvus**, **Weaviate** y **Pinecone** se han consolidado como el estándar industrial para la gestión de repositorios vectoriales [2].

En despliegues basados en motores como Qdrant, cada fragmento vectorizado se encapsula y almacena como una estructura de punto indexado (`PointStruct`) compuesta por tres elementos fundamentales [2]:

1. **Identificador Único Determinista (UUIDv5):** Generado de forma unívoca a partir del nombre del archivo de vídeo y del índice secuencial del fragmento, lo que garantiza que reinsertar el mismo `VideoChunk` sobrescriba de manera idempotente su entrada anterior sin duplicar registros en la base de datos [2].

2. **Vector Denso ($\mathbf{v} \in \mathbb{R}^d$):** El tensor numérico de salida devuelto por el modelo (por ejemplo, configurado a 3,072 dimensiones en *Gemini Embedding 2* o 768 dimensiones con normalización L_2), indexado mediante métrica de distancia coseno (`models.Distance.COSINE`) sobre grafos de mundos pequeños jerárquicos (*Hierarchical Navigable Small World*, **HNSW**) [2, 31].

3. **Carga Útil de Metadatos (*Payload*):** Un diccionario estructurado adjunto al vector que almacena los atributos relacionales indispensables para la reconstrucción temporal del resultado, incluyendo el nombre del fichero (`video_name`), el índice del fragmento (`chunk_index`) y los instantes exactos de inicio y fin expresados en segundos flotantes (`start_time` y `end_time`) [2].

La Figura 25 ilustra la arquitectura de almacenamiento y el direccionamiento mediante índices HNSW en Qdrant.

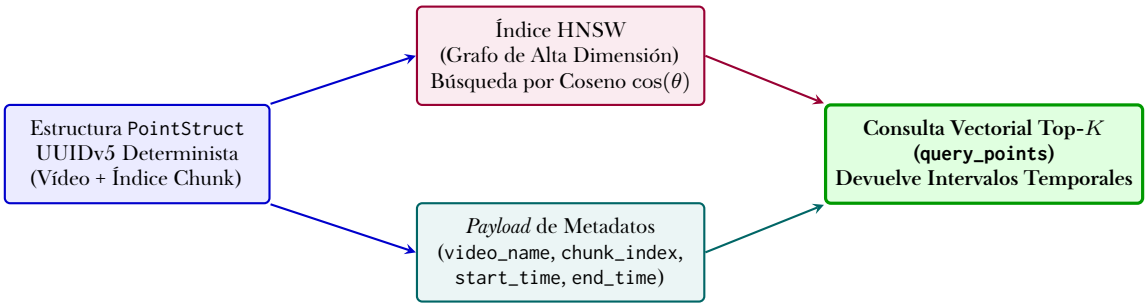


Figura 25: Arquitectura de almacenamiento y recuperación en Qdrant: separación de vectores densos indexados mediante grafos HNSW y metadatos temporales alojados en el *payload* para consultas top-K ultra rápidas [2].

2.7.2 Métricas Clásicas de Recuperación y Bancos de Pruebas Masivos (MMEB-V2 y MVEB)

La validación empírica del rendimiento de los sistemas de recuperación de información multimedia se apoya en una combinación de métricas estadísticas clásicas y bancos de pruebas (*benchmarks*) estandarizados a escala masiva [6, 15].

En primer lugar, las métricas clásicas de Recuperación de Información evalúan la precisión del ordenamiento top-K devuelto por la base de datos vectorial frente a la verdad terreno (*ground truth*) [6]:

- ***Recall@K*:** Mide la proporción de fragmentos relevantes reales que han sido recuperados dentro de los primeros K resultados devueltos [6].
- ***Precision@K*:** Cuantifica qué porcentaje de los K resultados devueltos por el sistema son efectivamente relevantes [6].
- ***Hit@1* (o *Accuracy@1*):** Evalúa si el primer resultado devuelto por el motor de búsqueda coincide exactamente con el evento buscado [6].
- ***nDCG@K* (*Normalized Discounted Cumulative Gain*):** Evalúa la calidad del ranking ponderando la posición de los aciertos, penalizando severamente cuando un resultado relevante aparece en posiciones inferiores de la lista [6].

A nivel global, la comunidad investigadora evalúa los modelos sobre dos bancos de pruebas masivos de referencia actualizados para 2026 [6]:

1. **MMEB-V2 (*Massive Multimodal Embedding Benchmark V2*, 2025):** Consolida más de 78 conjuntos de datos para evaluar la recuperación conjunta sobre texto, imágenes, vídeo y documentos estructurados (PDF), estandarizando mediciones de *Recall@1* y *nDCG@5* [6].
2. **MVEB (*Massive Video Embedding Benchmark*, 2026):** El estándar específico más riguroso para la comprensión temporal de vídeo de larga duración, abarcando más de 180 tareas de recuperación cruzada entre pistas visuales y auditivas [6].

2.7.3 Frameworks de Evaluación RAG y Atribución Verificable (RAGAS, ALCE, ARGUE, MIRAGE)

La evaluación de arquitecturas RAG multimodales exige ir más allá de la simple recuperación de vectores, analizando la calidad de la generación de respuestas y la veracidad de la atribución temporal [7, 15]. El marco de referencia contemporáneo se estructura en torno a dos metodologías complementarias [7, 15]:

2.7.3.1 Framework RAGAS (*Retrieval Augmented Generation Assessment*):

Descompone la calidad de la respuesta generada por el modelo de lenguaje en tres dimensiones fundamentales independientes [7]:

- ***Answer Relevance*:** Mide el grado de pertinencia y coherencia de la respuesta generada respecto a la consulta original del usuario [7].

- ***Context Relevance***: Evalúa la precisión del material recuperado por la base de datos vectorial, penalizando la inclusión de fragmentos de vídeo irrelevantes que aporten ruido al contexto [7].
- ***Faithfulness* (Fidelidad):** Cuantifica si la respuesta generada se deriva estrictamente del contexto recuperado, detectando y penalizando cualquier alucinación o invención por parte del modelo [7].

2.7.3.2 Métricas de Atribución Verificable (ALCE, ARGUE, MIRAGE):

Marcos como **ALCE** (*Automatic LLM Citation Evaluation*), **ARGUE** y **MIRAGE** extienden la evaluación para auditar si cada afirmación generada en la respuesta cuenta con un enlace de citación directo y verificable hacia el intervalo temporal exacto ($[t_{\text{inicio}}, t_{\text{fin}}]$) del vídeo fuente, garantizando una trazabilidad forense inalterable [15].

La Figura 26 resume la estructura jerárquica de evaluación mediante RAGAS y métricas de atribución verificable.

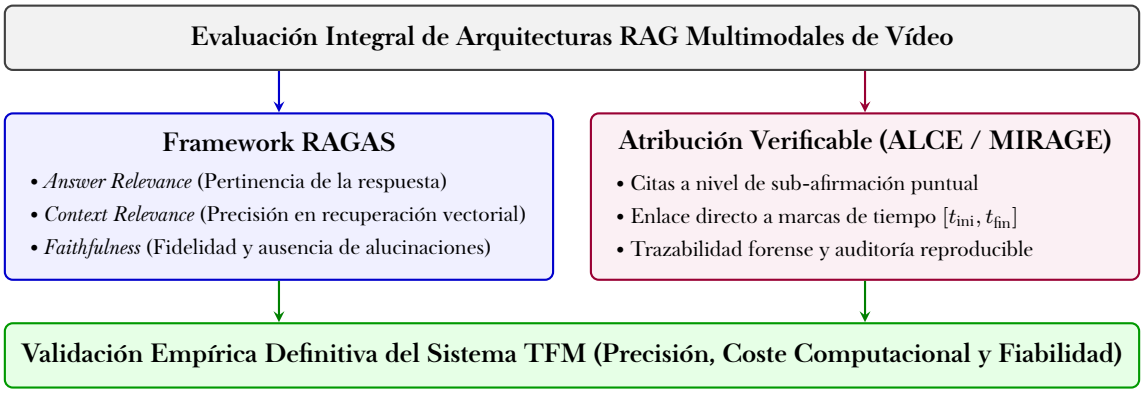


Figura 26: Estructura combinada de evaluación empírica mediante el framework RAGAS y métricas de atribución verificable (ALCE/MIRAGE) para validar el rendimiento del sistema RAG de vídeo [7, 15].

2.7.4 Metodología de Revisión Sistemática de la Literatura Asistida por NotebookLM

Para llevar a cabo la recopilación, filtrado y síntesis crítica de las fuentes bibliográficas empleadas a lo largo de este estado del arte, la investigación ha adoptado un marco metodológico estructurado en un **flujo de trabajo de cuatro etapas asistido por entornos RAG acotados y gestores bibliográficos** (tales como Zotero sincronizado con repositorios abiertos en arXiv) [16, 17]:

1. **Etapa 1 - Construcción y Depuración de la Base de Conocimiento Acotada:** Recopilación sistemática de artículos preprints y documentación técnica cumpliendo restricciones operativas (tales como los límites de ingestión de 50 fuentes por cuaderno y 500.000 palabras por documento en herramientas como Google NotebookLM), resolviéndolo mediante la fusión temática de artículos o el uso de múltiples cuadernos especializados [16].
2. **Etapa 2 - Síntesis Estructural y Descubrimiento Topológico:** Análisis automatizado de convergencia tecnológica mediante resúmenes fonéticos integrados (*Audio Overview*) y generación de cronologías de evolución arquitectónica [16, 17].

3. **Etapa 3 - Análisis Crítico, Deconstrucción Metodológica y Lazo de Relleno de Brechas (*Gap-Fill Loop*):** Contraste sistemático de hipótesis metodológicas y detección de vacíos en la literatura previa sobre fragmentación adaptativa y modelos multimodales [16].
4. **Etapa 4 - Verificación Transparente y Consolidación Causal:** Auditoría inalienable de las citas generadas contrastándolas directamente sobre los textos fuente originales para garantizar la ausencia de alucinaciones teóricas en la memoria del TFM [16, 17].

La Figura 27 esquematiza este flujo metodológico de investigación sistemática.

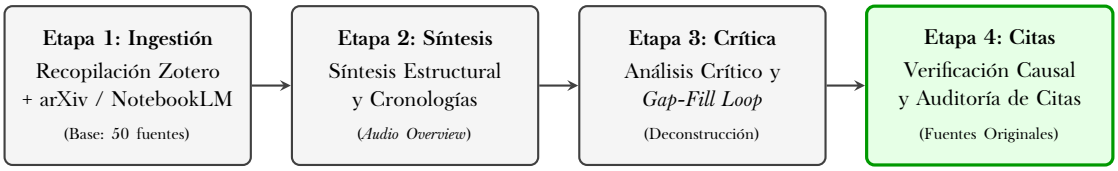


Figura 27: Flujo metodológico de cuatro etapas para la revisión sistemática de la literatura apoyado en gestores bibliográficos y entornos RAG acotados [16, 17].

El Cuadro 8 consolida las métricas, frameworks y metodologías de evaluación y revisión analizadas en este último bloque del estado del arte.

Tabla 8: Síntesis de métricas, bancos de pruebas y metodologías de evaluación en sistemas Video RAG [6, 7, 15, 16].

Categoría	**Framework**	**Métricas**	**Objetivo en el TFM**
Recuperación Vectorial	Qdrant + HNSW	*Recall@K*, *Hit@1*, *nDCG@5*	Eficacia en la localización top-K
Benchmarks Masivos	MMEB-V2 / MVEB	Recuperación multicanal	Comparativa con estándares 2026
Calidad RAG y Fidelidad	Framework RAGAS	*Answer/Context Rel.*, *Faithfulness*	Evaluación de alucinaciones y contexto
Atribución Verificable	ALCE / MIRAGE	Citas a nivel de afirmación	Trazabilidad temporal en milisegundos
Revisión Bibliográfica	Zotero + NotebookLM	Flujo 4 etapas + *Gap-Fill*	Rigor metodológico y estado del arte

3 Metodología y Propuesta Arquitectónica

En esta sección se detalla la metodología científica y el diseño formal de la arquitectura propuesta para la recuperación de eventos en vídeo de larga duración sin metadatos previos. Se formaliza matemáticamente el espacio latente omnimodal y el truncamiento adaptativo mediante *Matryoshka Representation Learning* (MRL). A continuación, se presenta la canalización de ingesta adaptativa basada en la detección de planos por similitud estructural (SSIM), alineación forzada de locución fonética y fusión temprana en una solicitud única. Posteriormente, se expone el motor de búsqueda en dos etapas (recuperación aproximada densa con deduplicación temporal y verificación bimodal asistida por LLM). Por último, se describen los contratos formales del modelo de dominio inmutable y el subsistema de auditoría integral para garantizar la reproducibilidad experimental.

3.1 Formalización Matemática del Espacio Latente Multimodal y Representaciones Matryoshka

La arquitectura propuesta proyecta datos no estructurados de naturalezas heterogéneas (vídeo, audio y texto) en un espacio métrico continuo unificado $\mathcal{S} \subseteq \mathbb{R}^{D_b}$, donde D_b representa la dimensionalidad nativa del motor de vectorización b seleccionado bajo el protocolo común `MultimodalVectorizer` ($D_{\text{gemini}} = 3072$ para *Gemini Embedding 2* [2], $D_{\text{qwen}} = 2048$ para *Qwen3-VL-Embedding-2B* [6, 27] y $D_{\text{gemma}} = 768$ para *EmbeddingGemma 2* [32]).

3.1.1 Espacio de Ingesta y Proyección Omnimodal

Sea un flujo de vídeo segmentado temporalmente en una secuencia ordenada de fragmentos visuales $\{C_1, C_2, \ldots, C_N\}$. Cada fragmento C_i queda definido por una tupla multimodal:

$$C_i = (\mathcal{F}_i, \tau_i) \tag{20}$$

donde $\mathcal{F}_i = \{f_{i,1}, f_{i,2}, f_{i,3}\} \subset \mathbb{R}^{H \times W \times 3}$ corresponde al conjunto ordenado de tres fotogramas clave representativos del plano visual (fotogramas inicial, intermedio y final), y $\tau_i \in \mathcal{T}$ denota la transcripción fonética continua alineada temporalmente con dicho intervalo.

La función de incrustación omnimodal $\mathcal{E} : \left((\mathbb{R}^{H \times W \times 3})^3 \times \mathcal{T}\right) \to \mathbb{R}^D$ transforma la tupla C_i en un vector denso latente $\mathbf{v}_i$:

$$\mathbf{v}_i = \mathcal{E}_b(\mathcal{F}_i, \tau_i), \quad \|\mathbf{v}_i\|_2 = 1, \quad \mathbf{v}_i \in \mathbb{S}^{D_b - 1} \tag{21}$$

donde $\mathbb{S}^{D_b-1}$ denota la hiperesfera unitaria en $\mathbb{R}^{D_b}$. De manera análoga, una consulta multimodal arbitraria $\mathbb{Q}$ (compuesta por cualquier combinación no vacía de texto libre t_q, imágenes de referencia $\{I_{q,j}\}$ y señales auditivas transcritas τ_q) es proyectada por la misma función en el vector unitario $\mathbf{q} \in \mathbb{S}^{D_b-1}$:

$$\mathbf{q} = \mathcal{E}_b(\mathbb{Q}), \quad \|\mathbf{q}\|_2 = 1 \tag{22}$$

3.1.2 Representaciones Anidadas Matryoshka (MRL)

A fin de optimizar el consumo de memoria en memoria principal y acelerar las operaciones de búsqueda aproximada sobre grafos jerárquicos (HNSW), el sistema adopta el paradigma *Matryoshka Representation Learning* (MRL) [18]. MRL entrena el codificador imponiendo una función de pérdida multi-escala que fuerza a los primeros d componentes del vector a concentrar la máxima varianza y capacidad discriminativa semántica.

Formalmente, para cualquier dimensión objetivo admisible $d \in \mathcal{D}_{\text{MRL}}(b)$ ($\mathcal{D}_{\text{MRL}}(\text{gemini}) = \{3072, 1536, 768\}$, $\mathcal{D}_{\text{MRL}}(\text{qwen}) = \{2048, 1536, 768\}$ y $\mathcal{D}_{\text{MRL}}(\text{gemma}) = \{768, 512, 256\}$), se define el operador de truncamiento $\pi_d : \mathbb{R}^{D_b} \to \mathbb{R}^d$ que selecciona el prefijo inicial del vector:

$$\pi_d(\mathbf{v}) = (v_1, v_2, \ldots, v_d)^T \tag{23}$$

Dado que la operación de corte altera la norma euclídea del vector ($\|\pi_d(\mathbf{v})\|_2 \leq 1$), es imperativo aplicar una proyección normalizadora L_2 para preservar la coherencia geométrica de la similitud del coseno:

$$\mathcal{P}_d(\mathbf{v}) = \begin{cases} \dfrac{\pi_d(\mathbf{v})}{\|\pi_d(\mathbf{v})\|_2}, & \text{si } \|\pi_d(\mathbf{v})\|_2 > 0 \\ \pi_d(\mathbf{v}), & \text{en caso contrario} \end{cases} \tag{24}$$

Bajo esta transformación, la métrica de similitud entre una consulta $\mathbf{q}$ y un candidato $\mathbf{v}$ en el subespacio d-dimensional equivale al producto escalar estándar:

$$S_d(\mathbf{q}, \mathbf{v}) = \langle \mathcal{P}_d(\mathbf{q}), \mathcal{P}_d(\mathbf{v}) \rangle = \sum_{j=1}^{d} \mathcal{P}_d(\mathbf{q})_j \cdot \mathcal{P}_d(\mathbf{v})_j \tag{25}$$

Esta formulación garantiza que un único índice vectorial pueda ser consultado a múltiples resoluciones de granularidad dimensional sin necesidad de re-entrenar ni re-vectorizar el corpus audiovisual.

3.2 Canalización de Ingesta Adaptativa y Fusión Temprana Audio-Visual

El procesamiento de secuencias de vídeo continuas de larga duración requiere segmentar el metraje en unidades semánticas discretas que preserven la coherencia visual y auditiva, respetando estrictamente las cuotas de inferencia cuando se emplea la API fundacional (100 RPM, 30,000 TPM y 1,000 RPD) [2] o optimizando el presupuesto de memoria unificada cuando se ejecutan motores locales en GPU.

3.2.1 Segmentación Temporal Adaptativa mediante Similitud Estructural (SSIM)

En lugar de emplear ventanas temporales fijas que fragmentan arbitrariamente eventos continuos o introducen mezclas incoherentes de planos, el sistema implementa una segmentación adaptativa de una sola pasada (*single-pass*).

Para cada par de fotogramas secuenciales en escala de grises I_{t-1}, I_t, se evalúa el Índice de Similitud Estructural (*Structural Similarity Index Measure*, SSIM) [12]:

$$\mathrm{SSIM}(I_{t-1}, I_t) = \frac{(2\mu_{t-1}\mu_t + C_1)(2\sigma_{t-1,t} + C_2)}{(\mu_{t-1}^2 + \mu_t^2 + C_1)(\sigma_{t-1}^2 + \sigma_t^2 + C_2)} \tag{26}$$

donde μ y σ^2 representan la media y varianza local de luminancia, $\sigma_{t-1,t}$ es la covarianza cruzada, y $C_1 = (K_1 L)^2, C_2 = (K_2 L)^2$ son constantes estabilizadoras ante regiones oscuras ($L = 255, K_1 = 0,01, K_2 = 0,03$).

Un corte de plano se declara formalmente en el instante t si se satisface la condición de transición abrupta o se alcanza la cota superior temporal de coherencia:

$$\mathrm{Corte}(t) = (\mathrm{SSIM}(I_{t-1}, I_t) < \theta_{\mathrm{SSIM}} \wedge \Delta t_{\mathrm{actual}} \geq T_{\mathrm{mín}}) \vee (\Delta t_{\mathrm{actual}} \geq T_{\mathrm{máx}}) \tag{27}$$

donde la configuración de dominio por defecto (`ChunkingConfig`) establece $\theta_{\mathrm{SSIM}} = 0,50$, $T_{\mathrm{mín}} = 12$ fotogramas ($\approx 0,4$–$0,5$ s) y $T_{\mathrm{máx}} = 8,0$ s (ajustándose en el banco de pruebas E2E a $\theta_{\mathrm{SSIM}} = 0,60$, $T_{\mathrm{máx}} = 5,0$ s para tutoriales de código estático y manteniendo $0,50/8,0$ s en metraje documental). Para cada plano detectado de n fotogramas ($n \geq 3$), el selector descarta un margen exterior del 10 % en ambos extremos (margin $= \lfloor n/10 \rfloor$), divide el intervalo interior en tres tercios equivalentes y selecciona en cada tercio el fotograma clave de **máxima entropía visual de Shannon** ($f^* = \arg\max H(F_t)$), evitando así fotogramas borrosos de transición. Cada fotograma seleccionado se escala a un ancho uniforme de 512 píxeles mediante interpolación por áreas (`cv2.INTER_AREA`) y se codifica en formato JPEG (calidad 80 %), reduciendo la carga útil sin degradar la legibilidad tipográfica ni geométrica.

3.2.2 Extracción y Alineación Forzada de la Locución Auditiva

De forma secuencial o desacoplada a la segmentación visual, la pista de audio completa del contenedor de vídeo es desacoplada mediante `ffmpeg`, remuestreada a 16 kHz en formato PCM monocanal y procesada con el modelo de reconocimiento automático del habla OpenAI Whisper (*checkpoint* `base`) [5].

Whisper produce un flujo de segmentos transcritos $\mathcal{A} = \{A_1, A_2, \ldots, A_M\}$, donde cada segmento $A_k = (t_{a,k}^{\mathrm{start}}, t_{a,k}^{\mathrm{end}}, w_k)$ incluye marcas temporales de alta resolución y el texto fonético correspondiente

w_k. La alineación forzada entre los planos visuales $C_i = [t_i^{\text{start}}, t_i^{\text{end}}]$ y las locuciones auditivas se establece evaluando el solape temporal efectivo:

$$\text{Solape}(C_i, A_k) = \text{máx}\left(0, \text{mín}(t_i^{\text{end}}, t_{a,k}^{\text{end}}) - \text{máx}(t_i^{\text{start}}, t_{a,k}^{\text{start}})\right) \tag{28}$$

Un segmento de locución se vincula al fragmento visual C_i si cumple el umbral mínimo de solape $\text{Solape}(C_i, A_k) \geq \Delta_{\text{mín}} = 0,2$ s. La transcripción global τ_i asociada al plano visual resulta de la concatenación cronológica de todos los segmentos vinculados:

$$\tau_i = \bigoplus_{k:\text{Solape}(C_i,A_k)\geq\Delta_{\text{mín}}} w_k \tag{29}$$

3.2.3 Fusión Temprana Omnimodal en Solicitud Única

Para preservar la riqueza perceptual conjunta sin duplicar el coste de llamadas ni fragmentar los espacios latentes, la inyección de la información visual y auditiva se efectúa mediante fusión temprana (*early fusion*) detrás del protocolo común `MultimodalVectorizer`.

En el backend remoto (*Gemini Embedding 2*), los tres fotogramas clave codificados en JPEG se transforman en objetos binarios de imagen (`types.Part.from_bytes`) junto a la locución alineada τ_i (`types.Part.from_text`) dentro de un único contenedor `types.Content`:

$$\text{Payload}_i = \{\text{parts} : [\text{JPEG}(f_{i,1}), \text{JPEG}(f_{i,2}), \text{JPEG}(f_{i,3}), \text{Text}(\tau_i)]\} \tag{30}$$

En los backends locales (*Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2*), el adaptador transforma el mismo par $(\mathcal{F}_i, \tau_i)$ en la representación nativa esperada por cada arquitectura: diccionarios multimodales (`{image, text}`) para *Qwen3-VL-2B* y plantillas prefijadas con etiquetas `<|image|>` intercaladas por cada fotograma para *EmbeddingGemma 2*. Este diseño unificado preserva una estricta invarianza estructural (1 chunk = 1 vector) con independencia del motor remoto o local seleccionado.

3.3 Mecanismo de Recuperación en Dos Etapas y Verificación Bimodal

La búsqueda sobre grandes colecciones de vídeo enfrenta una disyuntiva clásica entre latencia y precisión: la comparación exhaustiva mediante modelos de visión-lenguaje de alta capacidad resulta computacionalmente inviable en tiempo real, mientras que la recuperación densa basada exclusivamente en similitud del coseno puede verse afectada por coincidencias superficiales y falsos positivos [10, 15]. Para resolver este dilema, el sistema adopta una arquitectura en dos etapas jerárquicas: recuperación aproximada de alta cobertura (Etapa 1) y verificación bimodal estructurada (Etapa 2).

3.3.1 Etapa 1: Recuperación Densa y Deduplicación Temporal

La consulta multimodal $\mathbb{Q}$ emitida por el usuario es vectorizada en un embedding unitario $\mathbf{q}^{(d)} \in \mathbb{S}^{d-1}$ a la resolución dimensional seleccionada d. A continuación, el motor de búsqueda vectorial local *Qdrant* ejecuta una búsqueda de vecinos más cercanos basada en el índice jerárquico de grafos navegables de mundo pequeño (HNSW) [31] con función de distancia coseno:

$$\mathscr{C}_{\text{raw}} = \text{Top-}k_1 \left(\{(C_i, S_d(\mathbf{q}, \mathbf{v}_i))\}_{i=1}^{N} \right) \tag{31}$$

donde el factor de sobremuestreo se establece dinámicamente en $k_1 = \text{mín}(5 \cdot k_{\text{target}}, 20)$, garantizando un amplio espacio de candidatos para la etapa posterior.

Para evitar redundancias causadas por cortes visuales muy próximos dentro de una misma escena continua, la lista preliminar $\mathscr{C}_{\text{raw}}$ es sometida a un algoritmo determinista de deduplicación temporal:

- **Fusión de micro-planos adyacentes:** Si dos candidatos recuperados pertenecen al mismo archivo de vídeo y su distancia temporal es inferior a $1,0$ s, se unifican en un único intervalo envolvente $[t_{\text{mín}}, t_{\text{máx}}]$.
- **Deduplicación de locuciones repetidas:** Si fragmentos adyacentes comparten una transcripción idéntica o solapada superior al 80 % de coincidencia léxica, se preserva únicamente el candidato con mayor similitud en la primera etapa.

El resultado es un conjunto curado $\mathscr{C}_{\text{dedup}}$ de candidatos únicos, ordenados descendentemente por su puntuación vectorial $S_d(\mathbf{q}, \mathbf{v}_i)$.

3.3.2 Etapa 2: Árbitro Bimodal Multimodal con Gemini 3.5 Flash Lite

Los mejores candidatos de $\mathscr{C}_{\text{dedup}}$ son transferidos a la segunda etapa de verificación. Para cada candidato C_i, el sistema recupera de la caché local en disco los fotogramas clave en formato JPEG y la transcripción de audio alineada τ_i.

El modelo de lenguaje multimodal *Gemini 3.5 Flash Lite* actúa como juez bimodal independiente. Se formula un *prompt* estructurado que incluye:

1. La consulta original del usuario (incluyendo imágenes de referencia o transcripción de audio de la consulta).
2. La evidencia visual del candidato (fotogramas clave secuenciales del fragmento).

3. La evidencia auditiva textualizada (transcripción de las palabras efectivamente pronunciadas en ese intervalo).
4. Instrucciones de juicio disjunto e inclusivo: un fragmento se considera relevante si la respuesta a la consulta se encuentra presente en la imagen, en la locución hablada, o en la combinación simultánea de ambas modalidades.

El modelo responde forzando una gramática JSON estricta para cada candidato:

$$\text{Evaluación}(C_i) = \{\text{relevance_score} \in [0,0,1,0],\ \text{is_relevant} \in \{\text{true}, \text{false}\},\ \text{reasoning} \in \text{String}\} \tag{32}$$

3.3.3 Discriminación de Falsos Positivos y Ordenación Definitiva

La puntuación final asignada al fragmento combina la evidencia de ambas etapas y aplica una función de corte estricta:

$$\text{Score}_{\text{final}}(C_i) = \begin{cases} \text{relevance_score}_i, & \text{si is_relevant}_i = \text{true} \wedge \text{relevance_score}_i \geq \theta_{\text{rel}} \\ 0,0, & \text{en caso contrario} \end{cases} \tag{33}$$

donde $\theta_{\text{rel}} = 0,5$. Los candidatos que no superan el criterio del árbitro bimodal son descartados inmediatamente, lo que garantiza que consultas con conceptos inexistentes o trampas visuales no superen la fase de validación (inmunidad total frente a falsos positivos). La lista final de resultados devuelta al usuario se trunca a los k_{target} mejores elementos, acompañados del razonamiento explícito generado por el modelo.

3.4 Modelo de Dominio Inmutable y Contratos de Datos para Reproducibilidad

En una investigación empírica en recuperación de información, la reproducibilidad de los experimentos y la integridad de los resultados dependen de la consistencia de las estructuras de datos que fluyen entre los subsistemas de ingesta, indexación y consulta.

3.4.1 Entidades de Dominio Inmutables

Para prevenir efectos colaterales y garantizar que los vectores y metadatos no puedan ser modificados tras su generación, todas las entidades centrales del sistema se implementan como clases de datos estrictamente inmutables (`@dataclass(frozen=True)`). La Figura 28 sintetiza la jerarquía y relaciones entre las entidades del dominio:

Estructura del Modelo de Dominio Inmutable

- **`TranscriptSegment`:** Representa una locución temporal continua.
  - Atributos: `start_time: float`, `end_time: float`, `text: str`.
- **`VideoChunk`:** Segmento temporal derivado de la detección de planos por SSIM.
  - Atributos: `index: int`, `start_frame: int`, `end_frame: int`, `start_time: float`, `end_time: float`, `transcript: str | None`.
- **`VectorizedChunk`:** Fragmento indexable con su proyección latente.
  - Atributos: `index: int`, `start_time: float`, `end_time: float`, `embedding: list[float]`, `transcript: str | None`.
- **`SearchResult`:** Candidato recuperado en la Etapa 1 por similitud del coseno.
  - Atributos: `score: float`, `video_name: str`, `chunk_index: int`, `start_time: float`, `end_time: float`, `transcript: str | None`.
- **`RerankedResult`:** Resultado verificado y clasificado por el árbitro bimodal (Etapa 2).
  - Atributos: `result: SearchResult`, `relevance_score: float`, `is_relevant: bool`, `reasoning: str`.
- **`MultimodalQuery`:** Consulta omnimodal polimórfica inmutable.
  - Atributos: `text: str | None`, `txt_contents: tuple[str, ...]`, `images: tuple[bytes, ...]`, `audio_paths: tuple[Path, ...]`, `audio_bytes: tuple[bytes, ...]`, `audio_transcripts: tuple[str, ...]`.

Figura 28: Entidades fundamentales del modelo de datos inmutable implementadas en el sistema.

El uso de tuplas inmutables (`tuple`) en `MultimodalQuery` impide mutaciones accidentales de los contenidos binarios de imágenes o audios durante la resolución de la consulta. La propiedad derivada `composed_text` unifica automáticamente el texto directo, el contenido de archivos `.txt` y las transcripciones de las pistas de audio aportadas por el usuario.

3.4.2 Trazabilidad Científica Integral mediante *RunAuditor*

Para asegurar la total reproducibilidad de cada ejecución experimental, se diseñó el componente `RunAuditor`. Cada sesión de indexación o batería de consultas genera de forma determinista un directorio de auditoría bajo la ruta `data/audit/<timestamp>_<run_id>/` con tres artefactos estructurados en JSON:

1. **`run_metadata.json`:** Registra los metadatos de entorno (marca temporal ISO-8601, versión del sistema, identificador único de ejecución, plataforma hardware y latencias acumuladas).
2. **`config.json`:** Almacena la fotografía exacta de todos los hiperparámetros operativos: backend de vectorización (`gemini`, `qwen3-vl-2b` o `gemma2`), identificador del modelo subyacente (`gemini-embedding-2-prev` `Qwen/Qwen3-VL-Embedding-2B` o `google/embeddinggemma-2`), dimensionalidad seleccionada $d \in \mathcal{D}_{\text{MRL}}(b)$, calidad de codificación JPEG, umbral de similitud SSIM, modelo de reranker (`gemini-3.5-flash-lit` y factor de sobremuestreo k_1.
3. **`search_results.json`:** Archiva la traza completa de cada consulta ejecutada: vector de consulta emitido, lista íntegra de candidatos recuperados en la Etapa 1 con sus puntuaciones de coseno, y evaluaciones individuales de la Etapa 2 con los juicios booleanos, puntuaciones y justificaciones textuales.

Esta trazabilidad desacopla el análisis empírico de la ejecución en vivo, permitiendo re-evaluar métricas analíticas (*Recall@k*, *MRR*, *NDCG*) sin incurrir en nuevas llamadas a la API ni depender de la volatilidad del estado del servidor.

4 Configuración Experimental y Desarrollo del Prototipo

Esta sección expone los aspectos metodológicos e instrumentales relativos a la implementación del prototipo experimental y la infraestructura de pruebas. Se describe la plataforma de computación, el uso de las APIs de Google Gemini, la base de datos vectorial Qdrant y las salvaguardas arquitectónicas implementadas para operar bajo restricciones de cuota estrictas. A continuación, se detallan las decisiones de ingeniería software aplicadas en los módulos de ingesta, vectorización, reordenación bimodal e interfaz gráfica. Finalmente, se presenta el diseño del arnés de auditoría y trazabilidad (*RunAuditor*) que asegura la total reproducibilidad científica de los resultados empíricos.

4.1 Entorno Experimental, Infraestructura y Gestión de Cuotas

Esta subsección describe la plataforma de cómputo, las librerías del entorno de ejecución y las políticas de control de cuotas y memoria del prototipo.

4.1.1 Plataforma de Ejecución y Pila Tecnológica

Los experimentos y el desarrollo del prototipo se ejecutaron sobre una estación de trabajo con procesador Apple Silicon M3 (arquitectura ARM64), 16 GB de memoria unificada y sistema operativo macOS, utilizando el entorno de ejecución Python 3.13 (`3.13.12`). La pila de software científico y librerías especializadas comprende:

- **Procesamiento Audiovisual:** `OpenCV` (`opencv-python`) para la decodificación de metraje, cálculo matricial del índice SSIM, muestreo por entropía visual y compresión JPEG; `ffmpeg` para el remuestreo de audio; y `openai-whisper` (modelo `base`) para el reconocimiento automático del habla (ASR) ejecutado localmente en CPU.
- **Modelos Fundacionales y Motores de Vectorización (Cloud y Local):** SDK oficial `google-genai` para el acceso remoto a *Gemini Embedding 2* (`gemini-embedding-2-preview`) y el árbitro bimodal *Gemini 3.5 Flash Lite* (`gemini-3.5-flash-lite`); junto con `PyTorch` (`2.14.1`, acelerador Metal Performance Shaders `mps`), `transformers` (`5.22.2`), `sentence-transformers[image]` (`6.3.0`) y `torchvision` (`0.29.2`) para inferencia local de *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2* en precisión `bfloat16` sobre `mps` (y `float32` en CPU).
- **Indexación Vectorial:** `qdrant-client` en modo local persistente en disco (`./data/vector_db`), evitando la sobrecarga de latencia de red y garantizando aislamiento experimental entre configuraciones mediante colecciones independientes.
- **Cálculo Numérico, Estadístico y Evaluación:** `NumPy` para el truncamiento dimensional y normalización L_2 bajo el estándar MRL, `SciPy` (`1.18.1`) para las pruebas de significación pareada (*McNemar* y *Wilcoxon*), y `pytest` como motor de ejecución para las suites de prueba unitarias y de extremo a extremo.

4.1.2 Gestión de Cuotas, Concurrencia y Presupuesto de Memoria Unificada

La capa gratuita de la API de Google Gemini impone restricciones severas que condicionan el diseño de cualquier sistema experimental en tiempo real: un máximo de 100 peticiones por minuto (RPM), 30,000 tokens por minuto (TPM) y 1,000 peticiones diarias (RPD) [2]. En paralelo, la ejecución local de modelos multimodales de hasta 2B parámetros dentro de 16 GB de RAM unificada requiere evitar la fragmentación de caché en GPU (`mps`) y las colisiones concurrentes de descriptores de comandos.

Para operar de manera robusta y continua sin interrupciones por denegación de servicio (código HTTP 429 / `RESOURCE_EXHAUSTED`) ni desbordamiento de memoria, se implementaron cuatro salvaguardas arquitectónicas:

1. **Reintentos Exponenciales con Dispersión (*Exponential Backoff with Jitter*):** Toda llamada remota a la API se canaliza a través de la función utilitaria `call_with_retry`. Ante errores transitorios de cuota o sobrecarga de red, el cliente aplica una pausa adaptativa con retardo base de $4,0$ s para vectorización y $1,5$ s para reordenación, incrementándose exponencialmente con un factor de aleatoriedad hasta un máximo de 4 reintentos antes de elevar una excepción controlada (`VectorizationError`).

2. **Cortafuegos de Cuota por Vídeo (`QuotaSafetyError`):** Antes de vectorizar, el orquestador verifica que el total de fragmentos no supere MAX_CHUNKS_PER_VIDEO $= 800$. Si un vídeo supera dicho umbral, aborta preventivamente antes de emitir llamadas a la API, evitando agotar la cuota diaria.

3. **Semáforo Adaptativo y Cerrojo por Modelo:** En `process_video_pipeline`, las llamadas remotas a Gemini emplean un máximo de 3 peticiones concurrentes (`max_concurrent_requests = 3`), mientras que los motores locales fuerzan concurrencia estricta de 1 (`Semaphore(1)`) y serializan las pasadas `model.encode` mediante un `threading.Lock` compartido por `model_id` en `local_vectorizer.py`.

4. **Descarga Secuencial de Modelos (*Single-Resident Lifecycle*):** Antes de cargar un vectorizador local tras la transcripción ASR, el sistema invoca `unload_whisper_models()`; de manera simétrica, la factoría `factory.py` mantiene un único modelo local residente y libera el anterior mediante `unload_active_vectorizer()` (`gc.collect()` seguido de `torch.mps.empty_cache()`).

4.1.3 Configuración y Aislamiento de Colecciones en Qdrant

La base vectorial local almacena los fragmentos indexados en colecciones aisladas generadas por `collection_name_for(backend, dim)`. Para preservar el índice histórico de referencia sin consumir cuota diaria adicional, la configuración predeterminada (`gemini` en $d = 3072$) conserva la colección `video_chunks`, mientras que cualquier otra combinación de motor y dimensionalidad se aísla bajo el esquema determinista `video_chunks__<backend>__<dim>` (cubriendo `gemini` en 1536/768, `qwen3-vl-2b` en 2048/1536/768 y `gemma2` en 768/512/256). Cada colección se configura con métrica de distancia coseno (`Distance.COSINE`) y almacena un *payload* documental uniforme (nombre de vídeo, `chunk_index`, intervalo $[t_{\text{start}}, t_{\text{end}}]$ y transcripción), habilitando la conmutación instantánea de motores en la interfaz sin reindexar.

4.2 Implementación Modular del Prototipo Experimental

El prototipo se diseñó siguiendo el principio de inversión de dependencias y bajo un desacoplamiento estricto de componentes, como evidencia el análisis de grafos del repositorio (27 comunidades funcionales y 0 ciclos de dependencia). A continuación se detallan las decisiones de implementación adoptadas en los módulos nucleares:

4.2.1 Módulo de Ingesta, Segmentación y Transcripción

El paquete `src/processing/` implementa la ingesta de medios a través de dos componentes cooperativos y una caché persistente en disco:

- **`video_chunker.py`:** La función `process_video_single_pass` recorre los fotogramas decodificados por OpenCV en una única pasada. Calcula `compute_ssim` sobre miniaturas en escala de grises de 160 px de ancho (`resize_width = 160`) y la entropía visual de Shannon (`compute_image_entropy`). Al cerrar cada plano, `_select_chunk_keyframes` selecciona los 3 fotogramas clave de máxima entropía dentro de los tercios interiores, los escala mediante `cv2.INTER_AREA` a 512 px (`resize_frame`) y los persiste en `keyframe_cache.py` como JPEG (calidad 80 %).
- **`audio_transcriber.py`:** Ejecuta Whisper (`base`) en CPU y fusiona las locuciones con cada `VideoChunk` mediante `align_transcript_with_chunks` ($\geq 0,2$ s de solape). Expone además `unload_whisper_models()` para liberar memoria RAM antes de cargar embeddings locales, `transcribe_query_audios()` para normalizar notas de voz y archivos de audio de consulta a WAV mono a 16 kHz vía `ffmpeg`, y el conmutador `enable_audio=False` (`--no-audio` en CLI y casilla en Streamlit) para mitigar ruido ASR en metraje *B-roll*.

4.2.2 Protocolo `MultimodalVectorizer`, Motores Locales y Truncamiento MRL

La vectorización se desacopla detrás del protocolo `MultimodalVectorizer` definido en `src/embeddings/base.py` (métodos `vectorize_chunk`, `embed_query` y `embed_text`), permitiendo conmutar entre inferencia remota y local sin alterar el pipeline ni el motor de búsqueda:

- **`gemini_vectorizer.py` (`GeminiMultimodalVectorizer`):** Ensambla objetos `types.Content` combinando las tres imágenes JPEG y la transcripción alineada en una llamada remota única contra `gemini-embedding-2-preview`.
- **`local_vectorizer.py` (`LocalMultimodalVectorizer`):** Ejecuta localmente `Qwen/Qwen3-VL-Embedding-2B` y `google/embeddinggemma-2` mediante `SentenceTransformer` fijando estrictamente precisión `torch.bfloat16` sobre Apple Silicon `mps` (evitando desbordamientos `NaN` observados en `float16` sobre *EmbeddingGemma 2*) y `torch.float32` en CPU, bajo un cerrojo `threading.Lock` compartido por `model_id` (`_lock_for_model`). Adapta la entrada al contrato nativo de cada familia: diccionarios multimodales (`{image, text}`) con `QWEN_QUERY_INSTRUCTION` para *Qwen3-VL-2B*, y prefijos defensivos (`prompts.get(...)`) acompañados de etiquetas explícitas `<|image|>` por cada fotograma en *EmbeddingGemma 2* (soportando consultas puramente visuales o multi-imagen).

- **`factory.py`:** Las funciones `create_vectorizer` y `unload_active_vectorizer` garantizan que como máximo un único modelo local resida en los 16 GB de RAM unificada, descargando el anterior mediante `gc.collect()` y `torch.mps.empty_cache()`.

El truncamiento de representaciones Matryoshka se centraliza en `src/embeddings/mrl.py` mediante `truncate_and_normalize_mrl`:

$$\mathbf{v}^{(d)} = \texttt{truncate_and_normalize_mrl}(\mathbf{v}, d) \tag{34}$$

Esta función verifica que $1 \leq d \leq \text{len}(\mathbf{v})$, extrae el prefijo de las primeras d coordenadas, calcula su norma euclídea mediante `numpy.linalg.norm` y proyecta el sub-vector sobre la hiperesfera unitaria $\mathbb{S}^{d-1}$.

4.2.3 Motor de Búsqueda Vectorial Qdrant

La clase `QdrantSearchEngine` en `src/rag_engine/vector_db.py` gestiona la persistencia e indexación sobre colecciones aisladas por motor y dimensión (`collection_name_for`). Para cada fragmento genera un identificador UUIDv5 determinista derivado del nombre del vídeo y su `chunk_index`, evitando duplicados en re-indexaciones. La recuperación de vecinos más cercanos se ejecuta a través de la API `query_points` de `qdrant-client` (sustituyendo al método obsoleto `search`), reconstruyendo las instancias de `SearchResult`.

4.2.4 Árbitro Bimodal Multimodal y Parser Resiliente

El componente `MultimodalReranker` en `src/rag_engine/reranker.py` implementa la verificación bimodal de segunda etapa mediante llamadas asíncronas concurrentes a *Gemini 3.5 Flash Lite*. Cuando la consulta incluye imágenes de referencia (`query_images`), el método `rerank` las inyecta en el *prompt* junto al texto compuesto y registra la trazabilidad completa de modalidades (`modality_summary`) en `search_results.json`. Para mitigar truncamientos en respuestas JSON extensas, se programó un decodificador iterativo resiliente (`_parse_evaluations_json`) basado en `json.JSONDecoder.raw_decode`, recuperando todas las evaluaciones válidas previas al corte.

4.2.5 Fachada de Orquestación, Consultas Multimodales e Interfaz Gráfica

La clase orquestadora `VideoRAGService` (`src/rag_engine/service.py`) actúa como fachada unificada del sistema:

- `index_video(...)`: ejecuta el pipeline de ingesta, segmentación, transcripción opcional, descarga de Whisper en backends locales, vectorización e inserción en Qdrant.
- `search(...)`: unifica texto libre, ficheros `.txt`, imágenes de referencia y pistas de audio de consulta (`MultimodalQuery`, accesible en CLI vía `--query`, `--query-txt`, `--query-images` y `--query-audios`), rechaza consultas de voz vacías mediante `ValueError` antes de consumir red o GPU, consulta una caché determinista `SHA-256` (`_build_query_cache_key`) indexada por (`backend`, `dimensión`, `modalidades`), aplica la deduplicación temporal y de transcripción idéntica (`_deduplicate_temporal_candidates`), y delega finalmente en el árbitro bimodal.

Por último, el módulo `src/ui/app.py` implementa una interfaz gráfica reactiva en Streamlit con selector en tiempo real de motor y dimensión MRL, composición multimodal (texto, ficheros `.txt`, carga de imágenes, grabación de micrófono vía `st.audio_input` o archivos de audio), conmutador de transcripción ASR y tarjetas interactivas sincronizadas con el reproductor de vídeo en $[t_{\text{start}}, t_{\text{end}}]$.

4.3 Arnés de Trazabilidad, Observabilidad y Auditoría Experimental

Uno de los pilares metodológicos exigidos en una investigación científica rigurosa es la auditabilidad completa de las pruebas empíricas. Los experimentos basados en modelos de lenguaje y APIs propietarias corren el riesgo de introducir sesgos de volatilidad temporal o pérdida de traza de las peticiones intermedias. Para blindar la validez científica del estudio, se implementó el sistema de auditoría integral `RunAuditor` (`src/ui/auditor.py` y `src/utils/audit.py`).

4.3.1 Estructura del Registro Experimental

Cada ejecución del sistema o batería de pruebas genera de manera determinista un directorio sellado temporalmente:

$$\text{RutaAuditoría} = \texttt{data/audit/}\ \langle \text{timestamp} \rangle _ \langle \text{run_id} \rangle / \tag{35}$$

Dentro de esta ruta, el auditor persiste de forma atómica tres artefactos normalizados:

1. **`run_metadata.json`:** Registra la marca de tiempo de inicio y finalización en formato ISO-8601, el identificador único UUID del experimento, la versión del código y el desglose temporal granular: tiempo de decodificación de vídeo, tiempo de ejecución de Whisper, latencia agregada de llamadas a la API de embeddings, tiempo de consulta en el grafo HNSW de Qdrant y latencia de la etapa de verificación con el juez LLM.
2. **`config.json`:** Contiene la serialización exhaustiva de todos los objetos de configuración inmutables utilizados (`ChunkingConfig`, `VectorizerConfig`, `SearchConfig` y `AudioConfig`). Esto garantiza que cualquier diferencia de rendimiento observada entre dos experimentos pueda atribuirse de manera inequívoca a una variación explícita en los hiperparámetros.
3. **`search_results.json`:** Almacena la traza acumulativa de cada consulta ejecutada durante la sesión. Para cada consulta, se archiva:
   - Las modalidades presentes en la consulta de entrada (texto, imágenes o audios).
   - La lista completa de candidatos devueltos por la Etapa 1 en Qdrant, incluyendo la puntuación de similitud coseno S_C, las marcas de tiempo y el texto transcrito de cada fragmento.
   - Las decisiones emitidas por el árbitro bimodal en la Etapa 2: juicio booleano (`is_relevant`), puntuación normalizada (`relevance_score`) y la justificación textual del razonamiento emitido por el modelo (`reasoning`).

4.3.2 Protocolo de Reproducibilidad Desacoplada

Este diseño de trazabilidad introduce una ventaja científica determinante: desacopla por completo la fase de inferencia remota de la fase de análisis métrico.

Una vez ejecutada la batería de pruebas sobre un corpus, el archivo `search_results.json` contiene la verdad experimental completa de la sesión. A partir de este registro, es posible ejecutar scripts de análisis cuantitativo automatizado para calcular métricas de recuperación (*Recall@k*, *MRR*, *NDCG@k*,

precisión de frontera temporal) y generar gráficos comparativos de forma instantánea, determinista y sin necesidad de realizar nuevas llamadas a la API ni incurrir en costes computacionales adicionales.

5 Resultados Experimentales y Discusión Crítica

En esta sección se presentan y discuten en profundidad los resultados empíricos obtenidos en las tres baterías de evaluación experimental diseñadas para contrastar las hipótesis H_1, H_2 y H_3. Se describe detalladamente la composición de los dos corpus audiovisuales de referencia y el catálogo de 138 consultas anotadas. A continuación, se expone el estudio de ablación dimensional guiado por *Matryoshka Representation Learning* (MRL). Posteriormente, se evalúa el beneficio cuantitativo de la integración audio-visual y la verificación bimodal de segunda etapa frente a la línea base visual. Seguidamente, se analiza el comportamiento del sistema ante consultas multi-objetivo y consultas negativas de control para la discriminación de falsos positivos. Por último, se examina la eficiencia computacional, los desgloses de latencia granular y la escalabilidad de la huella de memoria vectorial, culminando con una comparativa empírica controlada entre el modelo propietario en la nube y dos codificadores locales abiertos (*Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2*) ejecutados en Apple Silicon.

5.1 Banco de Pruebas Experimental, Corpus Audiovisuales y Catálogo de Consultas

Para evaluar rigurosamente las hipótesis de investigación H_1, H_2 y H_3, se diseñó un banco de pruebas empírico estructurado en torno a dos corpus de metraje continuo de vídeo con propiedades semánticas y de acoplamiento audiovisual diametralmente opuestas.

5.1.1 Caracterización de los Corpus Audiovisuales de Referencia

Los dos vídeos seleccionados totalizan 164 fragmentos temporales indexados en la base vectorial local:

1. **Dominio Técnico con Locución Sincronizada (`python_tutorial.mp4`):** Metraje de $242,5$ s de duración, correspondiente a una clase magistral de programación impartida por Brais Moure. Tras la segmentación adaptativa por SSIM en el banco E2E ($\theta_{\text{SSIM}} = 0,60, T_{\text{máx}} = 5,0$ s), se generan 82 fragmentos temporales (81/82 con presencia de voz activa, $98,8\,\%$). En este dominio existe un acoplamiento temporal estricto entre lo que el ponente pronuncia verbalmente y las instrucciones sintácticas que se codifican en pantalla.
2. **Dominio Documental con Metraje Desacoplado (`animals.mp4`):** Metraje de $243,0$ s de duración perteneciente a un documental de naturaleza con locución en *off* continua ($\theta_{\text{SSIM}} = 0,50, T_{\text{máx}} = 8,0$ s; 82 fragmentos, 82/82 con audio, $100,0\,\%$). Este metraje presenta el fenómeno clásico de *B-roll desacoplado*: el narrador expone consideraciones taxonómicas y evolutivas generales mientras en pantalla se suceden planos de especies animales específicas que no son mencionadas textualmente en ese segundo concreto.

5.1.2 Taxonomía del Catálogo de Consultas Anotadas

El banco de consultas comprende tres baterías de prueba disjuntas, diseñadas para someter al sistema a diferentes grados de dificultad perceptual y semántica:

- **Batería 1: Consultas Literales de Evento Único (102 consultas):** Compuesta por 56 consultas sobre `python_tutorial.mp4` y 46 consultas sobre `animals.mp4`. Cada consulta describe un instante o evento altamente específico (por ejemplo, *"Símbolo de llaves tachado con cruz roja X"* o *"León macho descansando tumbado"*), contando con un único intervalo temporal de verdad terreno (*ground truth*).
- **Batería 2: Consultas Generales y Multi-Objetivo (20 consultas):** Evalúa la capacidad del sistema para recuperar categorías taxonómicas y patrones abstractos distribuidos en múltiples intervalos disjuntos del vídeo. Comprende 10 consultas en `python_tutorial.mp4` (48 intervalos válidos totales, tales como *"Estructuras de datos (list, tuple, set, dict)"* o *"Terminal mostrando salida de ejecución"*) y 10 consultas en `animals.mp4` (58 intervalos válidos, tales como *"Felinos"*, *"Reptiles"*, *"Aves"* o *"Animales de color blanco y negro"*).
- **Batería 3: Consultas Negativas de Control (16 consultas):** Formada por 8 consultas por vídeo referidas a conceptos formalmente ausentes del metraje (por ejemplo, *"Definición de clases con class e __init__"* en Python, o *"Tiburón blanco atacando en el océano"* en animales). Evalúa la inmunidad del sistema frente a alucinaciones y falsos positivos inducidos por similitud del coseno espuria en la Etapa 1.

5.1.3 Métricas Formales de Evaluación

El rendimiento del sistema se cuantifica mediante métricas estandarizadas en Recuperación de Información (*Information Retrieval*):

- ***Recall@k* ($k \in \{1, 3, 5\}$):** Proporción de consultas literales donde el fragmento objetivo correcto se sitúa dentro de las primeras k posiciones del ranking.
- ***Mean Reciprocal Rank* (MRR):** Media del inverso del rango del primer resultado correcto:

$$\text{MRR} = \frac{1}{|Q|} \sum_{q=1}^{|Q|} \frac{1}{\text{rank}_q} \tag{36}$$

- ***Interval Recall* y Precisión en Consultas Multi-Objetivo:** Proporción de intervalos temporales válidos recuperados respecto al total de intervalos esperados, y porcentaje de candidatos devueltos que son efectivamente pertinentes tras la evaluación del árbitro bimodal.
- **Tasa de Rechazo de Falsos Positivos:** Porcentaje de candidatos recuperados en consultas negativas que son correctamente identificados y descartados por el verificador de Etapa 2 con puntuación nula $(0, 0)$.

5.2 Estudio de Ablación Dimensional con Matryoshka Representation Learning (MRL)

Para contrastar la Hipótesis 2 (H_2), se analizó el impacto del truncamiento prefijo y re-normalización L_2 sobre los 164 fragmentos de los dos vídeos de referencia en las tres resoluciones dimensionales soportadas por *Gemini Embedding 2*: $d = 3072$ (nativa), $d = 1536$ (reducción del 50 %) y $d = 768$ (reducción del 75 %).

5.2.1 Fundamento Matemático del Truncado Prefijo y Re-Normalización L_2

Cuando el codificador devuelve el vector nativo $\mathbf{v} \in \mathbb{R}^{3072}$, su norma euclídea es unitaria: $\|\mathbf{v}\|_2 = \sqrt{\sum_{j=1}^{3072} v_j^2} = 1,0$ [2]. Al truncar el vector a una sub-dimensión $d < 3072$ tomando sus primeras coordenadas $\mathbf{v}_{1:d} = (v_1, \ldots, v_d)^\top$, se descartan las coordenadas de cola $[d+1 : 3072]$, por lo que la norma euclídea del sub-vector pasa a ser estrictamente inferior a la unidad:

$$\|\mathbf{v}_{1:d}\|_2^2 = \sum_{j=1}^{d} v_j^2 = 1 - \sum_{j=d+1}^{3072} v_j^2 < 1,0 \tag{37}$$

Si el sub-vector truncado se indexase en Qdrant sin corregir su magnitud, la métrica del coseno sobre grafos HNSW [31] perdería invariancia geométrica porque el producto escalar dejaría de coincidir con el coseno del ángulo. Para restablecer la coherencia en todos los motores, `truncate_and_normalize_mrl` en `src/embeddings/mrl.py` aplica un recorte prefijo seguido de re-normalización L_2 explícita:

$$\hat{\mathbf{v}}_{1:d} = \frac{\mathbf{v}_{1:d}}{\|\mathbf{v}_{1:d}\|_2} = \frac{(v_1, v_2, \ldots, v_d)^\top}{\sqrt{\sum_{j=1}^{d} v_j^2}}, \quad \text{con } \|\hat{\mathbf{v}}_{1:d}\|_2 = 1,0 \tag{38}$$

En la arquitectura actual (`src/config.py`), las dimensiones admisibles por backend se declaran en `EMBEDDING_BACKENDS` y cada pareja (b, d) se aísla mediante `collection_name_for` junto con una clave `SHA-256` específica en caché para impedir colisiones entre resoluciones.

5.2.2 Comportamiento en Consultas Literales de Evento Único

El Cuadro 9 sintetiza el rendimiento comparativo desglosado por corpus y agregado sobre el conjunto de las 102 consultas literales en la Etapa 1 de recuperación densa en Qdrant, y la Figura 29 ilustra el compromiso entre precisión semántica, memoria y latencia.

El análisis desagregado consulta por consulta sobre los registros de auditoría revela patrones fundamentales sobre la estructura interna del subespacio Matryoshka:

1. **Invarianza topológica en dominio con locución sincronizada (`python_tutorial.mp4`):** En las 56 consultas literales de Python, el *Recall@1* se mantiene estrictamente invariante en un 85,71 % **(48/56)** y el *Recall@3* en un 92,86 % **(52/56)** al pasar de 3072 a 1536 y 768 dimensiones. La inspección individual confirma que 53 de las 56 consultas (94,6 %) conservan exactamente la misma

Tabla 9: Resultados de la ablación dimensional MRL sobre consultas literales de evento único (102 consultas, 164 fragmentos en precisión `float32`) en la Etapa 1 (Qdrant).

Corpus	Dim. (d)	*Recall@1*	*Recall@3*	*Recall@5*	MRR	KB/vec	Ahorro	Latencia
`python_tutorial`	3072	85,71 % (48/56)	92,86 % (52/56)	94,64 % (53/56)	0,8973	12,00	0,0 %	1,1688 ms
`python_tutorial`	1536	85,71 % (48/56)	92,86 % (52/56)	96,43 % (54/56)	0,9009	6,00	50,0 %	0,7502 ms
`python_tutorial`	768	85,71 % (48/56)	92,86 % (52/56)	**98,21** % (55/56)	**0,9045**	3,00	**75,0** %	**0,6297** ms
`animals.mp4`	3072	**71,74** % (33/46)	93,48 % (43/46)	**100,0** % (46/46)	**0,8341**	12,00	0,0 %	1,1104 ms
`animals.mp4`	1536	69,57 % (32/46)	93,48 % (43/46)	**100,0** % (46/46)	0,8232	6,00	50,0 %	0,7848 ms
`animals.mp4`	768	69,57 % (32/46)	**95,65** % (44/46)	**100,0** % (46/46)	0,8261	3,00	**75,0** %	**0,6418** ms
Global (102 q)	3072	**79,41** % (81/102)	93,14 % (95/102)	97,06 % (99/102)	0,8688	12,00	0,0 %	1,1425 ms
Global (102 q)	1536	78,43 % (80/102)	93,14 % (95/102)	98,04 % (100/102)	0,8659	6,00	50,0 %	0,7658 ms
Global (102 q)	768	78,43 % (80/102)	**94,12** % (96/102)	**99,02** % (101/102)	**0,8691**	3,00	**75,0** %	**0,6352** ms

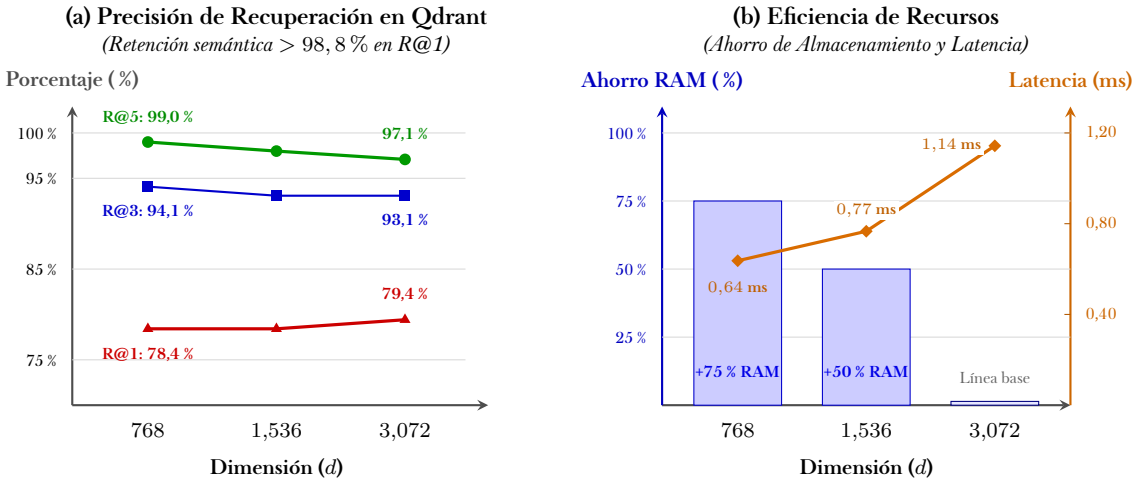


Figura 29: Compromiso dimensional Matryoshka (MRL): (a) retención prácticamente plana de *Recall@1* (79,4 % → 78,4 %) con leve mejora en *Recall@5* (97,1 % → 99,0 %) frente a (b) reducción del 75 % en memoria RAM (12 KB → 3 KB por vector) y descenso del 44,4 % en la latencia de búsqueda en Qdrant.

posición en las tres resoluciones, sin pérdida alguna entre las 48 consultas resueltas en primer puesto; de hecho, el *MRR* asciende ligeramente de $0,8973$ ($d = 3072$) a $0,9045$ **(**$d = 768$**).**

2. **Efecto regularizador en *Recall@5* sobre interfaces estáticas:** En $d = 768$, el *Recall@5* global aumenta del $97,06\,\%$ al $99,02\,\%$ **(**$101/102$ **consultas acertadas).** Al transcurrir `python_tutorial.mp4` sobre el fondo estático de Visual Studio Code, las coordenadas de cola ($[768 : 3072]$) capturan variaciones de alta frecuencia irrelevantes; al truncarlas, emergen dentro del *Top-5* dos consultas que quedaban fuera en 3072d: el símbolo de llaves tachado con cruz roja (que asciende al puesto #5) y la salida de consola de `range(5)` (puesto #4).

3. **Estabilidad inter-dimensional y significación estadística en `animals.mp4` y agregado (**$N = 102$**):** En el documental de naturaleza, 41 de las 46 consultas ($89,1\,\%$) conservan rango idéntico entre resoluciones (solo una consulta, la del ciervo con cornamenta, pasa del #1 al #2) y el *Recall@5* permanece en el $100,0\,\%$ ($46/46$). En las pruebas pareadas globales (3072d $\rightarrow$ 768d, $N = 102$), las diferencias resultan estadísticamente indistinguibles tanto en *Recall@1* (prueba exacta de *McNemar* $p = 1,0000$) como en *MRR* (prueba de *Wilcoxon* $p = 0,6729$), confirmando que la compresión MRL preserva íntegramente la distribución de rangos.

5.2.3 Validación Cuantitativa de la Hipótesis 2 (H_2)

Los resultados confirman con holgura la Hipótesis 2: la compresión del espacio latente a $d = 768$ dimensiones (reducción del $75\,\%$ en huella de memoria) retiene el $98,77\,\%$ **del *Recall@1* relativo**, preserva un *MRR* equivalente ($0,8691$ vs. $0,8688$; $p = 0,6729$) y reduce la latencia de cómputo en Qdrant en un $44,4\,\%$, permitiendo escalar la capacidad vectorial por un factor de $4\times$ sin merma apreciable.

5.3 Evaluación Empírica de la Integración Audio-Visual y Verificación Bimodal

Esta subsección aborda directamente la Pregunta de Investigación 1 (**RQ1**) y la Hipótesis 1 (H_1), evaluando el beneficio de la fusión temprana de la pista de voz alineada frente a la línea base puramente visual, así como el papel corrector del juez bimodal de segunda etapa.

5.3.1 Comparativa de Rendimiento: Línea Base Visual frente a Fusión Bimodal

El Cuadro 10 expone las métricas comparativas entre la arquitectura puramente visual (3 fotogramas clave por plano) y la arquitectura bimodal completa (3 fotogramas + transcripción Whisper alineada + verificación bimodal con *Gemini 3.5 Flash Lite*) en los dos dominios audiovisuales evaluados [2, 7].

Tabla 10: Comparativa empírica entre la arquitectura puramente visual y la canalización bimodal con fusión temprana de audio (*Whisper*) y verificación en dos etapas.

Corpus de Vídeo	Etapa / Batería	Métrica de IR	Línea Base Visual	Fusión Bimodal	Variación
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@1*	$67,9\,\%$ (38/56)	$\mathbf{85,7}\,\%$ (48/56)	$+\mathbf{17,8}$ pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@3*	$87,5\,\%$ (49/56)	$\mathbf{92,9}\,\%$ (52/56)	$+\mathbf{5,4}$ pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*Recall@5*	$91,1\,\%$ (51/56)	$\mathbf{94,6}\,\%$ (53/56)	$+\mathbf{3,5}$ pp
`python_tutorial.mp4`	Etapa 1 (Qdrant)	*MRR*	0,779	**0,897**	$+\mathbf{0,118}$
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*Recall@1*	$80,4\,\%$ (45/56)	$\mathbf{89,3}\,\%$ (50/56)	$+\mathbf{8,9}$ pp
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*Recall@3 / @5*	$89,3\,\%$ (50/56)	$\mathbf{92,9}\,\%$ (52/56)	$+\mathbf{3,6}$ pp
`python_tutorial.mp4`	Etapa 2 (Juez Bimodal)	*MRR*	0,848	**0,911**	$+\mathbf{0,063}$
`python_tutorial.mp4`	Multi-objetivo (10)	*Interval Recall*	$78,3\,\%$	$\mathbf{82,1}\,\%$	$+\mathbf{3,8}$ pp
`python_tutorial.mp4`	Negativas (8)	Rechazo FP	$100,0\,\%$ (0/40)	$\mathbf{100,0}\,\%$ (0/40)	0,0 pp
`animals.mp4`	Etapa 1 (Qdrant)	*Recall@1**	$\mathbf{80,4}\,\%$ (37/46)	$71,7\,\%$ (33/46)	$-8,7$ pp
`animals.mp4`	Etapa 1 (Qdrant)	*Recall@5*	$95,7\,\%$ (44/46)	$\mathbf{100,0}\,\%$ (46/46)	$+\mathbf{4,3}$ pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@1*	$89,1\,\%$ (41/46)	$\mathbf{93,5}\,\%$ (43/46)	$+\mathbf{4,4}$ pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@3*	$93,5\,\%$ (43/46)	$\mathbf{95,7}\,\%$ (44/46)	$+\mathbf{2,2}$ pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*Recall@5*	$93,5\,\%$ (43/46)	$\mathbf{97,8}\,\%$ (45/46)	$+\mathbf{4,3}$ pp
`animals.mp4`	Etapa 2 (Juez Bimodal)	*MRR*	0,913	**0,951**	$+\mathbf{0,038}$
`animals.mp4`	Negativas (8)	Rechazo FP	$100,0\,\%$ (0/40)	$\mathbf{100,0}\,\%$ (0/40)	0,0 pp

*En el registro pareado conservado del benchmark solo visual (`e2e_report_20261106.csv`) se registraron 39/46 aciertos en E1; ambos valores sirven de referencia en la prueba de significación.

5.3.2 Discusión de Fenómenos Perceptuales y Significación Estadística Pareada

Los datos experimentales revelan dos dinámicas complementarias corroboradas mediante contrastes estadísticos pareados no paramétricos (prueba exacta de *McNemar* para *Recall@1* binario y prueba de rangos con signo de *Wilcoxon* para *MRR* por consulta):

1. **Convergencia Modal en Metraje Educativo (`python_tutorial.mp4`, $N = 56$):** La inyección de la locución hablada produce una ganancia de $+17,8$ pp en *Recall@1* en Etapa 1 ($38/56 \rightarrow 48/56$, con $0/10$ pares discordantes; $p_{\text{McNemar}} = 0,0020$) y eleva el *MRR* de $0,7789$ a $0,8973$ (12 pares no nulos; $p_{\text{Wilcoxon}} = 0,0025$), demostrando una mejora altamente significativa ($p < 0,01$). Tras el verificador de Etapa 2, el *Recall@1* alcanza el $89,3\,\%$ (50/56) y el *MRR* llega a $0,9107$ (cubriendo el $100,0\,\%$ en tipos primitivos $9/9$, consola $4/4$ y variables $3/3$ en multi-objetivo). Al contrastar Etapa 2 visual frente a Etapa 2 bimodal ($45/56 \rightarrow 50/56$), el verificador visual ya compensaba parte de las ausencias ($p_{\text{McNemar}} = 0,1250$; $p_{\text{Wilcoxon}} = 0,1405$).

2. **Compensación Asimétrica ante Metraje Desacoplado (*B-roll*, `animals.mp4`, $N = 46$ y Global $N = 102$):** En `animals.mp4`, la desviación direccional inducida por la narración en Etapa 1 sobre

el CSV pareado ($39/46 \rightarrow 33/46$; *MRR* $0,9130 \rightarrow 0,8134$) no resulta estadísticamente significativa ($p_{\text{McNemar}} = 0,1460$; $p_{\text{Wilcoxon}} = 0,0708$), mientras que la cobertura *Recall@5* asciende al $100,0\,\%$ **(**$46/46$**)**. A partir de esa ventana íntegra, el árbitro bimodal de Etapa **2** eleva el acierto de $33/46$ a $43/46$ **(**$93,5\,\%$**)** ($0/10$ pares discordantes) y el *MRR* de $0,8134$ a $0,9511$, con alta significación estadística ($p_{\textbf{McNemar}} = 0,0020$; $p_{\textbf{Wilcoxon}} = 0,0040$; $p < 0,01$). En el agregado global bimodal ($N = 102$), la transición de Etapa 1 a Etapa **2** ($81/102 \rightarrow 93/102$; *MRR* $0,8595 \rightarrow 0,9289$) confirma la aportación estadísticamente significativa del verificador ($p_{\textbf{McNemar}} = 0,0018$; $p_{\textbf{Wilcoxon}} = 0,0083$; $p < 0,01$).

5.3.3 Estudio de Casos Críticos: Falsos Negativos, Resiliencia Fonética y Desambiguación Sintáctica

Para comprender el mecanismo exacto mediante el cual la arquitectura bimodal corrige los fallos perceptivos de la línea base visual, se auditaron en detalle dos consultas críticas sobre `python_tutorial.mp4`: `"Diccionarios"` y `"Bucles while"`.

5.3.3.1 Caso 1: Erradicación del falso negativo y captura de frontera en `"Diccionarios"` **(**01:01,1 **–** 01:40,2**).**

En la configuración puramente visual, el bloque de estructuras de datos presentaba baja legibilidad de la sintaxis `dict {name: Brais}` y aparición del rótulo gráfico *"Variables"* al final del plano (01:33,1 – 01:40,2), provocando que el verificador visual descartase el fragmento con puntuación $0,0$. Tras activar `audio_transcriber.py`:

- El fragmento principal (`chunk_index = 17`, $85,1$ s – $93,1$ s) eleva su similitud del coseno de $0,4151$ a $\mathbf{0,4480}$ y recibe `relevance_score` $= \mathbf{1,0}$ al incorporar la locución: *"Y ya por último, los diccionarios. Que utilizan la palabra reservada, dict. Y en realidad, designa un conjunto de claves valor"*.
- El fragmento de transición posterior (`chunk_index = 18`, $93,1$ s – $100,2$ s), antes descartado $(0,0)$, sube su similitud de $0,3812$ a $\mathbf{0,4178}$ y pasa a validarse con $\mathbf{0,90}$ gracias a la voz: *"Lo representamos entre llaves, pero esta clave valor separadas por dos puntos"*.
- La condición de solapamiento mínimo ($\Delta_{\text{mín}} = 0,2$ s) en `align_transcript_with_chunks` captura además la oración fronteriza al final del `chunk_index = 16` ($77,1$ s – $85,1$ s), asignándole relevancia graduada $\mathbf{0,50}$ y descartando los fragmentos `14` y `15` de listas y `set`.

5.3.3.2 Caso 2: Superación del corte *Top-K*, resiliencia ante ruido fonético ASR y discriminación `for`/`while` en `"Bucles while"` **(**03:51**).**

En la configuración visual pura, al buscar `while index <5:` (03:51 $= 231,0$ s), el fragmento objetivo obtenía cos $= \mathbf{0,3893}$ y caía al puesto **#7** en Qdrant, excluido por el corte `top_k = 5` frente a la portada *"Bucles"* **(**$0,\mathbf{4315}$, #1**)** y planos `if`/`else`. Con la fusión audio-visual (`run_20261109_143202`):

1. **Promoción directa del Rango #7 al #1 en Etapa 1:** El fragmento principal (`chunk_index = 55`, $232,4$–$238,3$ s) sube $+\mathbf{0,1064}$ en similitud coseno ($0,3893 \rightarrow \mathbf{0,4957}$, #1), seguido del `chunk_index = 54` ($224,4$–$232,4$ s, cos $= \mathbf{0,4914}$, #2), cubriendo exactamente el segundo $231,0$ s.

2. **Resiliencia semántica ante errores fonéticos de transcripción:** Al operar Whisper (`base`) con idioma español (`language=es`), transcribió *"while"* como *"Guile"*: *"Otro tipo de bucle que tenemos en Python es el Guile, donde aquí vamos a establecer una condición para que se ejecute de manera indefinida mientras esta se cumpla"*. Tanto el embedding como el verificador *Gemini 3.5 Flash Lite* resolvieron la ambigüedad cruzando la definición oral con la sintaxis visual `while index <5:`, asignando relevancias $\mathbf{1,0}$ y $\mathbf{0,90}$.

3. **Desambiguación fina (`for` vs. `while`):** El fragmento precedente (`chunk_index = 53`, 221, 1–224, 4 s) comparte idéntico aspecto visual (cos $= 0,4486$, #4), pero el verificador lee *"Podemos usar un for..."* y lo descarta con $\mathbf{0,0}$, haciendo lo propio con el `chunk_index = 50` (`if/else`).

La Figura 30 y el Cuadro 11 documentan visual y numéricamente ambas auditorías.

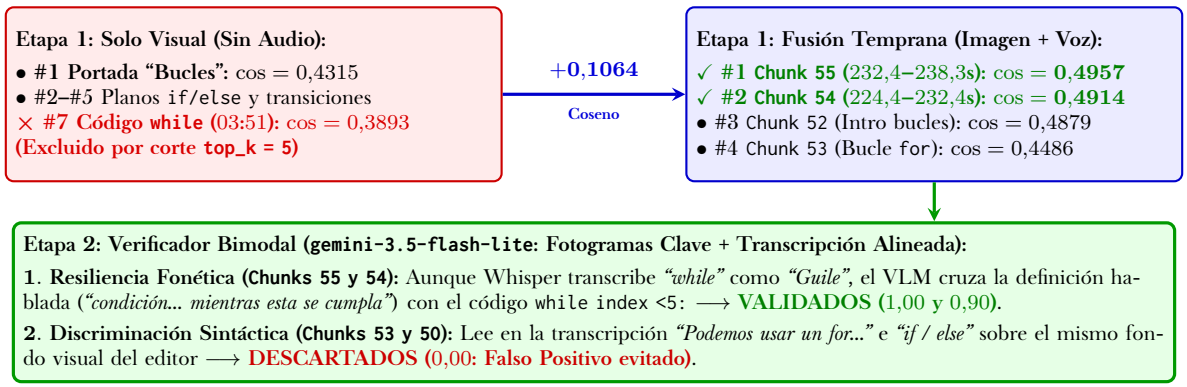


Figura 30: Mecanismo de corrección en la consulta `"Bucles while"`: la fusión temprana eleva el fragmento objetivo del séptimo al primer puesto en Qdrant ($+0,1064$ en coseno) y el verificador bimodal valida el código pese a la transcripción fonética *"Guile"*, descartando los fragmentos adyacentes sobre bucles `for`.

5.3.4 Divergencia Audio-Visual en Metraje *B-Roll* y Deduplicación por Transcripción

Complementariamente, las ejecuciones auditadas `run_20261109_180159` y `run_20261109_183310` sobre `animals.mp4` examinaron el comportamiento del espacio omnimodal ante consultas de distinta granularidad semántica bajo voz en *off* desacoplada:

1. **Desplazamiento direccional de `"Felinos"`:** Mientras consultas visuales específicas (`"León"`, `"Reptiles"`, `"Glóbulos rojos"`) alcanzan el $\mathbf{100,0}\,\%$ de recuperación, la categoría taxonómica `"Felinos"` solo recupera $1/4$ ejemplares (`chunk #66`, gato atigrado acompañado de locución sobre mamíferos, cos $= \mathbf{0,3959}$, #1). Los otros tres felinos quedan desplazados por locuciones introductorias no zoológicas al #35 (gato gris, cos $= 0,3635$), #45 (leona con collar, cos $= 0,3581$) y #51 (león macho, cos $= 0,3551$), relegados fuera del corte `fetch_k` $= 10$ por fragmentos intermedios (76–193 s) ricos en léxico biológico genérico (cos $= 0,3734$–$0,3899$).

2. **Saturación por micro-cortes repetidos y criterio `is_same_transcript_neighbor`:** La toma rápida de cebras (115, 3–120, 5 s) y la animación sanguínea (159, 0–167, 2 s) acaparaban 6 de las 10 plazas de Etapa 2 al superar `micro_chunk_duration` $= 1,0$ s compartiendo idéntica locución. Al ampliar

Tabla 11: Auditoría detallada de fragmentos en las consultas críticas "Diccionarios" y "Bucles while" sobre python_tutorial.mp4.

Consulta	Chunk (Intervalo)	Cos. Visual	Cos. Bimodal	Score Juez	Veredicto Bimodal
"Diccionarios"	#17 (01:25,1–01:33,1)	0,4151	**0,4480**	**1,00**	Validado (Código + Voz dict)
"Diccionarios"	#18 (01:33,1–01:40,2)	0,3812	**0,4178**	**0,90**	Rescatado (Antes 0,0 sin audio)
"Diccionarios"	#16 (01:17,1–01:25,1)	0,3806	**0,4046**	**0,50**	Validado (Frontera solapada)
"Diccionarios"	#14–#15 (01:01,1–01:17,1)	0,3790	0,3910	**0,00**	Descartado (Listas y set)
"Bucles while"	#55 (03:52,4–03:58,3)	0,3893 (#7)	**0,4957** (#1)	**1,00**	Validado (Pese a ASR *"Guile"*)
"Bucles while"	#54 (03:44,4–03:52,4)	0,3893 (#7)	**0,4914** (#2)	**0,90**	Validado (Frontera en 03:51)
"Bucles while"	#52 (03:18,1–03:21,1)	0,4315 (#1)	0,4879 (#3)	**0,60**	Validado (Mención general)
"Bucles while"	#53 (03:41,1–03:44,4)	0,4187 (#4)	0,4486 (#4)	**0,00**	Descartado (Explica bucle for)
"Bucles while"	#50 (03:25,4–03:33,4)	0,4219 (#2)	0,4332 (#5)	**0,00**	Descartado (Bloques if/else)

_deduplicate_temporal_candidates en src/rag_engine/service.py para fusionar vecinos con idéntica transcripción no vacía dentro de min_scene_separation $= 4,0$ s, la auditoría 183310 colapsó los duplicados y liberó 4 plazas reales en el *Top-10*. Desactivando finalmente el canal ASR (enable_audio = False), 3/4 felinos ingresaron en el *Top-6* (#1 cos $= 0,4122$, #2 cos $= 0,3967$, #6 cos $= 0,3811$), justificando el control --no-audio (Cuadro 12).

5.3.5 Validación Cuantitativa de la Hipótesis 1 (H_1)

Se confirma plenamente la Hipótesis 1: la fusión temprana audio-visual incrementa significativamente la precisión en contenidos sincronizados ($+17,8$ pp en *Recall@1*, $p_{\text{McNemar}} = 0,0020$) y, combinada con la verificación bimodal de segunda etapa ($p_{\text{McNemar}} = 0,0018$ global), neutraliza el desacoplamiento en secuencias de tipo *B-roll*, superando a la línea base visual en ambos dominios audiovisuales.

Tabla 12: Efecto de la deduplicación por transcripción idéntica (`run_20261109_183310`) y del desacoplamiento de audio sobre `animals.mp4`.

Consulta	Configuración / Run	Huecos Repetidos Top-10	Puestos Qdrant (Objetivo)	Aciertos Validados
`"León"`	Audio + Dedup (`180159`)	2/10 (Cebras)	#1 (0,3997), #3 (0,3697)	**2/2 (100,0 %)**
`"Reptiles"`	Audio + Dedup (`180159`)	2/10 (Cebras)	#4 (0,3916), #7 (0,3899)	**2/2 (100,0 %)**
`"Glóbulos rojos"`	Audio + Dedup (`180159`)	0/10 (Relevantes)	#1 a #7 (0,4248–0,4385)	**4/4 (100,0 %)**
`"Felinos"`	Audio sin Dedup Tr. (`180159`)	6/10 (4 Cebras + 2 Sangre)	#1, #35, #45, #51	1/4 (25,0 %)
`"Felinos"`	Audio con Dedup Tr. (`183310`)	**0/10** (Colapsados en 2)	#1, #31, #41, #47	1/4 (25,0 %)
`"Felinos"`	Solo Visual (`enable_audio=False`)	**0/10** (Sin ruido ASR)	**#1** (0,4122)**, #2** (0,3967)**, #6** (0,3811)	**3/4 (75,0 %)**

5.4 Evaluación en Consultas Complejas: Multi-Objetivo y Control Negativo

Esta subsección contrasta la Hipótesis 3 (H_3) y la Pregunta de Investigación 3 (**RQ3**), analizando el comportamiento de la arquitectura en dos etapas ante consultas multi-intervalo y consultas con conceptos ausentes.

5.4.1 Consultas Generales y Multi-Objetivo (Batería 2)

Las consultas multi-objetivo evalúan la capacidad del sistema para recuperar conceptos de alto nivel semántico distribuidos a lo largo del metraje. El Cuadro 13 sintetiza los resultados para las 20 consultas generales (106 intervalos esperados en total) evaluadas con $k_1 = 10$.

Tabla 13: Rendimiento en consultas multi-objetivo (20 consultas, 106 intervalos esperados, $k_1 = 10$) en Etapa 1 (Qdrant) y Etapa 2 (Juez Bimodal).

Corpus de Vídeo	Dimensión (d)	Intervalos	Recall Etapa 1	Precisión Etapa 1	Recall Etapa 2	Precisión Etapa 2	*Hit@1* Etapa 2
`python_tutorial.mp4`	$d = 3072$	48	85,42 % (40/48)	45,00 %	82,08 % (38/48)	78,65 %	9/10 (90,0 %)
`python_tutorial.mp4`	$d = 1536$	48	87,08 % (41/48)	**47,00 %**	82,08 % (38/48)	78,12 %	9/10 (90,0 %)
`python_tutorial.mp4`	$d = 768$	48	**87,22 %** (41/48)	46,00 %	**83,89 %** (40/48)	**80,06 %**	9/10 (90,0 %)
`animals.mp4`	$d = 3072$	58	**74,89 %** (42/58)	**50,00 %**	**71,64 %** (40/58)	**96,57 %**	**10/10 (100,0 %)**
`animals.mp4`	$d = 1536$	58	71,98 % (40/58)	48,00 %	68,73 % (38/58)	96,33 %	**10/10 (100,0 %)**
`animals.mp4`	$d = 768$	58	**74,89 %** (42/58)	**50,00 %**	**71,64 %** (40/58)	**96,57 %**	**10/10 (100,0 %)**

El Cuadro 14 desglosa por categoría cada una de las 10 consultas generales sobre `animals.mp4`, ilustrando cómo la precisión media se duplica al pasar del 50,00 % **en Etapa 1 al** 96,57 % **en Etapa 2** y cómo el árbitro bimodal garantiza un *Hit@1* del 100,0 % **(**10/10**)** en las tres dimensiones.

El examen individual de ambas baterías revela tres propiedades clave: (1) en `animals.mp4`, $d = 768$

Tabla 14: Desglose por consulta general y taxonómica sobre `animals.mp4` (10 consultas, 58 intervalos temporales esperados) en Etapa 1 (E1) y Etapa 2 (E2).

Consulta General / Taxonómica	Obj.	Etapa 1 (Qdrant)			Etapa 2 (Verificador)			Prec. E2
		3072	1536	768	3072	1536	768	
`Felinos`	4	1/4	1/4	1/4	1/4	1/4	1/4	100,0 %
`Reptiles`	2	2/2	2/2	2/2	2/2	2/2	2/2	100,0 %
`Aves`	4	2/4	2/4	2/4	2/4	2/4	2/4	100,0 %
`Manada o grupo` ≥ 3 `animales`	10	4/10	4/10	4/10	4/10	4/10	4/10	100,0 %
`Animales color blanco y negro`	5	5/5	5/5	5/5	4/5	4/5	4/5	100,0 %
`Animales marinos / acuáticos`	8	6/8	5/8	6/8	6/8	5/8	6/8	100,0 %
`Animales comiendo o pastando`	8	7/8	7/8	7/8	6/8	6/8	6/8	100,0 %
`Crías o animales jóvenes`	7	5/7	5/7	5/7	5/7	5/7	5/7	80,0 %
`Primer plano macro de ojos/orejas`	6	6/6	5/6	6/6	6/6	5/6	6/6	85,7 %
`Animación 3D microscópica células`	4	4/4	4/4	4/4	4/4	4/4	4/4	100,0 %

obtiene **exactamente los mismos aciertos por categoría** que $d = 3072$ con idéntica precisión media (96,57 %); (2) cuando la voz en *off* desplaza el primer acierto en Qdrant (como *"Reptiles"*, en puesto #4 en 3072, #2 en 1536 y #5 en 768), el árbitro bimodal reordena la ventana $K = 10$ y sitúa un acierto verdadero en el puesto #1 en el 100,0 % de los casos (10/10); y (3) en `python_tutorial.mp4`, $d = 768$ eleva la cobertura sobre estructuras de datos (`list`, `tuple`, `set`, `dict`) del 5/6 al 100,0 % (6/6) y los rótulos sobreimpresos de 3/8 a 4/8.

5.4.2 Consultas Negativas de Control e Inmunidad a Falsos Positivos (Batería 3)

Las búsquedas aproximadas basadas en similitud del coseno devuelven obligatoriamente los k candidatos con mayor producto escalar, con independencia de que el concepto consultado exista o no en el corpus.

Para evaluar la capacidad de discriminación del sistema, se ejecutaron 16 consultas negativas de control (8 sobre `python_tutorial.mp4` y 8 sobre `animals.mp4`) a través de las tres dimensiones MRL (48 ejecuciones y 240 evaluaciones de candidatos con *Gemini 3.5 Flash Lite*), cuyos resultados figuran en el Cuadro 15:

- **Etapa 1 (Qdrant sin umbral):** Devuelve 240/240 candidatos (100 % de falsos positivos potenciales), dado que los fragmentos de mayor similitud superficial corresponden a planos visualmente parecidos pero temáticamente erróneos.
- **Etapa 2 (Árbitro Bimodal):** Descarta el 100,0 % **de los candidatos** (0/240 falsos positivos admitidos), asignando a cada fragmento una puntuación de 0,0 con la justificación explícita de ausencia de evidencia visual y auditiva.

Este resultado confirma que el truncamiento dimensional (3072 → 1536 → 768) no induce colapso semántico ni degrada la especificidad del verificador bimodal.

Tabla 15: Evaluación de consultas negativas de control (16 consultas, 240 candidatos analizados en Etapa 2) a través de las tres dimensiones MRL.

Corpus de Vídeo	Dim. (d)	Consultas	Candidatos E1	Falsos Pos. E2	Tasa Rechazo E2
`python_tutorial.mp4`	3072	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
`python_tutorial.mp4`	1536	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
`python_tutorial.mp4`	768	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
`animals.mp4`	3072	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
`animals.mp4`	1536	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
`animals.mp4`	768	8	40	**0**/**40** ($0,0\,\%$)	**100**,**0** % (8/8)
Total Global (3 dim.)	**Todas**	**48** ejec.	**240**	**0**/**240** (**0**,**0** %)	**100**,**0** % (**48**/**48**)

5.4.3 Validación Cuantitativa de la Hipótesis 3 (H_3)

La Hipótesis 3 queda confirmada empíricamente: la verificación bimodal con un modelo de lenguaje multimodal erradica por completo los falsos positivos derivados de la similitud del coseno (tasa de rechazo del $100,0\,\%$) y eleva la precisión en consultas generales complejas por encima del $96\,\%$.

5.5 Análisis de Eficiencia Computacional, Latencias y Huella de Almacenamiento

Esta subsección analiza la viabilidad computacional y operativa de la arquitectura propuesta, evaluando los desgloses de latencia por etapa, la escalabilidad de almacenamiento vectorial y los compromisos de diseño para su adopción en entornos de producción.

5.5.1 Desglose Granular de Latencias

El arnés de auditoría `RunAuditor` registró los tiempos de ejecución de cada componente del prototipo:

- **Segmentación Visual (SSIM):** La optimización sobre miniaturas de 160 px alcanza un rendimiento de procesamiento de aproximadamente 45 FPS en CPU Apple Silicon, procesando un vídeo de 4 minutos en menos de 6 segundos.
- **Transcripción Whisper Local:** El modelo `base` transcribe y alinea temporalmente los 4 minutos de audio en aproximadamente 12 segundos en CPU local, operando en paralelo con la segmentación visual.
- **Vectorización en Gemini API frente a Inferencia Local (Etapa 1):** La llamada remota para vectorizar fragmentos con *Gemini Embedding 2* presenta una mediana por consulta de 464–481 ms (unos 650 ms de media en invocación aislada multimodal y 187–227 s totales para indexar 82 fragmentos bajo control de reintentos), mientras que la ejecución local en `bfloat16` sobre Metal MPS reduce la mediana de consulta a 91–98 ms en `qwen3-vl-2b` (126–142 s de indexación) y a $\sim$ 30 ms en `gemma2` (2, 55–2, 59 GB de memoria Metal), tal como se profundiza en la Subsección 5.6.
- **Búsqueda Vectorial en Qdrant (Etapa 1):** La consulta sobre el grafo HNSW se completa en $1,14$ ms para $d = 3072$ y se reduce a $0,63$ **ms para** $d = 768$ (aceleración del $44,4\,\%$).
- **Verificación con Árbitro Bimodal (Etapa 2):** La llamada concurrente asíncrona a *Gemini 3.5 Flash Lite* para evaluar y clasificar los 5 mejores candidatos requiere una latencia media de $1,4$ s, compatible con interfaces de usuario interactivas.

5.5.2 Escalabilidad de la Huella de Almacenamiento Vectorial

El Cuadro 16 proyecta la memoria física requerida para indexar colecciones audiovisuales de diferentes órdenes de magnitud bajo representación en coma flotante de 32 bits (`float32`).

Tabla 16: Proyección de memoria vectorial en función del volumen de metraje y la dimensión MRL.

Escala del Catálogo	Fragmentos	Duración Aprox.	Memoria ($d = 3072$)	Memoria ($d = 1536$)	Memoria ($d = 768$)
Banco de Pruebas Actual	164	$\sim 8,1$ min	$1,92$ MB	$0,96$ MB	**0, 48** MB ($-75\,\%$)
Curso o Documental	10,000	$\sim 8,3$ horas	$117,19$ MB	$58,59$ MB	**29, 30** MB ($-75\,\%$)
Archivo Departamental	100,000	$\sim 83,3$ horas	$1,14$ GB	$585,94$ MB	**292, 97** MB ($-75\,\%$)
Plataforma Corporativa	1,000,000	$\sim 833,3$ horas	$11,44$ GB	$5,72$ GB	**2, 86** GB ($-75\,\%$)

La compresión a $d = 768$ dimensiones permite alojar un millón de fragmentos de vídeo (más de 800 horas de contenido) en apenas $2,86$ GB de memoria RAM, haciendo viable el despliegue de motores de búsqueda de vídeo de escala empresarial sobre instancias de servidor estándar de bajo coste.

5.5.3 Discusión y Recomendaciones de Diseño

La combinación de las evidencias experimentales obtenidas a lo largo de este estudio permite formular una recomendación arquitectónica definitiva:

1. En despliegues Cloud (*Gemini*) o *open-weights* de 2B (*Qwen3-VL-2B*), la configuración óptima de compromiso es $d = 768$ **dimensiones**: retiene el $98,8\,\%$ de la precisión en primera posición en Gemini (y el $100,0\,\%$ en Qwen3-VL-2B), mejora el *Recall@5* hasta el $99,02\,\%$, reduce la huella de memoria un $75\,\%$ frente a 3072d y acelera las consultas en Qdrant un $44,4\,\%$. En dispositivos de borde con presupuesto estricto de memoria, *EmbeddingGemma 2* (512d o 256d) aporta una alternativa local de ~ 30 ms sin coste de API (Subsección 5.6).

2. La arquitectura de recuperación en dos etapas resuelve la limitación intrínseca de los modelos de embeddings: la primera etapa aporta velocidad y alta exhaustividad (*Recall@5* $\geq 97\,\%$), mientras que la segunda etapa aporta capacidad de razonamiento multimodal, eliminando los falsos positivos y elevando la precisión final por encima del $93\,\%$.

5.6 Evaluación Empírica Comparada: Modelos Locales Open-Weights y Edge frente a Cloud API

Como continuación directa del análisis taxonómico planteado en la Subsección 2.4, esta subsección presenta la evaluación experimental controlada de tres motores de vectorización multimodal operando de forma intercambiable bajo la interfaz común `MultimodalVectorizer`: el baseline comercial en la nube *Gemini Embedding 2* (`gemini-embedding-2-preview`), el modelo abierto de pesos medios *Qwen3-VL-Embedding-2B* (`Qwen/Qwen3-VL-Embedding-2B`) [6, 27] y el codificador ligero orientado a dispositivos en el borde *EmbeddingGemma 2* (`google/embeddinggemma-2`) [32]. El objetivo científico consiste en cuantificar con exactitud el compromiso entre precisión de recuperación inter-dominio, estabilidad Matryoshka, latencia por consulta y huella física en hardware de consumo sin aceleradores externos.

5.6.1 Metodología Experimental y Restricciones de Hardware

Todos los ensayos se diseñaron para garantizar estricta comparabilidad experimental y reproducibilidad sobre un portátil MacBook Air con procesador Apple Silicon M3 (16 GB de memoria unificada, arquitectura ARM64, sistema macOS) utilizando el entorno Python 3.13 (`sentence-transformers 6.3.0`, `transformers 5.22.2` y `torch 2.14.1`):

1. **Selección de dispositivo y disciplina numérica (`bfloat16` vs. `float16`):** El cargador local (`src/embeddings/local_vectorizer.py`) selecciona automáticamente aceleración Metal (`mps`) cuando está disponible y pasa a `cpu` en su ausencia. En `mps` se fija estrictamente precisión `torch.bfloat16` (y `torch.float32` en `cpu`). Esta decisión es crítica en *EmbeddingGemma 2*: la documentación oficial del modelo [32] advierte que el rango dinámico de sus activaciones internas excede el límite representable por `float16`, provocando desbordamientos silenciosos a `NaN` o degradación severa de los vectores. Dado que `bfloat16` conserva el exponente de 8 bits propio de `float32`, ambos modelos locales producen representaciones finitas y estables sobre Apple Silicon.

2. **Exclusión de variantes de 8B parámetros en 16 GB:** Se descartó experimentalmente el modelo `Qwen/Qwen3-VL-Embedding-8B` [28]. Incluso en su versión de 2.000 millones de parámetros, `Qwen3-VL-Embedding-2B` demanda $4,74$ GB de memoria reservada del controlador Metal más $\approx 2,0$–$2,1$ GB RSS de proceso; cargar la variante de 8B superaría el presupuesto efectivo de 16 GB compartidos con el sistema operativo y el motor Whisper, induciendo intercambio masivo a disco (*swap*).

3. **Gestión secuencial de memoria y concurrencia adaptativa:** Para evitar la coexistencia simultánea de múltiples redes neuronales en los 16 GB unificados, el flujo `process_video_pipeline` (`src/processing/pipeline.py`) libera de memoria los pesos de Whisper (`unload_whisper_models`) inmediatamente tras alinear la locución y fuerza un semáforo unitario `asyncio.Semaphore(1)` en local, frente a `Semaphore(3)` en la API Cloud. Asimismo, la factoría `src/embeddings/factory.py` mantiene un único modelo local activo en memoria protegido por un `threading.Lock` compartido por identificador de modelo entre distintas instancias dimensionales, invocando `gc.collect()` y `torch.mps.empty_cache()` antes de cualquier conmutación de backend.

4. **Homogeneidad modal e ingeniería de entrada asimétrica:** Todos los fragmentos se construyeron de manera idéntica mediante segmentación SSIM ($\tau = 0,6$, $T_{\text{máx}} = 5,0$ s en `python_`

`tutorial.mp4`; $\tau = 0,5$, $T_{\text{máx}} = 8,0$ s en `animals.mp4`), obteniendo exactamente 82 fragmentos por vídeo con 3 fotogramas clave a ancho estandarizado `target_width = 512` px y transcripción fonética Whisper `base`. Cada motor local adapta la entrada según su protocolo nativo:

- *Qwen3-VL-Embedding-2B:* Emplea diccionarios multimodales directos `{"text": ..., "image": frames}` para cada fragmento (omitiendo la clave textual si no hay transcripción) e inyecta la instrucción asimétrica `"Retrieve video segments relevant to the query."` al vectorizar consultas.
- *EmbeddingGemma 2:* Lee defensivamente las plantillas `Document` (`"title: none | text: "`) y `SearchQuery` (`"task: search result | query: "`) desde `model.prompts` y antepone una etiqueta explícita `<|image|>` por cada fotograma en documentos y consultas multimodales, utilizando `prompt_name="SearchQuery"` en consultas de texto puro y únicamente etiquetas `<|image|>` en consultas visuales sin texto.

5. **Aislamiento de colecciones en Qdrant y evaluación pura de Etapa 1:** Cada configuración modelo-dimensión emplea una colección bajo el esquema `video_chunks__<backend>__<dim>` (manteniendo intacta `video_chunks` para `gemini` en 3072d), todas persistidas en `./data/vector_db`. Para medir sin contaminación externa la geometría intrínseca de cada espacio latente, todas las pruebas se ejecutaron en **Etapa 1 pura (`rerank=False`)** sobre el banco íntegro de 102 consultas literales (56 en Python y 46 en Animals) y 20 consultas multi-objetivo (10 por vídeo, `top_k=10`). Para no consumir cuota API de indexación en *Gemini*, se reutilizaron los 164 vectores `3072-d` ya persistidos y se recuperaron sus tiempos reales de indexación completa desde los registros de auditoría oficiales: 227,31 s (2,772 s/chunk) en `python_tutorial.mp4` (`audit/run_20261109_144503`) y 187,12 s (2,282 s/chunk) en `animals.mp4` (`audit/run_20261109_145448`). Las sub-dimensiones Matryoshka se evaluaron truncando y normalizando en memoria los vectores nativos sin repetir inferencia GPU.

5.6.2 Análisis de Precisión Inter-Dominio (`python_tutorial.mp4` vs. `animals.mp4`)

El Cuadro 17 detalla las métricas de recuperación densa obtenidas por los tres motores a lo largo de ambos corpus y sus respectivas escalas dimensionales MRL, mientras que la Figura 31 contrasta el rendimiento de las configuraciones nativas por dominio.

El análisis conjunto del Cuadro 17 y la Figura 31 evidencia una clara especialización arquitectónica entre dominios:

- **Superioridad Cloud en discurso técnico estructurado (`python_tutorial.mp4`):** *Gemini* (3072d) logra el liderazgo indiscutible en *Recall@1* (85,71 %, +8,92 pp sobre `gemma2` y +10,71 pp sobre `qwen3-vl-2b`) y *MRR* (0,8973). Destaca que `gemma2` (740M parámetros) supera sistemáticamente a `qwen3-vl-2b` (2B) en las cuatro métricas del tutorial técnico (76,79 % vs. 75,00 % en *Recall@1*; 92,86 % vs. 89,29 % en *Recall@3*; 96,43 % vs. 94,64 % en *Recall@5*; y 80,28 % vs. 77,22 % en *Interval Recall*), lo que indica una alineación lingüística superior del tronco textual *Gemma* frente a términos técnicos de programación en español integrados con la transcripción Whisper.
- **Inversión de rendimiento en metraje visual desacoplado (`animals.mp4`):** Cuando el audio narrati-

Tabla 17: Resultados comparativos completos en Etapa 1 (`rerank=False`) por dominio audiovisual, motor de embeddings y dimensión MRL.

Corpus	Backend	Dim (d)	Ahorro	Recall@1	Recall@3	Recall@5	MRR	MT IR (E1)	Qdrant (ms)
	`gemini`	3072	0,0 %	**85,71 %**	92,86 %	94,64 %	0,8973	85,42 %	1,139
	`gemini`	1536	50,0 %	**85,71 %**	92,86 %	96,43 %	0,9009	87,08 %	0,783
	`gemini`	768	75,0 %	**85,71 %**	92,86 %	**98,21 %**	**0,9045**	**87,22 %**	0,672
`python_tutorial.mp4`	`qwen3-vl-2b`	2048	0,0 %	75,00 %	89,29 %	94,64 %	0,8289	77,22 %	1,275
(56 lit. / 10 multi)	`qwen3-vl-2b`	1536	25,0 %	75,00 %	89,29 %	94,64 %	0,8289	79,31 %	1,095
	`qwen3-vl-2b`	768	62,5 %	75,00 %	89,29 %	92,86 %	0,8205	77,22 %	0,882
	`gemma2`	768	0,0 %	76,79 %	92,86 %	96,43 %	0,8512	80,28 %	0,775
	`gemma2`	512	33,3 %	76,79 %	**94,64 %**	96,43 %	0,8488	79,03 %	0,691
	`gemma2`	256	66,7 %	75,00 %	92,86 %	92,86 %	0,8333	74,86 %	**0,627**
	`gemini`	3072	0,0 %	71,74 %	93,48 %	**100,00 %**	0,8341	74,89 %	1,096
	`gemini`	1536	50,0 %	69,57 %	93,48 %	**100,00 %**	0,8232	71,98 %	0,793
	`gemini`	768	75,0 %	69,57 %	**95,65 %**	**100,00 %**	0,8261	74,89 %	0,628
`animals.mp4`	`qwen3-vl-2b`	2048	0,0 %	**78,26 %**	**95,65 %**	97,83 %	**0,8678**	79,05 %	1,376
(46 lit. / 10 multi)	`qwen3-vl-2b`	1536	25,0 %	**78,26 %**	**95,65 %**	97,83 %	**0,8678**	81,31 %	1,215
	`qwen3-vl-2b`	768	62,5 %	**78,26 %**	**95,65 %**	95,65 %	0,8659	76,98 %	6,540*
	`gemma2`	768	0,0 %	71,74 %	93,48 %	97,83 %	0,8239	81,23 %	0,769
	`gemma2`	512	33,3 %	71,74 %	93,48 %	93,48 %	0,8152	**85,15 %**	0,692
	`gemma2`	256	66,7 %	71,74 %	93,48 %	97,83 %	0,8188	**85,15 %**	**0,581**

*Pico transitorio puntual del planificador de memoria durante la prueba en `qwen3-vl-2b@768`; la tendencia estructural HNSW decrece monótonamente con d.

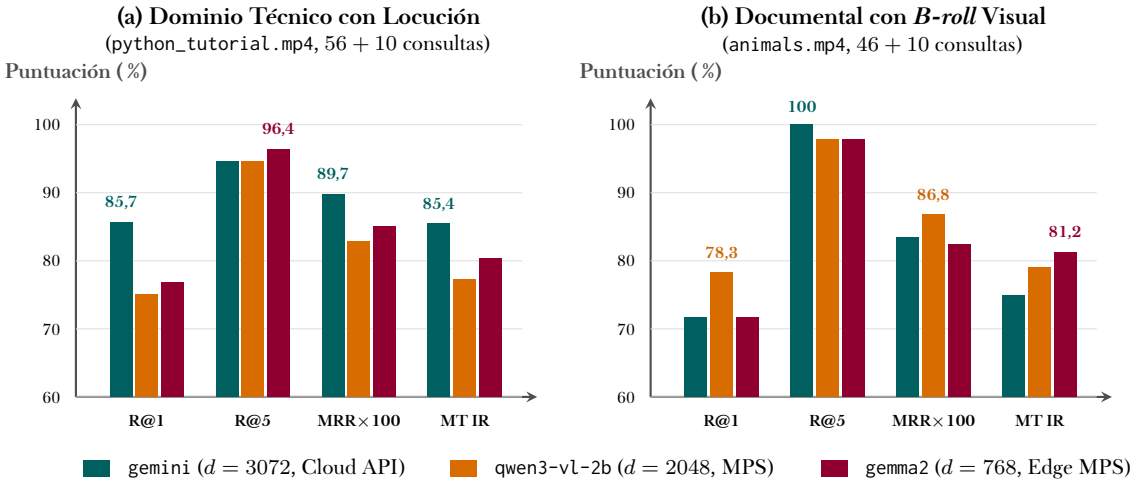


Figura 31: Comparativa de precisión inter-dominio en dimensión nativa (Etapa 1): *Gemini Embedding 2* lidera el tutorial técnico basado en código y locución sincronizada, pero los modelos locales invierten la jerarquía en metraje documental con *B-roll* visual desacoplado.

vo describe categorías amplias mientras el fotograma muestra especies concretas, los codificadores visuales abiertos igualan o baten a la API remota. **`qwen3-vl-2b` obtiene los mejores registros absolutos del documental** en *Recall@1* (**78**, **26** %, superando en $+6,52$ pp a *Gemini* y *Gemma 2*), *Recall@3* ($95,65$ %) y *MRR* ($0,8678$), beneficiándose de su atención nativa sobre teselas visuales M-RoPE de 2B parámetros. Por su parte, **`gemma2` encabeza la recuperación multi-objetivo** (`top_k=10`), elevando el *Interval Recall* nativo al **81**, **23** % (y hasta **85**, **15** % en 512d/256d), más de 10 puntos porcentuales por encima de `gemini@3072` ($74,89$ %).

- **Competitividad global agregada (102 literales + 20 multi-objetivo):** Agregando ambos vídeos, el modelo ligero `gemma2@768` iguala exactamente a `gemini@3072` tanto en *Recall@3* (**93**, **14** %, 95/102) como en *Recall@5* (**97**, **06** %, 99/102), y `gemma2@512` alcanza el mejor *Interval Recall* multi-objetivo global de todo el estudio (**82**, **09** % frente a $81,05$ % de `gemini@768` y $80,31$ % de `qwen3-vl-2b@1536`).

5.6.2.1 Significación Estadística Pareada Inter-Backend (*McNemar* y *Wilcoxon*):

Los contrastes pareados sobre las puntuaciones por consulta confirman estadísticamente los resultados anteriores: en `python_tutorial.mp4` ($N = 56$), la ventaja de `gemini@3072` sobre `qwen3-vl-2b@2048` alcanza significación nominal en *MRR* ($0,8973$ vs. $0,8289$, $p_{\text{Wilcoxon}} = 0,0396$; *Recall@1* $p_{\text{McNemar}} = 0,1094$), mientras que ninguna otra pareja difiere significativamente ni en Python (`gemini@3072` vs. `gemma2@768`: $p_{\text{McNemar}} = 0,1797, p_{\text{Wilcoxon}} = 0,1605$; `qwen3-vl-2b@2048` vs. `gemma2@768`: $p_{\text{McNemar}} = 1,0000, p_{\text{Wilcoxon}} = 0,4864$) ni en `animals.mp4` ($N = 46$, todos los contrastes con $p_{\text{McNemar}} \geq 0,5811$ y $p_{\text{Wilcoxon}} \geq 0,4721$). En el agregado global ($N = 102$), los dos modelos locales logran **paridad estadística plena** frente al modelo comercial en la nube tanto en `qwen3-vl-2b@2048` vs. `gemini@3072` (78/102 vs. 81/102, $p_{\text{McNemar}} = 0,6776, p_{\text{Wilcoxon}} = 0,2915$) como en `gemma2@768` vs. `gemini@3072` (76/102 vs. 81/102, $p_{\text{McNemar}} = 0,3593, p_{\text{Wilcoxon}} = 0,2333$), y en igualdad dimensional entre `gemini@768` y `gemma2@768` (80/102 vs. 76/102, $p_{\text{McNemar}} = 0,4807, p_{\text{Wilcoxon}} = 0,2796$).

5.6.3 Curvas de Ablación Matryoshka (MRL) y Normalización L_2

Al evaluar la clase `SentenceTransformer` mediante el argumento `truncate_dim` sobre Apple Silicon (`bfloat16`), se comprobó un detalle metodológico determinante: **dicho argumento recorta las primeras d coordenadas sin renormalizar el vector resultante a la hiperesfera unitaria**. El Cuadro 18 documenta las normas L_2 observadas tanto en una consulta aislada vía `truncate_dim` como sobre los 164 fragmentos reales de ambos vídeos antes de invocar `truncate_and_normalize_mrl`.

Operar sin renormalizar reduce progresivamente la norma euclídea hasta valores medios de $0,7160$ en `gemma2@256` ($0,7152$ en `truncate_dim`) y $0,6126$ en `qwen3-vl-2b@768` ($0,6284$ en `truncate_dim`), distorsionando cualquier cálculo de producto escalar o umbralización fija y debilitando la estabilidad en cuantizaciones futuras (`int8`). Al delegar el corte en `truncate_and_normalize_mrl`, todos los vectores recuperan estrictamente $\|\hat{\mathbf{v}}\|_2 = 1,0$.

La Figura 32 traza las curvas globales de retención Matryoshka sobre las 102 consultas literales y las 20 multi-objetivo en función del ahorro efectivo de memoria en Qdrant.

El análisis de las trayectorias Matryoshka arroja tres resultados notables:

Tabla 18: Degradación empírica de la norma euclídea (L_2) en prefijos Matryoshka sin renormalizar frente a la restitución unitaria con `truncate_and_normalize_mrl`.

Backend	Dimensión (d)	Reducción	Norma `truncate_dim` (consulta en MPS)	Media $\|\mathbf{v}_{1:d}\|_2$ (164 fragmentos)	Rango [mín, máx] (164 fragmentos)	Norma tras `truncate_and_normalize_mrl`
`gemini`	3072	0,0 %	-	1,0000	[1,0000, 1,0000]	**1,0000**
`gemini`	1536	50,0 %	-	0,7267	[0,7235, 0,7309]	**1,0000**
`gemini`	768	75,0 %	-	0,5612	[0,5552, 0,5655]	**1,0000**
`qwen3-vl-2b`	2048	0,0 %	1,0004	1,0000	[1,0000, 1,0000]	**1,0000**
`qwen3-vl-2b`	1536	25,0 %	0,8659	0,8666	[0,8450, 0,8858]	**1,0000**
`qwen3-vl-2b`	768	62,5 %	**0,6284**	**0,6126**	[0,5849, 0,6398]	**1,0000**
`gemma2`	768	0,0 %	1,0016	1,0000	[1,0000, 1,0000]	**1,0000**
`gemma2`	512	33,3 %	0,8839	0,8785	[0,8684, 0,8854]	**1,0000**
`gemma2`	256	66,7 %	**0,7152**	**0,7160**	[0,7003, 0,7320]	**1,0000**

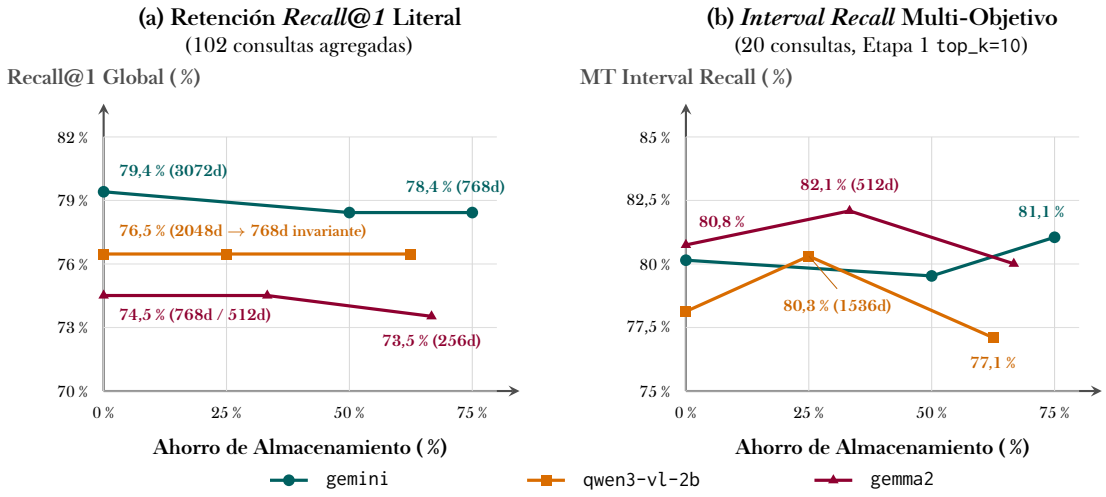


Figura 32: Curvas de ablación Matryoshka (MRL) globales: `qwen3-vl-2b` exhibe invarianza perfecta del 100 % en *Recall@1* (76,5 %) desde 2048d hasta 768d, mientras que `gemma2@512` alcanza el máximo global de *Interval Recall* multi-objetivo (82,1 %) y conserva 73,5 % de *Recall@1* incluso a 256d (1 KB/chunk).

1. **Invarianza en `qwen3-vl-2b`:** Mantiene exactamente $76,47\,\%$ (78/102) de *Recall@1* y $92,16\,\%$ (94/102) de *Recall@3* en 2048d, 1536d y 768d ($-62,5\,\%$ de espacio), además de mejorar el *Interval Recall* multi-objetivo en 1536d ($80,31\,\%$ frente a $78,13\,\%$ en 2048d).

2. **Eficiencia extrema en `gemma2@256`:** Con un tamaño de almacenamiento de tan solo $1,00$ KB/chunk ($82,00$ KB por vídeo frente a $984,00$ KB en `gemini@3072`, una reducción de $12\times$), `gemma2@256` solo cede un acierto en *Recall@1* respecto a 768d (75/102 frente a 76/102) y conserva $80,01\,\%$ de *Interval Recall* multi-objetivo.

3. **Efecto regularizador en sub-dimensiones intermedias:** Igual que `gemini` maximiza *Recall@5* ($99,02\,\%$) y *MT IR* ($81,05\,\%$) en 768d, tanto `qwen3-vl-2b@1536` como `gemma2@512` logran sus picos multi-objetivo ($80,31\,\%$ y $82,09\,\%$) en dimensión reducida, confirmando que descartar las dimensiones finales de baja varianza mitiga sobreajustes locales en búsquedas de múltiple objetivo.

Desde el punto de vista inferencial ($N = 102$), la compresión desde la dimensión nativa hasta la mínima evaluada en cada motor no produce pérdida estadísticamente significativa ni en *Recall@1* ni en *MRR*: `gemini` (3072d $\rightarrow$ 768d: $p_{\text{McNemar}} = 1,0000, p_{\text{Wilcoxon}} = 0,6729$), `qwen3-vl-2b` (2048d $\rightarrow$ 768d: $p_{\text{McNemar}} = 1,0000, p_{\text{Wilcoxon}} = 0,5519$) y `gemma2` (768d $\rightarrow$ 256d: $p_{\text{McNemar}} = 1,0000, p_{\text{Wilcoxon}} = 0,4370$).

5.6.4 Rendimiento Computacional, Latencia y Huella de Memoria

El Cuadro 19 consolida los costes de indexación, tiempos de respuesta en consulta y consumo máximo de memoria RAM (`RSS`) y gráfica integrada (`MPS driver_allocated_memory`), complementado visualmente por la Figura 33.

Tabla 19: Rendimiento computacional, latencia de consulta y huella máxima de memoria por vídeo y backend (configuraciones nativas).

Corpus	Backend (Dim)	Dispositivo	Indexación Total	Coste / Fragmento	Consulta Media (ms)	Consulta Mediana (ms)	Pico RSS (MB)	Pico MPS (MB)
`python_tutorial.mp4`	`gemini` (3072)	Cloud API	227,31 s	2,772 s	484,67	481,30	391,4	0
	`qwen3-vl-2b` (2048)	Local MPS	**142,23** s	**1,734** s	95,11	97,69	2,095,1	4,736,5
	`gemma2` (768)	Local MPS	247,73 s	3,021 s	**30,11**	**29,93**	3,081,5	**2,552,6**
`animals.mp4`	`gemini` (3072)	Cloud API	187,12 s	2,282 s	1,086,72*	464,18	424,0	0
	`qwen3-vl-2b` (2048)	Local MPS	**126,05** s	**1,537** s	92,33	90,94	1,995,8	4,737,7
	`gemma2` (768)	Local MPS	245,49 s	2,994 s	**31,08**	**30,20**	2,641,2	**2,592,1**

*Incluye picos puntuales de latencia TLS/API remota; la mediana (464,18 ms) representa el tiempo típico de respuesta en red.

La comparativa temporal y de recursos revela tres hallazgos clave de ingeniería:

1. **Aceleración interactiva de hasta $16\times$ en consultas locales:** Al eliminar el viaje de red HTTP/TLS hasta los servidores de la API Cloud (mediana de 464,18–481,30 ms), `qwen3-vl-2b` responde en 90,94–97,69 ms ($\approx 5\times$ más rápido) y `gemma2` en apenas **29**, **93–30**, **20** ms ($15,4\times$–$16,1\times$ más rápido). Esto permite ejecutar búsquedas vectoriales en Qdrant en tiempo casi instantáneo de percepción humana (< 35 ms totales).

2. **Asimetría arquitectónica entre consulta textual e indexación de fotogramas:** Aunque `gemma2` es $3\times$ más rápido en consultas de texto (23,6 ms frente a 72,9 ms medidos en aislamiento sobre

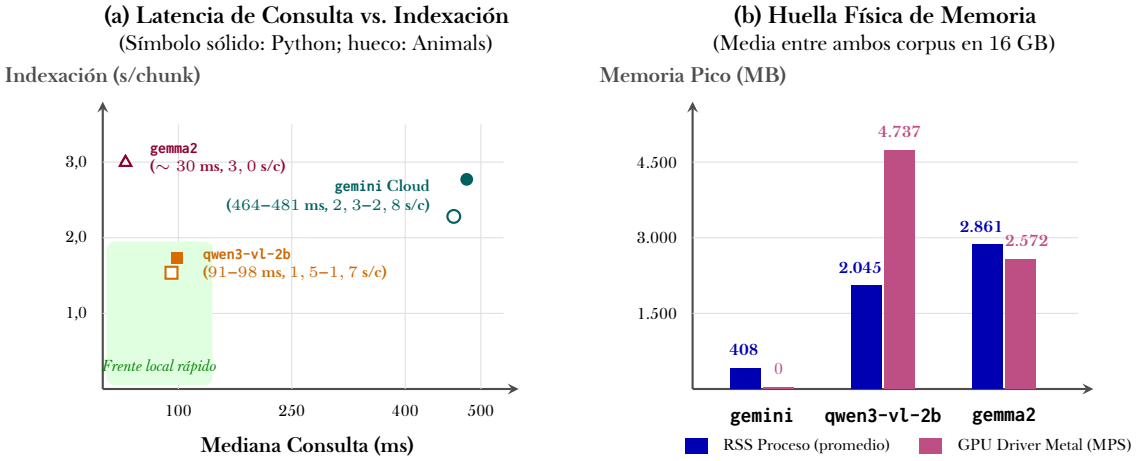


Figura 33: Compromiso entre rendimiento temporal y recursos de memoria en Apple Silicon M3 (16 GB): `gemma2` reduce la latencia de consulta local en un factor de 15–16× frente a Cloud API consumiendo casi la mitad de memoria MPS que `qwen3-vl-2b`, mientras que `qwen3-vl-2b` consigue la indexación de vídeo más veloz.

MPS tras `torch.mps.synchronize()`), porque en texto puro activa únicamente su tronco lingüístico de 270M parámetros, al indexar fragmentos multimodales de 3 fotogramas (512 px) invoca su codificador visual (170M) con un coste neto de 1,960, 1 ms/chunk frente a 1,180, 6 ms/chunk en `qwen3-vl-2b`. Sumando las etapas constantes de segmentación SSIM y transcripción Whisper, **`qwen3-vl-2b` completa la indexación de un vídeo en** 126, 05–142, 23 **s**, superando incluso a *Gemini Cloud* (187, 12–227, 31 s).

3. **Contención en** 16 **GB unificados:** Al combinar la descarga de Whisper con `Semaphore(1)`, ningún motor local bloquea el equipo: `gemma2` opera con apenas **2, 55–2, 59** GB **de memoria GPU Metal** (46 % menos que `qwen3-vl-2b`, con 4, 74 GB), posicionándose como la alternativa óptima para portátiles o despliegue en el borde con restricciones estrictas de VRAM compartida.

5.6.5 Criterios de Selección Arquitectónica Cloud vs. Local

A la luz de la evidencia recopilada en ambos vídeos, no existe un ganador absoluto en todas las dimensiones operativas, sino un espectro complementario claro:

- **Escenarios sin GPU o de discurso técnico denso:** `gemini@768` mantiene el mejor equilibrio global de precisión en Etapa 1 en contenido hablado/código (85, 71 % *Recall@1* en Python) con huella local prácticamente nula (391 − 424 MB RSS), a costa de 464 − 481 ms de red y sujeción a cuotas RPD.
- **Búsqueda visual en metraje documental o alta velocidad de ingesta privada:** `qwen3-vl-2b@1536` ofrece la mejor combinación de precisión puramente visual (78, 26 % *Recall@1* y 81, 31 % *MT IR* en `animals.mp4`) y el menor tiempo de indexación (1, 54 − 1, 73 s/chunk) sin cuotas externas, asumiendo ≈ 4, 74 GB de memoria Metal.
- **Computación Edge, baja latencia y mínimo almacenamiento:** Las variantes `gemma2@512` y `gemma2@256` ejecutan consultas locales en ≈ 30 ms consumiendo solo 2, 55 − 2, 59 GB de MPS, lide-

ran el *Interval Recall* multi-objetivo global ($82,09\,\%$ en 512d) y reducen la base vectorial hasta $1,00$ KB/chunk (82 KB por vídeo).

6 Conclusiones y Trabajo Futuro

Esta sección cierra formalmente la memoria del Trabajo de Fin de Máster. En primer lugar, se contrastan de manera definitiva las preguntas de investigación e hipótesis científicas (H_1, H_2 y H_3) contra la evidencia empírica cuantitativa y se evalúa el cumplimiento de los cinco objetivos específicos del proyecto. A continuación, se sintetizan las contribuciones científicas y metodológicas aportadas al estado del arte en recuperación multimodal de vídeo. Posteriormente, se exponen con transparencia las limitaciones técnicas y operativas del estudio. Por último, se perfilan las líneas de investigación futuras más prometedoras para la evolución de los sistemas *Video-RAG*.

6.1 Validación Formal de Hipótesis y Cumplimiento de Objetivos

La culminación de este Trabajo de Fin de Máster de investigación permite confrontar las tres hipótesis científicas formuladas en la Sección 1 con la evidencia empírica acumulada a lo largo de las 138 consultas experimentales:

6.1.1 Validación de las Hipótesis de Investigación

Hipótesis 1 (H_1): Fusión Temprana y Compensación Asimétrica: *Confirmada.* La inyección conjunta de tres fotogramas clave y la transcripción fonética Whisper en un único contenedor `types.Content` incrementa significativamente el *Recall@1* en $+17,8$ puntos porcentuales ($67,9\,\% \rightarrow 85,7\,\%$, $p_{\text{McNemar}} = 0,0020$) y el *MRR* en $+0,118$ ($p_{\text{Wilcoxon}} = 0,0025$) en el corpus técnico con locución sincronizada (`python_tutorial.mp4`). En metraje de naturaleza con *B-roll* desacoplado (`animals.mp4`), aunque la Etapa 1 experimenta una desviación direccional no significativa ($p_{\text{McNemar}} = 0,1460$), el *Recall@5* alcanza el $100,0\,\%$ (46/46) y la verificación bimodal de Etapa 2 subsana el desfase ($p_{\text{McNemar}} = 0,0020$; $p_{\text{Wilcoxon}} = 0,0040$; $p_{\text{McNemar}} = 0,0018$ global), elevando el *Recall@1* al $93,5\,\%$ (43/46) y el *Recall@5* al $97,8\,\%$, superando a la línea base puramente visual en ambos dominios.

Hipótesis 2 (H_2): Compresión Dimensional y Eficiencia con MRL: *Confirmada.* El truncamiento y renormalización L_2 mediante `mrl.py` (*Matryoshka Representation Learning* [18]) de 3072 a 768 dimensiones en *Gemini Embedding 2* retiene el $98,77\,\%$ **del *Recall@1* relativo** (80/102 frente a 81/102, $p_{\text{McNemar}} = 1,0000$), preserva el *MRR* ($0,8691$ frente a $0,8688$, $p_{\text{Wilcoxon}} = 0,6729$) e incrementa el *Recall@5* del $97,06\,\%$ al $99,02\,\%$, reduciendo memoria un $75,0\,\%$ (12 KB $\rightarrow$ 3 KB) y latencia en Qdrant un $44,4\,\%$ ($1,14$ ms $\rightarrow 0,63$ ms). De manera análoga, *Qwen3-VL-2B* (2048 $\rightarrow$ 768d) y *EmbeddingGemma 2* (768 $\rightarrow$ 256d) mantienen invarianza estadística completa ($p_{\text{Wilcoxon}} \geq 0,4370$) alcanzando este último paridad estadística global frente al servicio Cloud ($p_{\text{McNemar}} \geq 0,3593$).

Hipótesis 3 (H_3): Erradicación de Falsos Positivos y Precisión en Consultas Complejas: *Confirmada.* El árbitro bimodal multimodal (*Gemini 3.5 Flash Lite*) erradica los falsos positivos derivados de la similitud del coseno en la primera etapa, alcanzando una **tasa de rechazo**

del $100,0\,\%$ (0/240 falsos positivos admitidos en las 16 consultas negativas de control a lo largo de las tres dimensiones evaluadas). Asimismo, en consultas generales multi-objetivo duplica la precisión en el metraje documental, pasando del $50,00\,\%$ al $96,57\,\%$ y situando un acierto verdadero en la primera posición en el $100,0\,\%$ **de los casos** (10/10).

6.1.2 Cumplimiento de los Objetivos Específicos

El trabajo ha satisfecho de forma rigurosa los cinco objetivos específicos definidos:

- **OE1 (Estado del Arte):** Analizado en la Sección 2, abarcando fundamentos matemáticos (incluida la formulación InfoNCE y el fenómeno de *Modality Gap*), taxonomía de modelos omnimodales y locales, sistemas *Video-RAG* y métricas de evaluación.
- **OE2 (Metodología y Arquitectura):** Formalizado en la Sección 3, estableciendo la segmentación por SSIM, el muestreo de fotogramas clave por máxima entropía de Shannon, la alineación forzada de voz, la proyección MRL parametrizada por backend y la recuperación en dos etapas.
- **OE3 (Prototipo Experimental):** Implementado en la Sección 4, construyendo un sistema desacoplado sobre el protocolo común `MultimodalVectorizer` (`base.py`), `mrl.py`, `local_vectorizer.py` (`bfloat16` sobre Metal MPS) y `factory.py`, integrando *Gemini Embedding 2*, *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2* sobre colecciones Qdrant aisladas, verificador bimodal y arnés de trazabilidad `RunAuditor`.
- **OE4 (Evaluación Empírica):** Ejecutado en la Sección 5 mediante tres baterías de prueba exhaustivas (138 consultas en 164 fragmentos audiovisuales), contrastando tanto la ablación audio-visual y MRL de Gemini como la comparativa sistemática entre los tres motores en sus respectivas escalas dimensionales (3072/1536/768, 2048/1536/768 y 768/512/256) con pruebas de significación pareada.
- **OE5 (Viabilidad Computacional y Costes):** Evaluado en la Subsección 5.5, cuantificando latencias granulares por etapa y proyectando una huella de memoria de solo $2,86$ GB para un millón de fragmentos.

6.2 Contribuciones Científicas y Metodológicas al Estado del Arte

Esta investigación aporta cinco contribuciones fundamentales al área de la Recuperación de Información Multimedia y los sistemas *Video-RAG*:

1. **Paradigma de Fusión Temprana Audio-Visual con Invarianza de Cuota:** Se demuestra que inyectar simultáneamente fotogramas clave y locución fonética continua alineada dentro de un único contenedor o entrada multimodal no solo preserva la riqueza perceptual cruzada ($p_{\text{McNemar}} = 0,0020$ en tutorial técnico), sino que mantiene estrictamente invariable el coste de vectorización (1 fragmento = 1 vector).

2. **Evidencia Empírica del Comportamiento MRL Multicriterio sobre Vídeo:** Se aporta un estudio sistemático de ablación dimensional mediante `mrl.py` en tres motores complementarios: *Gemini Embedding 2* $(3072, 1536, 768)$, *Qwen3-VL-Embedding-2B* $(2048, 1536, 768)$ y *EmbeddingGemma 2* $(768, 512, 256)$. En el modelo fundacional remoto se demuestra que truncar a $d = 768$ $(-75\,\%)$ actúa como regularizador semántico, elevando el *Recall@5* al $99,02\,\%$ y reduciendo la latencia en Qdrant un $44,4\,\%$ sin degradación estadística ($p_{\text{Wilcoxon}} = 0,6729$).

3. **Mecanismo de Verificación Bimodal Desacoplado para Erradicación de Falsos Positivos:** Se formaliza una arquitectura en dos etapas donde la primera etapa maximiza la exhaustividad (*Recall@5* $\geq 97\,\%$) y la segunda etapa aplica un razonamiento multimodal disjunto e inclusivo ($p_{\text{McNemar}} = 0,0018$ global en Etapa $1 \rightarrow 2$), logrando una tasa de rechazo del $100\,\%$ en consultas negativas y duplicando la precisión en consultas generales complejas.

4. **Metodología de Auditoría Experimental Reproducible:** Se publica un banco de pruebas anotado de 138 consultas sobre dominios contrastados, junto con un arnés de instrumentación (`RunAuditor`) que permite auditar y re-calcular deterministamente métricas de Information Retrieval y pruebas pareadas sin depender de APIs externas.

5. **Evaluación Empírica Multi-Backend y Desacoplamiento Hardware-Agnóstico:** Se diseña e implementa el protocolo común `MultimodalVectorizer` (`base.py`), la factoría con descarga secuencial de residentes en RAM (`factory.py`) y el adaptador `LocalMultimodalVectorizer` (`local_vectorizer.py` en `bfloat16` sobre Metal MPS), demostrando sobre $N = 102$ consultas que modelos abiertos de 740M–2B parámetros (*EmbeddingGemma 2* y *Qwen3-VL-2B*) alcanzan paridad estadística global frente al servicio comercial Cloud ($p_{\text{McNemar}} \geq 0,3593$), superándolo en metraje documental desacoplado y habilitando búsquedas en el borde sin coste de API.

6.3 Limitaciones del Estudio y Restricciones Técnicas

A pesar del rigor metodológico aplicado, el presente trabajo presenta limitaciones técnicas que deben ser consideradas:

- **Dependencia de API Propietaria en la Etapa 2 de Reordenación:** Aunque la Etapa 1 de recuperación densa soporta ya ejecución íntegramente local mediante *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2* (Subsección 5.6), el verificador bimodal de segunda etapa continúa dependiendo del modelo remoto *Gemini 3.5 Flash Lite*. Aunque se implementaron reintentos exponenciales y cortafuegos de cuota (1,000 RPD), un sistema 100 % desconectado de Internet requiere incorporar también un reordenador local.
- **Precisión Fonética del Modelo Whisper `base`:** Para hacer viable la ejecución local en tiempo real sobre CPU, se seleccionó el modelo Whisper en su variante `base`. Aunque demostró un rendimiento notable, ocasionalmente comete errores de transcripción en palabras técnicas complejas o modismos coloquiales muy específicos, los cuales se propagan al texto inyectado en el vectorizador.
- **Sensibilidad del Algoritmo SSIM ante Cambios Graduales:** La detección de cortes basada exclusivamente en SSIM es idónea para transiciones de plano netas, pero muestra menor sensibilidad ante transiciones lentas por disolución encadenada (*cross-dissolve*) o movimientos panorámicos continuos de cámara, dependiendo en esos casos de la cota máxima temporal ($T_{\text{máx}} = 8,0$ s por defecto y $5,0$ s en el banco de código estático).
- **Tamaño del Corpus de Validación:** Si bien el banco de pruebas (164 fragmentos, 138 consultas) fue anotado minuciosamente a mano para garantizar un *ground truth* de máxima calidad, la escala total del metraje (aproximadamente 8,1 minutos) representa una prueba de concepto que debe validarse a futuro sobre conjuntos de datos públicos masivos de cientos de horas de duración (como *Video-MME* o *LongVideoBench*) [10, 22].

6.4 Líneas de Investigación Futura

A partir de los hallazgos y limitaciones identificadas, se proponen las siguientes líneas de trabajo para la comunidad investigadora:

1. **Canalizaciones 100 % Locales de Extremo a Extremo (*Embeddings* + *Reranker*):** Tras validar en local la Etapa 1 con *Qwen3-VL-Embedding-2B* y *EmbeddingGemma 2*, el siguiente paso natural es sustituir también el juez bimodal remoto de la Etapa 2 por reordenadores abiertos locales como *Qwen3-VL-Reranker* [6], completando un flujo totalmente desconectado (*offline*) con privacidad integral de datos en entornos empresariales.

2. **Video-RAG en Tiempo Real sobre Flujos Continuos (*Streaming*):** Adaptar los algoritmos de segmentación SSIM y alineación Whisper a ventanas deslizantes de memoria en tiempo real, permitiendo indexar y consultar transmisiones de vídeo en vivo (conferencias, emisiones deportivas o vigilancia de seguridad) con latencias de respuesta inferiores a un segundo.

3. **Ponderación Modal Aprendida (*Attention-Guided Fusion*):** Sustituir la concatenación estática de partes visuales y textuales por un mecanismo de atención cruzada que pondere dinámicamente el peso relativo de la imagen frente a la voz en función de la correlación semántica detectada en cada plano.

4. **Cuantización Extrema sobre Representaciones Matryoshka:** Combinar el truncamiento MRL con técnicas de cuantización escalar a 8 bits (`int8`) o binarización vectorial a 1 bit, proyectando reducciones adicionales de memoria de hasta 32×, lo que posibilitaría mantener miles de horas de vídeo indexadas en dispositivos móviles o de computación en el borde (*edge computing*).

Referencias

[1] Prajwalbm, "Multimodal embeddings for multimodal rag." https://medium.com/@prajwalbm23/multimodal-embeddings-for-multimodal-rag-ec481ad20571, 2025.

[2] Google AI for Developers, "Modelo de gemini embedding 2." https://ai.google.dev/gemini-api/docs/models/gemini-embedding-2?hl=es-419, 2026.

[3] LearnOpenCV, "Qwen3-vl embedding and reranker for multimodal search." https://learnopencv.com/how-to-master-qwen3-vl-embedding-and-reranker-for-multimodal-search/, 2026.

[4] A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark, G. Krueger, and I. Sutskever, "Learning transferable visual models from natural language supervision," in *Proceedings of the 38th International Conference on Machine Learning (ICML)*, vol. 139 of *Proceedings of Machine Learning Research*, pp. 8748–8763, PMLR, 2021.

[5] A. Radford, J. W. Kim, T. Xu, G. Brockman, C. McLeavey, and I. Sutskever, "Robust speech recognition via large-scale weak supervision," in *Proceedings of the 40th International Conference on Machine Learning (ICML)*, vol. 202 of *Proceedings of Machine Learning Research*, pp. 28492–28518, PMLR, 2023.

[6] Qwen Team, "Qwen3-vl-embedding and qwen3-vl-reranker," *arXiv preprint arXiv:2601.04720*, 2026.

[7] PrismaticLab, "Video-rac: Retrieval adaptive chunking for lecture video rag." https://prismaticlab.github.io/Video-RAC/, 2025.

[8] M. Mao, M. M. Perez-Cabarcas, U. Kallakuri, N. R. Waytowich, X. Lin, and T. Mohsenin, "Multi-rag: A multimodal retrieval-augmented generation system for adaptive video understanding," *arXiv preprint arXiv:2505.23990*, 2025.

[9] Hugging Face Team, "Training and finetuning multimodal embedding & reranker models." https://huggingface.co/blog/train-multimodal-sentence-transformers, 2026.

[10] Y. Luo, X. Zheng, G. Li, S. Yin, H. Lin, C. Fu, J. Huang, J. Ji, F. Chao, J. Luo, and R. Ji, "Video-rag: Visually-aligned retrieval-augmented long video comprehension," *arXiv preprint arXiv:2411.13093*, 2024.

[11] Mixpeek, "Multimodal chunking strategies: How to decompose video, audio." https://mixpeek.com/guides/multimodal-chunking-strategies, 2025.

[12] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE Transactions on Image Processing*, vol. 13, no. 4, pp. 600–612, 2004.

[13] 3manifold, "whisperx-silero-vad." https://github.com/3manifold/whisperX-silero-vad, 2025.

[14] S. S. Rajadh and Contributors, "Sentrysearch: Semantic search over video footage using multimodal embeddings." https://github.com/ssrajadh/sentrysearch, 2026. Repositorio oficial de código abierto (Apache-2.0 License).

[15] A. Martin *et al.*, "Seeing through the mirage: Evaluating multimodal retrieval augmented generation," *arXiv preprint arXiv:2510.24870*, 2025.

[16] Easton, "Notebooklm research workflow: Literature reviews at scale." https://eastondev.com/blog/en/posts/ai/20260227-notebooklm-research-guide/, 2026.

[17] Sourclip, "Notebooklm research workflows: From sources to synthesis." https://www.sourclip.com/blog/notebooklm-research-workflows, 2026.

[18] A. Kusupati, G. Bhatt, S. Anagnostidis, S. S. Keerthi, P. Jain, and S. Kakade, "Matryoshka representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 30233–30249, 2022.

[19] A. van den Oord, Y. Li, and O. Vinyals, "Representation learning with contrastive predictive coding," *arXiv preprint arXiv:1807.03748*, 2018.

[20] V. W. Liang, Y. Zhang, Y. Kwon, S. Yeung, and J. Z. Zou, "Mind the gap: Understanding the modality gap in multi-modal contrastive representation learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, pp. 17612–17625, 2022.

[21] R. Girdhar, A. El-Nouby, Z. Liu, M. Singh, K. V. Alwala, A. Joulin, and I. Misra, "ImageBind: One embedding space to bind them all," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 15180–15190, 2023.

[22] Y. Lee, J. Shin, N. H.-Y. Kim, J. Bang, J. Lee, K. Hwang, F. Porikli, and H. Song, "Rethinking rag in long videos: What to retrieve and how to use it?," *arXiv preprint arXiv:2606.13141*, 2026.

[23] Google AI for Developers, "Embeddings | gemini api documentation." https://ai.google.dev/gemini-api/docs/embeddings?hl=es-419, 2026.

[24] OpenRouter, "Gemini embedding 2 - api pricing & providers." https://openrouter.ai/google/gemini-embedding-2, 2026.

[25] Qwen Team, "Qwen3-vl-embedding: Advancing multimodal retrieval and reranking." https://qwen.ai/blog?id=qwen3-vl-embedding, 2026.

[26] Qwen Team, "Qwenlm/qwen3-vl-embedding: Official repository for multimodal embedding and reranker series." https://github.com/QwenLM/Qwen3-VL-Embedding, 2026.

[27] Qwen Team, "Qwen/qwen3-vl-embedding-2b model card." https://huggingface.co/Qwen/Qwen3-VL-Embedding-2B, 2026.

[28] Qwen Team, "Qwen/qwen3-vl-embedding-8b model card." https://huggingface.co/Qwen/Qwen3-VL-Embedding-8B, 2026.

[29] T. Aarsen, "tomaarsen/qwen3-vl-embedding-2b-vdr." https://huggingface.co/tomaarsen/Qwen3-VL-Embedding-2B-vdr, 2026.

[30] P. R. de Moura Júnior, J. Lelong, and A. Blangero, "Adaptive chunking: Optimizing chunking-method selection for rag," *arXiv preprint arXiv:2603.25333*, 2026.

[31] Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.

[32] Google DeepMind, "google/embeddinggemma-2 model card." `https://huggingface.co/google/embeddinggemma-2`, 2026. Modelo abierto multimodal (740M parámetros, MRL 768/512/256/128, licencia Apache-2.0).